POC-001 — Gate M1 Design-Readiness Specification v1.13 Final (Normative Contract Proposal)

Canonical Domain Types, Evidence Allowlist, Enforceable Verification Claims, Currency Registry & Deterministic Serialization Contracts

Document Status: AMENDED FINAL NORMATIVE CONTRACT (Incorporating Owner Audit Closures AUDIT-01 through AUDIT-52) — awaiting Project Owner Design Freeze
Date: 2026-10-11
Version: v1.13 Final (Normative Contract incorporating AUDIT-01 through AUDIT-52 closures)
Target Repository: `D:\Market HFT OS`
Git Baseline Commit: `582637b` (Gate M0 Governance Baseline & Charter v0.2.4 on `main`)
Specification File: `file:///C:/Users/spaye/.gemini/antigravity/brain/f8d383dc-74a5-44e7-8222-da945383593a/POC-001_Gate_M1_Design_Readiness_Specification.md`
Author: AI Technical Lead & Architecture Team
Reviewer: Project Owner / Principal Capital Allocator

---

1. Executive Summary & Core Design Invariants

This specification defines the normative design contract for **POC-001 Gate M1**, governing the transition from foundational governance (Gate M0) to active implementation. Gate M1 establishes the immutable type definitions, cryptographic claim structures, currency registries, and deterministic serialization contracts required to support high-frequency decisioning, capital allocation, and algorithmic market simulation without floating-point drift, untyped state mutations, or unverified supplier data.

Core Architecture Invariants (Audited & Normative):

1. **Quarantine Discipline**: Legacy assets in `_salvage/AIStudio` (commit `0c67e26`) and `_salvage/external/` are strictly read-only reference repositories. Zero legacy code is directly imported or executed.
2. **Zero Implementation Prior to Freeze**: Gate M1 remains strictly a specification contract. Package initialization (`npm init`), dependency installation, and application development remain completely blocked until this specification is explicitly frozen by the Project Owner.
3. **Four-Claim Promotion Gate**: An offer cannot attain `VERIFIED_EXECUTABLE_OFFER` merely by asserting a tuple. It must pass an explicit normative gate requiring four distinct, fresh, and corroborated evidentiary claims (`SOURCE_AUTHENTICITY`, `PRICE_CONFIRMATION`, `STOCK_AVAILABILITY`, `COMMERCIAL_TERMS`) bound to the specific offer and product identity.
4. **Exhaustive Input-Shape Validation & Field Reconciliation**: The promotion gate executes fail-closed structural validation on assessment shapes before field dereferencing (`E-ERR-024`), followed by exact field-by-field equality checks comparing the offer's displayed terms against authenticated claims (normalized unit price, MOQ, payment terms, Incoterms).
5. **Freshness from Authoritative Source Time**: Evidence freshness is calculated strictly from authoritative source observation time (`observed_at_utc`) or authenticated feed issuance time (`occurred_at_raw`), never platform ingestion time (`recorded_at_utc`). Historical imports (`is_historical_import: true`) are structurally barred from executable offers without real-time authenticated reconfirmation. Commercial terms evidence observation age cannot exceed 7 days.
6. **Unified Atomic Promotion, Co-located Storage & Comprehensive Evaluator Authentication (AUDIT-35, AUDIT-36, AUDIT-37, AUDIT-41, AUDIT-42, AUDIT-43, AUDIT-46, AUDIT-47)**: Promotion evaluation cleanly separates read-only evaluation from atomic persistence. Phase 1 validates all four claims, signatures, authorities, and commercial reconciliation without consuming nonces, and generates an RFC 8785 canonical `EvaluatorDecisionSigningPayload` HMAC (`draft.evaluator_auth_token`) under domain tag `POC-001/EVALUATOR_DECISION_AUTH/v1` covering the entire decision state (assigned statuses, rejection reason, evaluator version, full claims summary, nonces). The evaluator key resides strictly within the trusted service boundary, verified via an injected `EvaluatorTokenVerifier` using constant-time byte comparison (`crypto.timingSafeEqual`). Phase 2 executes a single atomic persistence transaction `commitPromotionIfVersionAndConsumeNonces()` under PostgreSQL `READ COMMITTED` isolation with write-intent row locking (`SELECT ... FOR UPDATE` on `offer_stream_heads`). Inside the transaction after row-lock acquisition, trusted time is sampled at the Post-Lock Authorization Instant ($T_{\text{auth}} = \text{clock.nowUtc()}$, `tx_auth_time_utc`), receipt expiration ($T_{\text{auth}} \le T_{\text{expires}}$), gateway active keys, attester authority/revocation, and claim payload digests are revalidated (AUDIT-47, AUDIT-51), nonces are inserted with gateway IDs in `offer_nonce_store`, canonical assessment bytes are stored in `offer_assessment_store`, sequence CAS is evaluated via BigInt numeric comparisons on canonical decimal strings, the stream head sequence and status are updated via `updateOfferStreamHead()`, and the promotion event is appended. All four database mutations commit atomically within the single transaction; if any validation fails prior to commit, the entire transaction rolls back, burning zero nonces. If connection drops while COMMIT is in flight, pessimistic row-lock reconciliation resolves the outcome authoritatively before retry.
7. **Two-Tier Gross-Clearing Accounting Model & Cumulative Operating Flow Series (AUDIT-34, AUDIT-38, AUDIT-44, AUDIT-48)**: Operating cash flows enforce a Two-Tier Gross-Clearing Accounting Model. Tier 1 (Trade Clearing Asset B_{clearing}) recognizes gross customer checkout proceeds (P_{gross}) as operating inflows at point of sale, with itemized marketplace commissions, payment processor fees, fulfillment deductions, output taxes withheld, and customer refunds/chargebacks recorded as discrete operating deductions against the clearing asset. Tier 2 (Treasury Operating Bank Account) governs deployable liquidity. Crucially, Available Bank Liquidity ($L_{\text{bank}}(t) = B_0 + \sum F_j + \sum C_j^{\text{bank_ops}}$) is strictly separated from the Cumulative Operating Cash Flow Series ($B_{\text{ops}}(k) = B_{\text{ops}}(0) + \sum_{j=1}^k C_j^{\text{ops}}$ with pre-event baseline $B_{\text{ops}}(0) = 0$ at position inception t_0, and operating transactions $j \in \{1, \dots, K\}$). Peak Operating Funding Requirement ($C_{\text{peak, ops}} = \max(0, \max_{k \le K}(-\sum_{j=1}^k C_j^{\text{ops}}))$) measures the maximum deficit generated by the operating cycle itself, invariant to starting equity B_0 or owner financing injections F_j. Net merchant bank settlement payouts (P_{net}) are recorded strictly as internal Liquidity Transfers from clearing to bank balances ($\Delta B_{\text{clearing}} = -P_{\text{net}}$, $\Delta L_{\text{bank}} = +P_{\text{net}}$), never as secondary revenue inflows (`E-ERR-038: CASH_FLOW_DOUBLE_COUNTING_PROHIBITED`). Marked-to-Market Return on Peak Funding (RoPF_{mtm}) evaluates cumulative operating flows and verified inventory valuation divided by $C_{\text{peak, ops}}$.
8. **Disambiguated Funding Metrics**: The financial calculation layer formally separates Intraday Peak Operating Funding ($C_{\text{peak, ops}}$) evaluated across sequentially ordered operating transactions `(entry_timestamp_utc, sequence_number)` from Net Daily Operating Funding ($C_{\text{peak, daily_net_ops}}$) evaluated at midnight UTC calendar boundaries. Intraday capital deployed in the morning is strictly accounted for even if reimbursed before end-of-day.
9. **Dual Disambiguated Capital Recovery Timelines**: Formal separation between `Capital Cash Recovery Duration (from Outlay)` (integer days from first net negative operating cash balance $B_{\text{ops}} < 0$ to operating cash breakeven $B_{\text{ops}} \ge 0$ via net disbursements) and `Position Lifecycle Recovery Duration` (integer days from position inception T_0 to operating breakeven).
10. **Distinct Return Metrics**: Realized Operating Cash Multiple (Cash-on-Cash multiple of operating inflows over outflows, excluding unsold inventory) is strictly distinguished from Marked-to-Market Return on Peak Funding (RoPF_{mtm}, evaluating net cumulative operating cash flows plus verified inventory

valuation over peak operating funding). Unverified ending inventory emits `DATA_REQUIRED` and is never coerced to zero.

11. **Deterministic Product Identity Normalization**: Product identity cores undergo Unicode Normalization Form C (`NFC`), control-character sanitization, HSN tariff format/length validation (4, 6, or 8 digits), and critical dimension key deduplication. Manufacturer part number character casing, hyphens, and punctuation are strictly preserved unless the registered manufacturer namespace explicitly authorizes case-folding. HSN format validation confirms syntax and length; it does not establish legal assignment or applicability under the authoritative HSN/CBIC tariff schedule effective on the relevant date. Inventory and position quantities enforce non-negative safe integers ($0 \le q \le 10^9$).
12. **Fixed-Precision Decoupled Currency Registry**: All monetary quantities are represented as exact signed integer subunits with currency-specific scale factors decoupled into ISO minor units, internal accounting scale (4 decimals for INR/USD to eliminate intermediate drift), and external settlement scale. Floating-point numbers are prohibited in financial state.
13. **Point-of-Use FX Conversion Receipts**: Cross-currency summations require explicit, authenticated, and unexpired `FXConversionReceipt` instances validated at point-of-use (`E-ERR-008`). All conversions use BigInt rational integer arithmetic with Stage 1 intermediate sub-subunit preservation and Banker's rounding.
14. **Dual-Ledger Cryptographic Write Isolation**: Absolute separation between paper simulation and actuals execution. Direct writes to the actuals ledger are blocked fail-closed at runtime unless authorized by an authenticated execution context bearing an Ed25519-signed `ActualsAuthTicketEnvelope` binding the exact RFC 8785 canonical hash of the mutation (`E-ERR-020` , `E-ERR-025` , `E-ERR-026`).
15. **Auditable Hash-Chained Stream History**: Every state mutation appends an immutable event to a hash-chained stream ($H_n = \text{SHA-256}(H_{n-1} \parallel \text{canonicalizeJCS}(e_n))$), anchored by a virtual genesis event (H_{-1}). Periodically, streams are sealed via Ed25519-signed checkpoint receipts verifying hash continuity and active authority status.
16. **Strict Production RFC 8785 Canonicalizer Contract & Disambiguated Package Provenance (AUDIT-22, AUDIT-32, AUDIT-39, AUDIT-45)**: Runtime canonicalization pins exact NPM registry metadata: `json-canonicalize` v3.0.1 (`https://github.com/snowyu/json-canonicalize.ts` , integrity `sha512-xEzkTucfHhDooWOSMOvrlll9oTk6/mXZIerF2DFWA2chHZsnc2zrmAKjVOyDbSYHr0e3Djx9F89nZ3U2Bn+lSw==` , sha1 `9da7f919f28e2622610b34a32e51f06a6c3c02cb`). Pinned RFC 8785 test suite fixtures originate from the separate normative reference repository `https://github.com/cyberphone/json-canonicalization` (Anders Rundgren, commit `19d51d7fe467d4706a3ff08adf8a748f29fc21e0`). Conformance testing of `json-canonicalize` against this suite is designated strictly as `Specified – Pending Verification` until implementation authorization. Input validation strictly prohibits non-finite numbers (`NaN` , `Infinity`), lone surrogates, and duplicate JSON parser keys.
17. **In-Transaction Lock-Acquired Fail-Closed TrustedClock Boundary (AUDIT-39, AUDIT-43, AUDIT-49, AUDIT-51)**: The persistence boundary obtains wall-clock time internally via the designated `TrustedClock` interface (`clock.nowUtc()`) strictly **after** acquiring the stream head row lock inside the database transaction, formally designated as the Post-Lock Authorization Instant (T_{auth}, `tx_auth_time_utc`), preventing receipt expiration races during lock queue waits. The clock interface throws `E-ERR-007: TRUSTED_CLOCK_UNAVAILABLE` fail-closed if the clock source is unreachable, desynchronized (> 5,000 ms skew against trusted NTP/PTP reference), or non-monotonic. Receipt expiration enforces inclusive validity $T_{\text{auth}} \le T_{\text{expires}}$, and any pre-commit error, lock timeout, or clock failure causes an explicit database ROLLBACK preserving unburned nonces. Connection loss during COMMIT enters an ambiguous outcome state; the caller MUST execute pessimistic row-lock reconciliation (`SELECT ... FOR UPDATE` on the primary) to wait for resolution, inspect the event at `attempted_resulting_sequence` (returning `COMMITTED` upon independently recomputing and verifying event digest and retrieved assessment bytes hash even if head advanced subsequently), establish definitive rollback only when head is unchanged and event absent, or preserve `OUTCOME_UNKNOWN` with truthful `stream_sequence: null` on timeout/disconnect (AUDIT-51).

1.1 Gate M0 Baseline & Lineage Tracking

Attribute	Baseline Value	Governance Significance
Gate M0 Commit	`582637b`	Clean root anchor on branch `main` at `D:\Market HFT OS`
Project Charter	`v0.2.4`	Established core governance, 6 engines, and non-negotiable boundaries
Specification Lineage	`v1.0` -> `v1.12` -> `v1.13 Final`	Progressive refinement through 49 targeted owner audit closures
Quarantine Commit	`0c67e26`	Read-only reference anchor for `_salvage/AIStudio`
External Repositories	4 submodules	Read-only reference anchor in `_salvage/external/`
Implementation Status	**BLOCKED**	Gate M1 remains strictly a design contract pending owner freeze

---

1.2 Consolidated Audit Closure Ledger (AUDIT-01 – AUDIT-49)

Final Owner Audit Round 2 Closure Amendments (AUDIT-41 – AUDIT-49)

This revision resolves the remaining contract and integrity findings identified across Project Owner audits of v1.13:

Audit Finding ID	Severity	Specification Clauses Affected	Concrete Design Resolution in Final v1.13 Normative Contract	Pinned Test / KAT ID
AUDIT-41	**P1**	§3.3, §3.4	**Single Canonicalization, Immutable Byte Retention & Evaluator Provenance**: Eliminated TOCTOU gap by executing fail-closed structural validation on `raw_assessment` before field dereferencing, canonicalizing exactly once into `canonical_assessment_bytes`, and returning sealed `ValidatedPromotionEvaluation`. Implemented tamper-proof evaluator authentication token (`draft.evaluator_auth_token`) cryptographically verified in `validateEvaluationDraftIntegrity()`. Replaced in-tx re-canonicalization with persistence of retained verified bytes.	`ASSESSMENT-SINGLE-CANON-033`
AUDIT-42	**P1**	§3.4, §4.1	**Nonce Schema Alignment, Atomic Stream Head Update & Canonical Sequence**: Bound and persisted `gateway_id` in `tx.insertNonceIfUnused(nonce, gateway_id, expires_at_utc, consumed_at_utc)`. Made nonce insertion, assessment storage, stream event append, and stream head update (`tx.updateOfferStreamHead()`) one unified atomic transaction. Established strict canonical sequence decimal format (`CanonicalStreamSequence`, regex `/^(0	[1-9]\d*)$/) and SQL CHECK constraint, barring leading zeros ("01" vs "1").
AUDIT-43	**P1**	§3.4, §4.1	**In-Transaction Lock-Acquired Trusted Clock Sampling**: Shifted trusted time sampling from pre-transaction to strictly **after row-lock acquisition** inside `executeTransaction()` (`tx_auth_time_utc = clock.nowUtc()`). Nonce expiration is revalidated against `tx_auth_time_utc` immediately before consumption, preventing receipt invalidation during lock queue waits.	`CLOCK-TX-LOCK-BOUNDARY-036`
AUDIT-44	**P1**	§7.1, §7.2	**Two-Tier Gross-Clearing Accounting Model & Liquidity Boundary**: Formally partitioned operating cash flows into Tier 1 (Trade Clearing Asset B_{clearing}) and Tier 2 (Available Operating Bank Cash B_{bank}). Gross customer proceeds (P_{gross}) and itemized deductions settle in escrow clearing; net merchant bank disbursements (P_{net}) are recorded as internal Liquidity Transfers ($\Delta B_{\text{clearing}} = -P_{\text{net}}, \Delta B_{\text{bank}} = +P_{\text{net}}$). Capital outlays and peak funding ($C_{\text{peak, intraday}}$) are evaluated against B_{bank}, preventing escrow-held funds from overstating deployable purchasing liquidity.	`FIN-GROSS-CLEARING-TIER-037`
AUDIT-45	**P1**	§6.1, §8.1	**Disambiguated Canonicalizer Package Provenance & Conformance Boundary**: Documented exact separation between the runtime NPM package `json-canonicalize@3.0.1` (`https://github.com/snowyu/json-canonicalize.ts`, integrity `sha512-xEzkTucfHhDooWOSMOvrlll9oTk6/mXZIerF2DFWA2chHZsnc2zrmAKjVOyDbSYHr0e3Djx9F89nZ3U2Bn+lSw==`) and the upstream normative IETF RFC 8785 reference standard and test corpus repository (`https://github.com/cyberphone/json-canonicalization`, Anders Rundgren, commit `19d51d7fe467d4706a3ff08adf8a748f29fc21e0`). Full RFC 8785 corpus test execution maintained as `Specified – Pending Verification`.	`JCS-PROVENANCE-SEPARATION-038`
AUDIT-46	**P1**	§3.3, §3.4	**Comprehensive Evaluator Decision HMAC Authentication**: Upgraded evaluator HMAC from 6 ad-hoc fields to full RFC 8785 canonical serialization of `EvaluatorDecisionSigningPayload` under domain separator `POC-001/EVALUATOR_DECISION_AUTH/v1` (covering `assigned_offer_status`, `assigned_verification_status`, `rejection_reason`, `evaluator_version`, and full canonical `claims_evidence_summary` and `nonces_to_consume`). Isolated evaluator secret strictly within the trusted service boundary, verified via injected `EvaluatorTokenVerifier` using constant-time comparison (`crypto.timingSafeEqual`). Removed `evaluator_secret_hex` from public caller parameter lists.	`EVAL-DECISION-HMAC-COVERAGE-039`
AUDIT-47	**P1**	§3.4, §4.1	**Commit-Time Receipt, Authority & Digest Binding Revalidation**: Inside the transaction after row-lock acquisition, revalidate the gateway receipt's cryptographic signature, active key in `GATEWAY_AUTHORITY_REGISTRY`, operator role/revocation in `ATTESTER_AUTHORITY_REGISTRY`, and claim payload digest binding, in addition to nonce expiration. Nonce insertion, assessment storage, stream event append, and stream head update commit atomically.	`COMMIT-TX-RECEIPT-REVALIDATION-040`
AUDIT-48	**P1**	§7.1, §7.2, §7.3	**Peak Funding Independent of Initial Bank Balance**: Separated Available Bank Liquidity ($L_{\text{bank}}(t) = B_0 + \sum F_j + \sum C_j^{\text{bank_ops}}$) from Cumulative Operating Cash Flow Series ($B_{\text{ops}}(k) = \sum_{j=1}^k C_j^{\text{ops}}$ with pre-event baseline $B_{\text{ops}}(0) \equiv 0$ at position inception t_0, and operating transactions $j \in {1, \dots, K}$). Peak operating funding $C_{\text{peak, ops}} = \max(0, \max_{k \le K}(-\sum_{j=1}^k C_j^{\text{ops}}))$ measures the operating cycle's deficit independently of initial owner equity (B_0) or financing injections (F_j). Reconciled strictly to the RoPF_{mtm} denominator while preserving the gross-clearing no-double-counting invariant.	`FIN-PEAK-INDEPENDENT-EQUITY-041`
AUDIT-49	**P2**	§3.4, §4.1, §8.0	**Conformance Semantics, Clock Fail-Closed & Sequence SQL Constraint**: Specified explicit error `E-ERR-007: TRUSTED_CLOCK_UNAVAILABLE` fail-closed if clock is unreachable, desynchronized (>5s), or non-monotonic. Added explicit uint64 upper bound to SQL `CHECK` constraint: `CHECK (sequence_number ~ '^(0 | [1-9][0-9]*)$' AND CAST(sequence_number AS NUMERIC) <= 18446744073709551615)`. `Formally defined 3-state acceptance semantics` (`Specified — Pending Verification`, `Design Reviewed`, `Verified — Test Evidence Recorded`) with strict designated evidence criteria.	
AUDIT-50	**P1**	§7.1, §7.3, §8.1	**Operating Cash Flow Indexing & Disambiguated Baseline ($B_{\text{ops}}(0) \equiv 0$)**: Formally defined $B_{\text{ops}}(0) \equiv 0$ as the pre-event baseline state at position inception t_0, indexed subsequent operating transactions as $j \in {1, \dots, K}$, and defined $B_{\text{ops}}(k) = B_{\text{ops}}(0) + \sum_{j=1}^k C_j^{\text{ops}} = \sum_{j=1}^k C_j^{\text{ops}}$ for sequence steps $k \in {0, \dots, K}$. Eliminated mathematical collision between $B_{\text{ops}}(0) = 0$ and $C_0 \ne 0$. Provided complete step-by-step numerical proof for fixture `REC-OUTLAY-001` (Pre-event: 0; $j=1$ advance $+500 \implies B=+500$; $j=2$ outlay $-1,500 \implies B=-1,000$, $k_{\text{outlay}}=2$, Day 10; $j=3$ recovery $+1,200 \implies B=+200$, $k_{\text{rec}}=3$, Day 25; Outlay Recovery = 15 days, Lifecycle Recovery = 25 days, $C_{\text{peak, ops}} = 1,000$).	`REC-OUTLAY-001`

Audit Finding ID	Severity	Specification Clauses Affected	Concrete Design Resolution in Final v1.13 Normative Contract	Pinned Test / KAT ID
AUDIT-51	**P1**	§3.4, §4.1, §8.1	**Post-Lock Authorization Instant (T_{auth}), Expiration Inclusivity & Pessimistic Row-Lock Reconciliation**: Formally designated in-transaction clock sampling post-row-lock as Post-Lock Authorization Instant ($T_{\text{auth}} = \text{clock.nowUtc()}$, `tx_auth_time_utc`). Defined strict boundary inclusivity ($T_{\text{auth}} \le T_{\text{expires}}$). Replaced flawed unchanged-read rollback inference with mandatory pessimistic row-lock reconciliation (`reconcileAmbiguousPromotionCommit`, `SELECT ... FOR UPDATE` on primary): waits for in-flight transaction resolution before classifying outcome; inspects event at `attempted_resulting_sequence` with mandatory independent cryptographic recomputation of event digest and retrieved assessment bytes hash (returning `COMMITTED` even if subsequent events advanced head); establishes definitive rollback only when event absent and head unchanged; invalidates stale drafts on unrelated stream advance; and preserves `OUTCOME_UNKNOWN` with `stream_sequence: null` without retrying on lock timeout or database unavailability.	`COMMIT-TX-RECEIPT-REVALIDATION-040`, `RECONCILE-AMBIGUOUS-COMMIT-045`, `RECONCILE-SUBSEQUENT-EVENT-HEAD-046`, `RECONCILE-EVENT-HASH-MISMATCH-047`, `RECONCILE-ASSESSMENT-HASH-MISMATCH-048`, `RECONCILE-LOCK-TIMEOUT-UNKNOWN-049`
AUDIT-52	**P2**	§8.0	**Automated Consistency Check Scoping & Invariant Auditing**: Explicitly scoped `verify_spec_final.py` as an automated document-level structural and syntax regression check (verifying cross-section invariant phrasing, regex bounds, type signatures, and absence of deprecated terms), explicitly distinguished from runtime cryptographic proof, test execution, or mathematical theorem proving.	`SPEC-CONSISTENCY-CHECK-044`

1.3 Design-Freeze Readiness Checklist (Final Normative Contract)

Using the auditable three-state convention:

- `Specified — Pending Verification`: Normative contract exists, but the required implementation-level test has not run.
- `Design Reviewed`: Logic, formulas, schemas, and error boundaries have been audited and verified without claiming runtime test suite execution.
- `Verified — Test Evidence Recorded`: Designated test fixture or cryptographic vector was independently verified with reproducible results retained.

Design Invariant / Contract Area	Status	Verification Evidence / Design Closure
1. Evidence Allowlist & Strict Tuple Enforcement	**Design Reviewed**	18-tuple Allowlist + `validateEvidenceAllowlistEligibility()` + fail-closed rejection (`E-ERR-001`). Zero untyped promotions.
2. Two-Phase Promotion Gate & Nonce Lifecycle	**Design Reviewed**	Phase 1 read-only evaluation + Phase 2 atomic persistence. Nonces consumed single-use only upon committed promotion; zero nonces burned on assessment failure.
3. Exact Assessment Claim Derivation & Evaluator Provenance	**Design Reviewed**	`validateEvaluationDraftIntegrity()` unpacks 4 assessment claims, verifies parameter digests, verifies evaluator auth token via `EvaluatorTokenVerifier` with `timingSafeEqual` (AUDIT-46), strictly compares all fields, derives nonces with 1-to-1 bijection, and deep-freezes validated snapshot.
4. Single Canonicalization & In-Transaction Assessment Storage	**Design Reviewed**	Assessment canonicalized once into `canonical_assessment_bytes` . Retained bytes stored in `offer_assessment_store` within same transaction as nonce consumption, stream head update, and event append.
5. Normative PostgreSQL Contract & Atomic Stream Head Update	**Design Reviewed**	PostgreSQL `READ COMMITTED` with `SELECT ... FOR UPDATE` on `offer_stream_heads` , database primary key constraints, `updateOfferStreamHead()` , and BigInt numeric sequence comparisons on canonical decimal strings (`CanonicalStreamSequence` , bounded by uint64 max in SQL `CHECK`).
6. In-Transaction Lock-Acquired Fail-Closed Clock & Revalidation	**Design Reviewed**	Clock sampled after row-lock acquisition at Post-Lock Authorization Instant ($T_{\text{auth}} = \text{clock.nowUtc()}$, `tx_auth_time_utc` , fails closed with `E-ERR-007: TRUSTED_CLOCK_UNAVAILABLE`), revalidating receipt expiration ($T_{\text{auth}} \le T_{\text{expires}}$), gateway key, attester role/revocation, and digest binding inside tx. Pre-commit failures trigger explicit ROLLBACK; connection loss during COMMIT mandates pessimistic row-lock reconciliation (`reconcileAmbiguousPromotionCommit` , `SELECT ... FOR UPDATE`) inspecting attempted event with independent cryptographic recomputation for `COMMITTED` across subsequent head advances before inferring rollback (AUDIT-47, AUDIT-49, AUDIT-51).
7. Two-Tier Gross-Clearing Model & Cumulative Operating Series	**Design Reviewed**	Two-tier gross-clearing model partitions Trade Clearing Asset (B_{clearing}) from Operating Bank Cash (B_{bank}). Net merchant payouts are clearing-to-bank transfers, not secondary revenue. Available liquidity L_{bank} separated from cumulative operating series $B_{\text{ops}}(k) = B_{\text{ops}}(0) + \sum_{j=1}^k C_j^{\text{ops}}$ with pre-event baseline $B_{\text{ops}}(0) \equiv 0$ (AUDIT-44, AUDIT-48, AUDIT-50).
8. Disambiguated Peak Operating Funding & Equity Independence	**Design Reviewed**	$C_{\text{peak, ops}} = \max(0, \max_{k \le K}(-\sum_{j=1}^k C_j^{\text{ops}}))$ evaluated from pre-event baseline $B_{\text{ops}}(0) = 0$ is strictly invariant to initial bank equity B_0 or owner financing injections F_j (AUDIT-48, AUDIT-50). Realized Cash Multiple distinguished from Marked-to-Market RoPF.
9. Deterministic Product Identity & HSN Syntax Scope	**Design Reviewed**	`normalizeProductIdentityCore()` applies Unicode NFC, dimension key deduplication, and MPN casing preservation. HSN syntax validation explicitly scoped as format check, not legal assignment.
10. Decoupled 3-Tier Currency Scales & Exact Money	**Design Reviewed**	Subunit integer arithmetic with decoupled scales. Cross-currency addition strictly barred (`E-ERR-008`). Point-of-use FX receipt verification with BigInt rational arithmetic.
11. Disambiguated JCS Package Provenance & Verification	**Design Reviewed**	Pinned `json-canonicalize@3.0.1` (`snowyu/json-canonicalize.ts`) tarball URL and SHA-512 integrity hash distinguished from `cyberphone/json-canonicalization` commit `19d51d7fe467d4706a3ff08adf8a748f29fc21e0` . Full RFC 8785 corpus status maintained strictly as `Specified — Pending Verification` .
12. Volumetric Weight Calculation (KAT-03)	**Verified — Test Evidence Recorded**	Independently verified exact safe-integer formula: $300 \times 200 \times 100\text{ mm} = 1{,}200\text{ g}$ against configured divisor $5{,}000\text{ cm}^3/\text{kg}$.
13. Rational FX Arithmetic (KAT-04)	**Verified — Test Evidence Recorded**	Independently verified BigInt rational arithmetic against KAT-04 (Parts A & B): produces `865086` , `865260` , and `779122735535095894` bitwise; exposes ₹0.0086 Float64 drift.
14. Ed25519 Gateway Receipt Signature (KAT-05B)	**Verified — Test Evidence Recorded**	Independently verified in Node.js crypto: derived public key, canonical payload SHA-256 (`c434bdf...`), and signature verify validly against deterministic seed.
15. Dual-Ledger Isolation & Ticket Verification	**Design Reviewed**	Bidirectional guard (`E-ERR-020` / `E-ERR-025`) + cryptographic gateway ticket verification binding exact mutation digest (`E-ERR-026`).
16. Acceptance Test Suite Execution (Suites 01–11)	**Specified — Pending Verification**	11 comprehensive suites with explicit test IDs defined in §8; test suite execution remains strictly blocked pending implementation authorization.
17. Governance Baseline & Audit Truthfulness	**Verified — Test Evidence Recorded**	Git repository remains 100% clean at commit `582637b` ; zero packages installed; specification SHA-256 reported as an independent artifact fact.

2. Canonical Domain Types Specification

2.1 Currency Registry, Multi-Currency Aggregation & Money Representation

Three-Tier Currency Precision Model

Each supported currency defines three explicit scales:

1. `iso4217_minor_unit_scale` : The official ISO 4217 minor-unit exponent (e.g. INR=2, USD=2, JPY=0).
2. `internal_accounting_scale` : POC-001's internal representation precision (e.g. INR=4, USD=4, JPY=2), providing 2 decimal places of sub-unit precision to eliminate cumulative rounding drift.
3. `settlement_scale` : The supported external transaction / payment scale (e.g. INR=2, USD=2, JPY=0).

```
export type CurrencyCode = 'INR' | 'USD' | 'EUR' | 'GBP' | 'JPY' | 'SGD' | 'AED';

export interface CurrencyDefinition {
  code: CurrencyCode;
  name: string;
  iso4217_minor_unit_scale: number;  // Standard ISO minor unit (e.g. 2 for INR)
  internal_accounting_scale: number; // Internal platform precision (e.g. 4 for INR)
  settlement_scale: number;          // External settlement precision (e.g. 2 for INR)
  rounding_mode: 'HALF_EVEN';        // Banker's rounding
}

export const CURRENCY_REGISTRY: Record<CurrencyCode, CurrencyDefinition> = {
  INR: { code: 'INR', name: 'Indian Rupee',     iso4217_minor_unit_scale: 2, internal_accounting_scale: 4, settlement_scale: 2, rounding_mode: 'HALF_EVEN' },
  USD: { code: 'USD', name: 'US Dollar',        iso4217_minor_unit_scale: 2, internal_accounting_scale: 4, settlement_scale: 2, rounding_mode: 'HALF_EVEN' },
  EUR: { code: 'EUR', name: 'Euro',             iso4217_minor_unit_scale: 2, internal_accounting_scale: 4, settlement_scale: 2, rounding_mode: 'HALF_EVEN' },
  GBP: { code: 'GBP', name: 'British Pound',    iso4217_minor_unit_scale: 2, internal_accounting_scale: 4, settlement_scale: 2, rounding_mode: 'HALF_EVEN' },
  JPY: { code: 'JPY', name: 'Japanese Yen',     iso4217_minor_unit_scale: 0, internal_accounting_scale: 2, settlement_scale: 0, rounding_mode: 'HALF_EVEN' },
  SGD: { code: 'SGD', name: 'Singapore Dollar', iso4217_minor_unit_scale: 2, internal_accounting_scale: 4, settlement_scale: 2, rounding_mode: 'HALF_EVEN' },
  AED: { code: 'AED', name: 'UAE Dirham',       iso4217_minor_unit_scale: 2, internal_accounting_scale: 4, settlement_scale: 2, rounding_mode: 'HALF_EVEN' }
};

/**
 * Canonical Decimal String Bounds:
 * - Validated against regex: /^(0|-?[1-9][0-9]*)$/ (no leading zeros; "-0" rejected)
 * - Exact signed 128-bit interval: -2^127 <= n <= 2^127 - 1
 */
export interface CanonicalMoney {
  amount_subunits: string; // e.g. "105000" representing 10.5000 INR
  scale: number;           // MUST match CURRENCY_REGISTRY[currency].internal_accounting_scale
  currency: CurrencyCode;
}

/**
 * Runtime Boundary Constructor & Validator for CanonicalMoney
 */
export function createCanonicalMoney(amount_subunits: string, currency: CurrencyCode, scale: number): CanonicalMoney {
  const def = CURRENCY_REGISTRY[currency];
  if (!def) {
    throw new Error(`E-ERR-010: INVALID_CURRENCY: Unsupported currency code '${currency}'`);
  }
  if (scale !== def.internal_accounting_scale) {
    throw new Error(`E-ERR-010: INVALID_CURRENCY_SCALE: Scale ${scale} does not match internal accounting scale ${def.internal_accounting_scale} for ${currency}`);
  }
  if (!/^(0|-?[1-9][0-9]*)$/.test(amount_subunits)) {
    throw new Error(`E-ERR-010: CANONICAL_DECIMAL_FORMAT_ERROR: Subunits '${amount_subunits}' is not a valid non-empty canonical integer string`);
  }
  const val = BigInt(amount_subunits);
  const min128 = -(2n ** 127n);
  const max128 = (2n ** 127n) - 1n;
  if (val < min128 || val > max128) {
    throw new Error(`E-ERR-014: ARITHMETIC_OVERFLOW: Subunit value exceeds signed 128-bit integer boundaries`);
  }
  return { amount_subunits, scale, currency };
}
```

Scale Conversion & Two-Stage Rounding Architecture:

1. **Strict Scale Enforcement**: `CanonicalMoney` instances must strictly declare `scale === CURRENCY_REGISTRY[currency].internal_accounting_scale`. Constructing `CanonicalMoney` with a non-matching scale throws `E-ERR-010: INVALID_CURRENCY_SCALE`.
2. **Ingestion Upscaling (Exact Zero-Padding)**:
   $$\text{Subunits}_{\textit{internal}} = \textit{Subunits}_{\text{settlement}} \times 10^{(\text{internal_accounting_scale} - \text{settlement_scale})}$$
3. **Stage 1 — FX Conversion Rounding (Internal Scale)**: When converting between currencies via FX rate, the exact rational product is rounded to the target currency's `internal_accounting_scale` using banker's rounding (`HALF_EVEN`).
4. **Stage 2 — Settlement Downscaling Rounding**: When outputting an internal amount to external payment settlement:
   $$\text{Subunits}_{\textit{settlement}} = \textit{round}_{\text{half-even}}\left(\frac{\text{Subunits}_{\text{internal}}}{10^{(\text{internal_accounting_scale} - \text{settlement_scale})}} \right)$$

Multi-Currency Prohibition & Exact Rational FX Arithmetic

- **Summation Prohibition**: Summing monetary values of differing currencies directly is prohibited (`E-ERR-008: CROSS_CURRENCY_SUMMATION_ILLEGAL`).
- **Quote Direction Convention**: All exchange rates use the **direct quote convention**: units of target currency per one unit of source currency (`UNITS_TARGET_PER_ONE_SOURCE`).
- **Strict Prohibition of IEEE 754 Floating-Point**: Using JavaScript `Number` for intermediate FX conversion arithmetic is strictly forbidden.
- **Exact Rational Integer Algorithm**:
  Exchange rate decimal strings (e.g. `"86.500000"`) are parsed into an exact integer numerator $R_{\text{num}} = 86500000$ and rate scale $S_{\text{rate}} = 6$. Let S_{src} and S_{tgt} be the respective internal accounting scales, and $\Delta S = S_{\text{tgt}} - S_{\text{src}}$.

  $$\text{If } \Delta S \ge 0: \quad \text{Numerator} = A_{\text{src}} \times 10^{\Delta S} \times R_{\text{num}}, \quad \text{Denominator} = 10^{S_{\text{rate}}}$$
  $$\text{If } \Delta S < 0: \quad \text{Numerator} = A_{\text{src}} \times R_{\text{num}}, \quad \text{Denominator} = 10^{-\Delta S} \times 10^{S_{\text{rate}}}$$

Using integer division with remainder:
$$Q = \text{Numerator} \div \text{Denominator}, \quad R = |\text{Numerator} \pmod{\text{Denominator}}|$$
$$H = \text{Denominator} \div 2$$
Banker's rounding (`HALF_EVEN`):

- If $R > H \implies |A_{\text{tgt}}| = |Q| + 1$
- If $R < H \implies |A_{\text{tgt}}| = |Q|$
- If $R == H \implies |A_{\text{tgt}}| = (|Q| \pmod 2 \ne 0) ,?, |Q| + 1 : |Q|$
- Sign of A_{tgt} matches sign of Numerator. Intermediate and resulting values are checked against signed 128-bit bounds (`E-ERR-014: ARITHMETIC_OVERFLOW`).

```
export interface FXConversionReceipt {
  receipt_id: string;
  source_currency: CurrencyCode;
  target_currency: CurrencyCode;
  quote_convention: 'UNITS_TARGET_PER_ONE_SOURCE';
  exchange_rate: string;                          // Decimal string with scale=6 (e.g. "86.500000")
  rate_publication_date: string;                  // YYYY-MM-DD
  effective_timestamp_utc: string;                // ISO-8601 UTC
  rate_source: 'RBI_OFFICIAL_REFERENCE' | 'ECB_REFERENCE' | 'FED_H10';
  rounding_stage: 'STAGE1_INTERNAL_HALF_EVEN';    // Rounded to target internal accounting scale
  oracle_public_key_hex: string;                  // 64-character lowercase hex Ed25519 key
  oracle_signature: string;                       // 128-char hex Ed25519 signature over canonical receipt bytes
}

export function validateFXReceiptAtPointOfUse(
  receipt: FXConversionReceipt,
  source_currency: CurrencyCode,
  target_currency: CurrencyCode,
  transaction_time_utc: string
): void {
  if (receipt.source_currency !== source_currency || receipt.target_currency !== target_currency) {
    throw new Error('E-ERR-008: FX_RECEIPT_CURRENCY_MISMATCH');
  }
  if (receipt.quote_convention !== 'UNITS_TARGET_PER_ONE_SOURCE') {
    throw new Error('E-ERR-008: INVALID_FX_QUOTE_CONVENTION');
  }
  validateIso8601UtcTimestamp(receipt.effective_timestamp_utc);
  validateIso8601UtcTimestamp(transaction_time_utc);
  const tx_ms = Date.parse(transaction_time_utc);
  const fx_ms = Date.parse(receipt.effective_timestamp_utc);
  if (Math.abs(tx_ms - fx_ms) > 86400000) {
    throw new Error('E-ERR-008: FX_RECEIPT_STALE: Receipt effective date deviates by more than 24 hours from transaction time');
  }
  const { oracle_signature, ...unsigned_receipt } = receipt;
  const unsigned_bytes = Buffer.from(canonicalizeJCS(unsigned_receipt), 'utf8');
  if (!ed25519Verify(unsigned_bytes, receipt.oracle_public_key_hex, oracle_signature)) {
    throw new Error('E-ERR-008: FX_RECEIPT_SIGNATURE_INVALID');
  }
}
```

2.2 Deterministically Normalized Product Identity Core (AUDIT-17, AUDIT-25, AUDIT-33)

> [!NOTE]
> **Deterministic Identity Scope & HSN Tariff Syntax Clarification (AUDIT-40)**:
>
> 1. **Reproducible Identity String**: The SHA-256 digest of normalized `ProductIdentityCore` produces a deterministic, reproducible identity string across distributed platform components. It does not constitute an ontological or physical proof that two distinct real-world source records represent the identical physical item. Physical equality remains subject to physical inspection and supplier verification.
> 2. **HSN Format Validation**: Validation of `category_hsn` validates string syntax, length, and numerical structure (4, 6, or 8 digits matching Chapter/Heading/Subheading syntax). The syntax check does **not** establish legal assignment or applicability under the authoritative HSN/CBIC tariff schedule effective on the relevant date.

```
export interface ProductIdentityCore {
  category_hsn: string;                // e.g. "73269099" (syntax validated against 4, 6, or 8 digit tariff hierarchy)
  unit_of_measure: 'PIECE' | 'PACK' | 'KG';
  units_per_pack: number;              // Positive integer, e.g. 10
  standard_specification?: string;     // e.g. "DIN_3017_SS304_W4"
  manufacturer_id?: string;            // Stable manufacturer identity slug/hash, e.g. "mfr:in:bhiwadi-precision-clamps"
  manufacturer_part_number?: string;   // e.g. "BPC-3017-16-25"
  critical_dimension_key?: string;     // e.g. "clamping_range_mm:16-25;band_width_mm:9"
}

export interface ManufacturerNamespaceConfig {
  manufacturer_id: string;
  case_insensitive?: boolean;          // If true, MPN is uppercase-folded; default false preserves exact casing
}

/**
 * Deterministic Namespace-Specific Normalization Routine (AUDIT-17, AUDIT-25, AUDIT-33)
 * Applies Unicode NFC normalization, validates HSN tariff format, sanitizes control characters,
 * preserves MPN character casing and punctuation unless the manufacturer namespace explicitly defines case-insensitivity,
 * and rejects duplicate dimension keys fail-closed.
 */
export function normalizeProductIdentityCore(
  raw: ProductIdentityCore,
  namespace_registry?: Record<string, ManufacturerNamespaceConfig>
): ProductIdentityCore {
  // 0. Unicode Normalization Form C (NFC) & Control Character Sanitization
  const sanitizeUnicode = (s: string, fieldName: string): string => {
    const nfc = s.normalize('NFC');
    if (/[\u0000-\u001F\u007F-\u009F]/.test(nfc)) {
      throw new Error(`E-ERR-024: MALFORMED_IDENTIFIER: Control characters prohibited in field '${fieldName}'`);
    }
    return nfc;
  };

  // 1. HSN Tariff Syntax & Length Validation (4, 6, or 8 digits)
  const rawHsn = sanitizeUnicode(raw.category_hsn, 'category_hsn').trim().replace(/\s+/g, '');
  if (!/^\d{4}(\d{2}(\d{2})?)?$/.test(rawHsn)) {
    throw new Error('E-ERR-024: INVALID_HSN_FORMAT: category_hsn must be 4, 6, or 8 numeric digits matching Chapter/Heading/Subheading tariff hierarchy');
  }

  // 2. Unit of measure & pack quantity
  if (!['PIECE', 'PACK', 'KG'].includes(raw.unit_of_measure)) {
    throw new Error('E-ERR-024: INVALID_UNIT_OF_MEASURE');
  }
  if (!Number.isSafeInteger(raw.units_per_pack) || raw.units_per_pack <= 0) {
    throw new Error('E-ERR-024: INVALID_UNITS_PER_PACK: Must be positive safe integer');
  }

  const normalized: ProductIdentityCore = {
    category_hsn: rawHsn,
    unit_of_measure: raw.unit_of_measure,
    units_per_pack: raw.units_per_pack
  };

  // 3. Standard Specification (uppercase, single underscore, sanitized Unicode)
  if (raw.standard_specification && raw.standard_specification.trim()) {
    const spec = sanitizeUnicode(raw.standard_specification, 'standard_specification').trim();
    normalized.standard_specification = spec.toUpperCase().replace(/[\s-]+/g, '_');
  }

  // 4. Manufacturer ID & MPN Normalization (Safe Casing & Punctuation Preservation)
  if (raw.manufacturer_part_number && raw.manufacturer_part_number.trim()) {
    if (!raw.manufacturer_id || !raw.manufacturer_id.trim()) {
      throw new Error('E-ERR-013: UNNAMESPACED_MANUFACTURER_PART_NUMBER: manufacturer_id is mandatory when MPN is specified');
    }
    const mfrId = sanitizeUnicode(raw.manufacturer_id, 'manufacturer_id').trim();
    const rawMpn = sanitizeUnicode(raw.manufacturer_part_number, 'manufacturer_part_number').trim();
    normalized.manufacturer_id = mfrId;

    const nsConfig = namespace_registry?.[mfrId];
    if (nsConfig?.case_insensitive) {
      normalized.manufacturer_part_number = rawMpn.toUpperCase();
    } else {
      // Default: preserve exact character casing, hyphens, and internal punctuation
      normalized.manufacturer_part_number = rawMpn;
    }
  } else {
    // Generic item must provide critical dimension key
    if (!raw.critical_dimension_key || !raw.critical_dimension_key.trim()) {
      throw new Error('E-ERR-009: INSUFFICIENT_IDENTITY_DISAMBIGUATION: Generic item requires critical_dimension_key');
    }
    const sanitizedDimKey = sanitizeUnicode(raw.critical_dimension_key, 'critical_dimension_key');
    const rawPairs = sanitizedDimKey.trim().split(';')
      .map(p => p.trim())
      .filter(p => p.length > 0);

    // Check for duplicate dimension keys
    const seenKeys = new Set<string>();
    const normalizedPairs: string[] = [];
    for (const pair of rawPairs) {
      const idx = pair.indexOf(':');
      if (idx === -1) throw new Error('E-ERR-024: MALFORMED_DIMENSION_KEY_PAIR: Missing colon separator');
      const k = pair.substring(0, idx).trim().toLowerCase();
      const v = pair.substring(idx + 1).trim();
      if (!k || !v) throw new Error('E-ERR-024: MALFORMED_DIMENSION_KEY_PAIR: Empty dimension key or value');
      if (seenKeys.has(k)) {
```

```
        throw new Error(`E-ERR-024: DUPLICATE_DIMENSION_KEY: Key '${k}' duplicated in critical_dimension_key`);
      }
      seenKeys.add(k);
      normalizedPairs.push(`${k}:${v}`);
    }
    normalized.critical_dimension_key = normalizedPairs.sort().join(';');
  }

  return normalized;
}
```

Safe-Integer Volumetric Weight Specification & Configured Freight Profile

- **Supported Dimension Bounds**: Dimensions must satisfy safe integers $1 \le L_{\text{mm}}, W_{\text{mm}}, H_{\text{mm}} \le 10,000\text{ mm}$ validated via `Number.isSafeInteger()`. Mass in grams must satisfy $1 \le M_{\text{g}} \le 100,000,000\text{ g}$. Values outside fail closed with `E-ERR-021: DIMENSION_BOUNDS_EXCEEDED`.
- **Proof of Safe-Integer Exactness**:
  For all inputs within bounds, volume product $V = L_{\text{mm}} \times W_{\text{mm}} \times H_{\text{mm}} \le 10^{12}\text{ mm}^3$.
  Since $10^{12} < 2^{53} - 1 = 9,007,199,254,740,991$ (`Number.MAX_SAFE_INTEGER`), V is represented with 100% exactness in IEEE 754 binary64 integer range.
- **Configured Freight Profile**:
  Physical logistics calculation uses the configured **IATA Standard Airfreight / Courier Profile** (Divisor: $5,000\text{ cm}^3/\text{kg} = 5,000\text{ mm}^3/\text{g}$, Rounding: **round-half-up** to nearest gram):
  $$W_{\mathrm{vol,g}} = \left\lfloor \frac{V_{\mathrm{mm^3}} + 2500}{5000} \right\rfloor$$
  Equivalently: `quotient = Math.floor(V / 5000)`, `remainder = V % 5000`. If `remainder >= 2500`, then `quotient + 1`, else `quotient`.

```
export function calculateVolumetricWeightGrams(length_mm: number, width_mm: number, height_mm: number): number {
  if (!Number.isSafeInteger(length_mm) || !Number.isSafeInteger(width_mm) || !Number.isSafeInteger(height_mm) ||
      length_mm < 1 || length_mm > 10000 || width_mm < 1 || width_mm > 10000 || height_mm < 1 || height_mm > 10000) {
    throw new Error('E-ERR-021: DIMENSION_BOUNDS_EXCEEDED');
  }
  const volume_product = length_mm * width_mm * height_mm;
  const quotient = Math.floor(volume_product / 5000);
  const remainder = volume_product % 5000;
  return remainder >= 2500 ? quotient + 1 : quotient;
}

export interface PhysicalDimensions {
  length_mm: number;               // Safe integer millimeters in [1, 10000]
  width_mm: number;                // Safe integer millimeters in [1, 10000]
  height_mm: number;               // Safe integer millimeters in [1, 10000]
  net_weight_g: number;            // Safe integer grams in [1, 100000000]
  gross_packaged_weight_g: number; // Safe integer grams in [1, 100000000]
  calculated_volumetric_weight_g: number; // Evaluated via calculateVolumetricWeightGrams
}

export function createPhysicalDimensions(
  length_mm: number,
  width_mm: number,
  height_mm: number,
  net_weight_g: number,
  gross_packaged_weight_g: number
): PhysicalDimensions {
  const vol_g = calculateVolumetricWeightGrams(length_mm, width_mm, height_mm);
  if (!Number.isSafeInteger(net_weight_g) || net_weight_g < 1 || net_weight_g > 100000000 ||
      !Number.isSafeInteger(gross_packaged_weight_g) || gross_packaged_weight_g < 1 || gross_packaged_weight_g > 100000000) {
    throw new Error('E-ERR-021: DIMENSION_BOUNDS_EXCEEDED');
  }
  return {
    length_mm,
    width_mm,
    height_mm,
    net_weight_g,
    gross_packaged_weight_g,
    calculated_volumetric_weight_g: vol_g
  };
}

export interface CanonicalProduct {
  product_id: string; // Deterministic hash of normalized ProductIdentityCore
  identity_core: ProductIdentityCore;
  version: number;
  sku: string;
  display_name: string;
  description: string;
  physical_specs: PhysicalDimensions | null;
  domain_metadata?: Record<string, unknown>;
}

export interface CanonicalSupplier {
  supplier_id: string; // Deterministic hash: supp_sha256(...)
  legal_name: string;
  trade_name?: string;
  manufacturing_cluster: string;
  gstin?: string;
  msme_registration?: string;
  verification_status: VerificationStatus;
  contact_evidence_id?: string;
}

export type InventoryState =
  | 'IN_TRANSIT_INBOUND'
  | 'WAREHOUSE_RECEIVED'
  | 'AVAILABLE_FOR_SALE'
  | 'RESERVED_ORDER'
  | 'DAMAGED_IMPAIRED'
  | 'RETURNED_CUSTOMER'
  | 'RETURNED_SUPPLIER';

export interface InventoryBucket {
  product_id: string;
  state: InventoryState;
  quantity: number;                     // Validated safe integer in [0, 1000000000]
  batch_lot_id: string;
  unit_landed_cost: CanonicalMoney;
  last_reconciliation_sequence: string; // String-encoded uint64
}

export function validateInventoryQuantity(quantity: number): void {
  if (!Number.isSafeInteger(quantity) || quantity < 0 || quantity > 1000000000) {
    throw new Error('E-ERR-034: INVALID_INVENTORY_QUANTITY: Quantity must be non-negative safe integer <= 10^9');
  }
}
```

2.3 Event Envelope, Ledger & Position Domain Contracts

Canonical Event Envelope

```
export interface CanonicalEventEnvelope<T = unknown> {
  event_id: string;
  stream_id: string;
  sequence_number: string;        // String-encoded monotonic uint64
  previous_event_hash: string;    // SHA-256 digest of prior event or genesis head H_-1
  payload_kind: string;
  payload: T;
  provenance: ProvenanceMetadata;
  recorded_at_utc: string;        // Ingestion timestamp assigned by platform
  event_sha256: string;           // RFC 8785 canonical hash omitting event_sha256
}
```

Actuals-Ledger Single Database Transaction & Trusted Kernel Context

```
export type LedgerType = 'ACTUALS_LEDGER' | 'SIMULATION_LEDGER';

export interface ActualsMutationContent {
  transaction_id: string;
  position_id: string;
  debit_account: string;
  credit_account: string;
  amount: CanonicalMoney;
  description: string;
}

export interface LedgerTransaction {
  transaction_id: string;         // Matches ActualsMutationContent
  ledger_type: 'ACTUALS_LEDGER';  // Server assigned
  position_id: string;            // Matches ActualsMutationContent
  entry_timestamp_utc: string;    // Server assigned (trusted engine clock)
  sequence_number: string;        // Server assigned monotonic sequence
  debit_account: string;          // Matches ActualsMutationContent
  credit_account: string;         // Matches ActualsMutationContent
  amount: CanonicalMoney;         // Matches ActualsMutationContent
  description: string;            // Matches ActualsMutationContent
  linked_event_id: string;        // Server assigned streaming bus event ID
}

export interface ActualsAuthTicketPayload {
  domain_tag: 'POC-001-ACTUALS-MUTATION-TICKET-V1';
  schema_version: '1.0.0';
  ticket_id: string;
  ledger_type: 'ACTUALS_LEDGER';
  permitted_action: 'MUTATE_ACTUALS_LEDGER';
  operator_id: string;
  operator_role: AttesterRole;
  auth_gateway_id: string;
  position_id: string;
  tx_mutation_sha256: string; // SHA-256 of RFC 8785 canonical bytes of ActualsMutationContent
  issued_at_utc: string;
  expires_at_utc: string;     // Explicit signed expiration (exactly issued_at_utc + 300s)
  nonce: string;              // High-entropy UUIDv4 or hex string matching /^[a-zA-Z0-9_-]{16,64}$/
}

export interface ActualsAuthTicketEnvelope {
  payload: ActualsAuthTicketPayload;
  gateway_signature: string; // 128-char hex Ed25519 signature over RFC 8785 canonical bytes of payload
}

export interface AuthenticatedSessionContext {
  readonly session_id: string;
  readonly operator_id: string;
  readonly verified_roles: readonly AttesterRole[];
  readonly authenticated_at_utc: string;
}

export interface ActualsLedgerContext {
  ledger_type: 'ACTUALS_LEDGER';
  session: AuthenticatedSessionContext;
  auth_ticket: ActualsAuthTicketEnvelope;
}

export interface SimulationLedgerContext {
  ledger_type: 'SIMULATION_LEDGER';
  simulation_run_id: string;
  scenario_parameters_hash: string;
}

export type LedgerContext = ActualsLedgerContext | SimulationLedgerContext;

export interface LedgerDatabaseSession {
  executeActualsMutationInTransaction(
    mutation: ActualsMutationContent,
    nonce: string,
    ticket_id: string,
    operator_id: string,
    current_time_utc: string,
    expires_at_utc: string
  ): Promise<LedgerTransaction>;
}
```

Enforceable Bidirectional Write Isolation Guard

```
export async function enforceActualsLedgerWrite(
  mutation: ActualsMutationContent,
  ctx: ActualsLedgerContext,
  current_time_utc: string,
  ledger_db: LedgerDatabaseSession
): Promise<LedgerTransaction> {
  // 1. Bidirectional Ledger Type Match
  if (ctx.ledger_type !== 'ACTUALS_LEDGER') {
    throw new Error('E-ERR-020: ACTUALS_LEDGER_WRITE_VIOLATION: Context is not ACTUALS_LEDGER');
  }
  if (!ctx.auth_ticket || !ctx.auth_ticket.gateway_signature) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Missing auth ticket or gateway signature');
  }
  const payload = ctx.auth_ticket.payload;

  // 2. Runtime Structure & Domain Tag Verification
  if (payload.domain_tag !== 'POC-001-ACTUALS-MUTATION-TICKET-V1' ||
      payload.schema_version !== '1.0.0' ||
      payload.ledger_type !== 'ACTUALS_LEDGER' ||
      payload.permitted_action !== 'MUTATE_ACTUALS_LEDGER') {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Invalid ticket schema or domain tag');
  }

  // 3. Live Session Identity & Role Verification
  if (payload.operator_id !== ctx.session.operator_id) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Ticket operator does not match kernel authenticated session operator');
  }
  if (!ctx.session.verified_roles.includes(payload.operator_role)) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Kernel session does not possess ticket operator role');
  }
  if (payload.position_id !== mutation.position_id) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Position mismatch');
  }

  // 4. Exact Mutation Content SHA-256 Verification
  const computed_mutation_sha256 = sha256(Buffer.from(canonicalizeJCS(mutation), 'utf8'));
  if (payload.tx_mutation_sha256 !== computed_mutation_sha256) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Mutation contents do not match ticket digest');
  }

  // 5. Strict Calendar & Timestamp Validation
  validateIso8601UtcTimestamp(payload.issued_at_utc);
  validateIso8601UtcTimestamp(payload.expires_at_utc);
  validateIso8601UtcTimestamp(current_time_utc);
  const current_ms = Date.parse(current_time_utc);
  const exp_ms = Date.parse(payload.expires_at_utc);
  const iss_ms = Date.parse(payload.issued_at_utc);
  if (current_ms > exp_ms || current_ms < iss_ms - 300000) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Ticket expired or clock skew exceeded');
  }

  // 6. Gateway Registration, Validity Window & Revocation/Compromise Check
  const gateway = GATEWAY_AUTHORITY_REGISTRY[payload.auth_gateway_id];
  if (!gateway || gateway.revoked || gateway.compromised) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Gateway revoked, compromised, or unregistered');
  }
  const gw_from_ms = Date.parse(gateway.valid_from_utc);
  const gw_until_ms = Date.parse(gateway.valid_until_utc);
  if (iss_ms < gw_from_ms || iss_ms > gw_until_ms) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Gateway validity window elapsed');
  }

  // 7. Ed25519 Cryptographic Signature Verification
  const ticket_bytes = Buffer.from(canonicalizeJCS(payload), 'utf8');
  if (!ed25519Verify(ticket_bytes, gateway.public_key_hex, ctx.auth_ticket.gateway_signature)) {
    throw new Error('E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT: Invalid gateway signature');
  }

  // 8. Atomic Nonce Insertion & Ledger Mutation in a Single ACID DB Transaction
  if (!/^[a-zA-Z0-9_-]{16,64}$/.test(payload.nonce)) {
    throw new Error('E-ERR-024: INVALID_NONCE_FORMAT');
  }
  try {
    const record = await ledger_db.executeActualsMutationInTransaction(
      mutation,
      payload.nonce,
      payload.ticket_id,
      payload.operator_id,
      current_time_utc,
      payload.expires_at_utc
    );
    return record;
  } catch (err: unknown) {
    const errorMsg = err instanceof Error ? err.message : String(err);
    if (errorMsg.includes('UNIQUE') || errorMsg.includes('duplicate')) {
      throw new Error('E-ERR-023: GATEWAY_ENDORSEMENT_NONCE_REPLAYED');
    }
    throw new Error(`E-ERR-026: ACTUALS_LEDGER_MUTATION_FAILED: ${errorMsg}`);
  }
}
```

3. Four-Dimensional Evidence Envelope & Fail-Closed Allowlist

3.1 Fail-Closed Allowlist (18 Permitted Tuples) & Claim Correspondence

Allowed Tuple ID	Evidence Kind	Verification Status	Commercial Offer Status	Permitted Commercial Meaning
AL-01	`OBSERVED`	`UNVERIFIED`	`INDICATIVE_QUOTE`	Raw market capture (e.g. web listing asking price)
AL-02	`OBSERVED`	`PARTIALLY_VERIFIED`	`INDICATIVE_QUOTE`	Corroborated price, pending MOQ/delivery terms
AL-03	`OBSERVED`	`VERIFIED`	`VERIFIED_EXECUTABLE_OFFER`	Formally confirmed actionable market listing (Requires passing Four-Claim Gate)
AL-04	`OBSERVED`	`STALE`	`EXPIRED_OFFER`	Previously verified listing past validity period
AL-05	`OBSERVED`	`REJECTED`	`REJECTED_OFFER`	Listing proven fraudulent or out of stock
AL-06	`OBSERVED`	`UNKNOWN`	`INDICATIVE_QUOTE`	Listing captured with indeterminate trust
AL-07	`MANUALLY_SUPPLIED`	`UNVERIFIED`	`INDICATIVE_QUOTE`	Operator-entered phone quote pending invoice
AL-08	`MANUALLY_SUPPLIED`	`VERIFIED`	`VERIFIED_EXECUTABLE_OFFER`	Confirmed supplier quotation with validated PI (Requires passing Four-Claim Gate)
AL-09	`MANUALLY_SUPPLIED`	`STALE`	`EXPIRED_OFFER`	Quotation elapsed past validity date
AL-10	`MANUALLY_SUPPLIED`	`REJECTED`	`REJECTED_OFFER`	Supplier repudiated quote or terms unacceptable
AL-11	`EXTERNALLY_PUBLISHED`	`VERIFIED`	`INDICATIVE_QUOTE`	Published catalog price (indicative until order confirmation)
AL-12	`EXTERNALLY_PUBLISHED`	`VERIFIED`	`NOT_APPLICABLE`	Official published tariff (e.g. Amazon fee card, RBI rate)
AL-13	`EXTERNALLY_PUBLISHED`	`STALE`	`NOT_APPLICABLE`	Superseded tariff or historical benchmark
AL-14	`SYNTHETIC_TEST`	`UNVERIFIED`	`INDICATIVE_QUOTE`	Simulated test quote fixture
AL-15	`SYNTHETIC_TEST`	`UNKNOWN`	`NOT_APPLICABLE`	Injected unit-test fixture
AL-16	`SCENARIO`	`UNKNOWN`	`NOT_APPLICABLE`	User sensitivity parameter set
AL-17	`FORECAST`	`UNKNOWN`	`NOT_APPLICABLE`	Demand model projection
AL-18	`DERIVED`	`VERIFIED`	`NOT_APPLICABLE`	Deterministic calculated fee or landed cost

```
export const PERMITTED_ALLOWLIST_TUPLES = new Set<string>([
  'OBSERVED:UNVERIFIED:INDICATIVE_QUOTE',
  'OBSERVED:PARTIALLY_VERIFIED:INDICATIVE_QUOTE',
  'OBSERVED:VERIFIED:VERIFIED_EXECUTABLE_OFFER',
  'OBSERVED:STALE:EXPIRED_OFFER',
  'OBSERVED:REJECTED:REJECTED_OFFER',
  'OBSERVED:UNKNOWN:INDICATIVE_QUOTE',
  'MANUALLY_SUPPLIED:UNVERIFIED:INDICATIVE_QUOTE',
  'MANUALLY_SUPPLIED:VERIFIED:VERIFIED_EXECUTABLE_OFFER',
  'MANUALLY_SUPPLIED:STALE:EXPIRED_OFFER',
  'MANUALLY_SUPPLIED:REJECTED:REJECTED_OFFER',
  'EXTERNALLY_PUBLISHED:VERIFIED:INDICATIVE_QUOTE',
  'EXTERNALLY_PUBLISHED:VERIFIED:NOT_APPLICABLE',
  'EXTERNALLY_PUBLISHED:STALE:NOT_APPLICABLE',
  'SYNTHETIC_TEST:UNVERIFIED:INDICATIVE_QUOTE',
  'SYNTHETIC_TEST:UNKNOWN:NOT_APPLICABLE',
  'SCENARIO:UNKNOWN:NOT_APPLICABLE',
  'FORECAST:UNKNOWN:NOT_APPLICABLE',
  'DERIVED:VERIFIED:NOT_APPLICABLE'
]);

export const CLAIM_PERMITTED_DOCUMENT_TYPES: Record<string, string[]> = {
  SOURCE_AUTHENTICITY: ['BUSINESS_REGISTRY_FILING', 'GSTIN_CERTIFICATE', 'BANK_LETTER', 'ON_SITE_AUDIT_REPORT'],
  PRICE_CONFIRMATION: ['PROFORMA_INVOICE', 'SIGNED_QUOTATION', 'PUBLISHED_OFFICIAL_TARIFF'],
  STOCK_AVAILABILITY: ['SUPPLIER_INVENTORY_REPORT', 'FACTORY_LEAD_TIME_COMMITMENT'],
  COMMERCIAL_TERMS: ['SUPPLIER_CONTRACT', 'PROFORMA_INVOICE', 'FORMAL_TERMS_SCHEDULE']
};

export function validateEvidenceAllowlistEligibility(
  kind: string,
  verification_status: string,
  target_offer_status: string,
  claim_type: string,
  document_type: string
): void {
  const tupleKey = `${kind}:${verification_status}:${target_offer_status}`;
  if (!PERMITTED_ALLOWLIST_TUPLES.has(tupleKey)) {
    throw new Error(`E-ERR-001: FAIL_CLOSED_DISALLOWED_TUPLE: Tuple (${tupleKey}) is not in the fail-closed allowlist`);
  }
  const allowedDocs = CLAIM_PERMITTED_DOCUMENT_TYPES[claim_type];
  if (!allowedDocs || !allowedDocs.includes(document_type)) {
    throw new Error(`E-ERR-033: EVIDENCE_CLAIM_TYPE_MISMATCH: Document type '${document_type}' is not permitted for claim '${claim_type}'`);
  }
}
```

3.2 Normative Claim Payload Mapping & Discriminated Union Authentication

```
export type AttesterRole =
  | 'PORTFOLIO_MANAGER'
  | 'COMMERCIAL_SOURCING_SPECIALIST'
  | 'WAREHOUSE_OPERATIONS_MANAGER'
  | 'SUPPLY_CHAIN_ANALYST'
  | 'COMPLIANCE_OFFICER'
  | 'LEGAL_COUNSEL'
  | 'RISK_OFFICER'
  | 'VERIFIED_EXTERNAL_ORACLE';

export interface RegisteredPrincipal {
  principal_id: string;               // e.g. "op:sec:shrom-principal-01"
  principal_type: 'HUMAN_OPERATOR' | 'CRYPTOGRAPHIC_ORACLE';
  assigned_roles: AttesterRole[];
  public_key_hex: string;             // 64-character lowercase hex Ed25519 public key
  valid_from_utc: string;             // ISO-8601 UTC
  valid_until_utc: string;           // ISO-8601 UTC
  revoked: boolean;
  revoked_at_utc?: string;
  compromised?: boolean;              // If true, retroactively invalidates all signatures
}

export interface RegisteredGateway {
  gateway_id: string;                 // e.g. "gw:auth:platform-core-01"
  public_key_hex: string;             // 64-character lowercase hex Ed25519 public key
  valid_from_utc: string;
  valid_until_utc: string;
  revoked: boolean;
  revoked_at_utc?: string;
  compromised?: boolean;
}

export interface GatewayClaimEndorsementReceipt {
  auth_gateway_id: string;            // Must resolve in GATEWAY_AUTHORITY_REGISTRY
  claim_signing_payload_sha256: string; // Exact SHA-256 of Canonical JCS ClaimSigningPayload
  domain_tag: 'GATEWAY_CLAIM_ENDORSEMENT';
  expires_at_utc: string;             // Explicit signed expiration (MUST BE EXACTLY issued_at_utc + 300,000 ms)
  issued_at_utc: string;              // Must be within 300s of attested_at_utc
  nonce: string;                      // Globally unique high-entropy nonce
  operator_id: string;                // Must match attester_id bitwise
  operator_role: AttesterRole;         // Must authorize the claim_type
  schema_version: 'POC-001/GATEWAY_ENDORSEMENT/v1';
  gateway_signature: string;          // 128-char hex Ed25519 signature over canonical endorsement fields (excluding gateway_signature)
}

export type ClaimAuthentication =
  | {
      auth_mechanism: 'DIRECT_KEY_SIGNATURE';
      attestation_signature: string;  // 128-char hex Ed25519 signature over ClaimSigningPayload
    }
  | {
      auth_mechanism: 'GATEWAY_CLAIM_ENDORSEMENT';
      endorsement_receipt: GatewayClaimEndorsementReceipt;
    };

export interface ClaimAttestation {
  evidence_id: string;
  evidence_sha256: string;
  attester_id: string;
  attested_at_utc: string;
  claimed_role: AttesterRole;
  epistemic_type: 'HUMAN_EXPERT_DOCUMENT_APPRAISAL' | 'AUTOMATED_CRYPTOGRAPHIC_EXTRACT';
  auth: ClaimAuthentication;
}

export interface SourceAuthenticityParameters {
  verification_mechanism: 'BUSINESS_REGISTRY_GSTIN_LOOKUP' | 'DOMAIN_CERTIFICATE_PKI' | 'PHYSICAL_AUDIT_ON_SITE';
}

export interface PriceConfirmationParameters {
  confirmed_unit_price: CanonicalMoney;
  price_evidence_kind: 'PROFORMA_INVOICE' | 'SIGNED_QUOTATION' | 'PUBLISHED_OFFICIAL_TARIFF';
}

export interface StockVerificationParameters {
  availability_evidence_kind: 'SUPPLIER_INVENTORY_REPORT' | 'FACTORY_LEAD_TIME_COMMITMENT';
  confirmed_quantity_available: number;
  inventory_location: string;
}

export type IncotermsCode = 'EXW' | 'FCA' | 'CPT' | 'CIP' | 'DAP' | 'DPU' | 'DDP' | 'FAS' | 'FOB' | 'CFR' | 'CIF';

export interface CommercialTermsParameters {
  incoterms: IncotermsCode;
  minimum_order_quantity: number;
  payment_terms_days: number;
  terms_evidence_kind: 'SUPPLIER_CONTRACT' | 'PROFORMA_INVOICE' | 'FORMAL_TERMS_SCHEDULE';
  valid_until_utc: string;
}

export interface VerificationClaim<TParams> {
  claim_type: 'SOURCE_AUTHENTICITY' | 'PRICE_CONFIRMATION' | 'STOCK_AVAILABILITY' | 'COMMERCIAL_TERMS';
  claim_parameters: TParams;
  attestation: ClaimAttestation;
```

```
}

export type AnyVerificationClaim =
  | VerificationClaim<SourceAuthenticityParameters>
  | VerificationClaim<PriceConfirmationParameters>
  | VerificationClaim<StockVerificationParameters>
  | VerificationClaim<CommercialTermsParameters>;

export interface VerificationClaimsAssessment {
  claim_source_authenticity: VerificationClaim<SourceAuthenticityParameters>;
  claim_price_confirmation: VerificationClaim<PriceConfirmationParameters>;
  claim_stock_availability: VerificationClaim<StockVerificationParameters>;
  claim_commercial_terms: VerificationClaim<CommercialTermsParameters>;
}

export interface ClaimSigningPayload {
  schema_version: 'POC-001/CLAIM_ATTESTATION/v1';
  domain_tag: 'CLAIM_ATTESTATION';
  offer_id: string;
  product_id: string;
  claim_type: 'SOURCE_AUTHENTICITY' | 'PRICE_CONFIRMATION' | 'STOCK_AVAILABILITY' | 'COMMERCIAL_TERMS';
  claimed_role: AttesterRole;         // Cryptographically bound inside signed bytes!
  evidence_id: string;
  evidence_sha256: string;
  attester_id: string;
  attested_at_utc: string;
  claim_parameters: Record<string, unknown>;
}

/**
 * Strict Input-Shape Validation Before Dereferencing (AUDIT-14)
 */
export function validateAssessmentStructure(assessment: unknown): asserts assessment is VerificationClaimsAssessment {
  if (!assessment || typeof assessment !== 'object') {
    throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Assessment payload must be a non-null object');
  }
  const obj = assessment as Record<string, unknown>;
  const requiredSlots = [
    'claim_source_authenticity',
    'claim_price_confirmation',
    'claim_stock_availability',
    'claim_commercial_terms'
  ];
  for (const slot of requiredSlots) {
    if (!obj[slot] || typeof obj[slot] !== 'object') {
      throw new Error(`E-ERR-024: INVALID_CLAIM_PARAMETERS: Missing or malformed required slot '${slot}'`);
    }
  }
  const allowedKeys = new Set(requiredSlots);
  for (const key of Object.keys(obj)) {
    if (!allowedKeys.has(key)) {
      throw new Error(`E-ERR-024: INVALID_CLAIM_PARAMETERS: Unexpected extraneous slot '${key}' in assessment`);
    }
  }

  // Validate internal slot types
  const src = obj.claim_source_authenticity as Record<string, unknown>;
  const prc = obj.claim_price_confirmation as Record<string, unknown>;
  const stk = obj.claim_stock_availability as Record<string, unknown>;
  const com = obj.claim_commercial_terms as Record<string, unknown>;

  if (src.claim_type !== 'SOURCE_AUTHENTICITY') throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid claim_type in claim_source_authenticity');
  if (prc.claim_type !== 'PRICE_CONFIRMATION') throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid claim_type in claim_price_confirmation');
  if (stk.claim_type !== 'STOCK_AVAILABILITY') throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid claim_type in claim_stock_availability');
  if (com.claim_type !== 'COMMERCIAL_TERMS') throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid claim_type in claim_commercial_terms');

  for (const item of [src, prc, stk, com]) {
    if (!item.attestation || typeof item.attestation !== 'object') {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Missing attestation object');
    }
    if (!item.claim_parameters || typeof item.claim_parameters !== 'object') {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Missing claim_parameters object');
    }
  }
}
```

Shared Strict Timestamp & Authority Validation Routines

```
export function validateIso8601UtcTimestamp(ts: string): void {
  const match = /^(\d{4})-(\d{2})-(\d{2})T(\d{2}):(\d{2}):(\d{2})(?:\.(\d{3}))?Z$/.exec(ts);
  if (!match) {
    throw new Error(`E-ERR-025: MALFORMED_TIMESTAMP: Non-canonical UTC ISO-8601 string: '${ts}'`);
  }
  const year = parseInt(match[1], 10);
  const month = parseInt(match[2], 10);
  const day = parseInt(match[3], 10);
  const hour = parseInt(match[4], 10);
  const minute = parseInt(match[5], 10);
  const second = parseInt(match[6], 10);

  if (year < 2000 || year > 2099) throw new Error(`E-ERR-025: MALFORMED_TIMESTAMP: Year out of bounds: ${year}`);
  if (month < 1 || month > 12) throw new Error(`E-ERR-025: MALFORMED_TIMESTAMP: Invalid month ${month}`);
  if (hour > 23 || minute > 59 || second > 59) throw new Error(`E-ERR-025: MALFORMED_TIMESTAMP: Invalid time ${hour}:${minute}:${second}`);

  const isLeap = (year % 4 === 0 && (year % 100 !== 0 || year % 400 === 0));
  const daysInMonth = [31, isLeap ? 29 : 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31];
  if (day < 1 || day > daysInMonth[month - 1]) {
    throw new Error(`E-ERR-025: MALFORMED_TIMESTAMP: Invalid calendar day ${day} for month ${month}/${year}`);
  }
}

/**
 * Shared Normative Gateway Receipt Validator (AUDIT-13, AUDIT-24)
 * Shared between receipt generation and promotion gate evaluation to prevent validation drift.
 */
export function validateGatewayEndorsementReceipt(
  rcpt: GatewayClaimEndorsementReceipt,
  current_time_utc: string,
  gateway_registry: Record<string, RegisteredGateway>
): void {
  validateIso8601UtcTimestamp(rcpt.issued_at_utc);
  validateIso8601UtcTimestamp(rcpt.expires_at_utc);
  validateIso8601UtcTimestamp(current_time_utc);

  const issued_ms = Date.parse(rcpt.issued_at_utc);
  const expires_ms = Date.parse(rcpt.expires_at_utc);
  const current_ms = Date.parse(current_time_utc);

  // 1. Exact 300,000 ms duration check
  if (expires_ms - issued_ms !== 300000) {
    throw new Error('E-ERR-027: INVALID_GATEWAY_RECEIPT_LIFETIME: Expiration interval must be exactly 300,000 ms (5 minutes)');
  }

  // 2. Strict Future clock-skew upper bound (5,000 ms policy tolerance, AUDIT-31)
  const MAX_FUTURE_CLOCK_SKEW_MS = 5000;
  if (issued_ms > current_ms + MAX_FUTURE_CLOCK_SKEW_MS) {
    throw new Error('E-ERR-007: CLOCK_SKEW_EXCEEDED: Gateway receipt issuance timestamp exceeds future tolerance (5,000 ms)');
  }

  // 3. Inclusive upper bound on expiry (current_ms <= expires_ms is valid; current_ms > expires_ms rejects)
  if (current_ms > expires_ms) {
    throw new Error('E-ERR-027: GATEWAY_RECEIPT_EXPIRED: Gateway receipt validity window has expired');
  }

  // 4. Nonce regex check
  if (!/^[a-zA-Z0-9_-]{16,64}$/.test(rcpt.nonce)) {
    throw new Error('E-ERR-024: INVALID_NONCE_FORMAT: Nonce must be 16-64 base64url/alphanumeric characters');
  }

  // 5. Gateway Authority Registry Pre-Validation
  const gateway = gateway_registry[rcpt.auth_gateway_id];
  if (!gateway) {
    throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION: Unregistered gateway ID');
  }
  validateIso8601UtcTimestamp(gateway.valid_from_utc);
  validateIso8601UtcTimestamp(gateway.valid_until_utc);
  const gw_from_ms = Date.parse(gateway.valid_from_utc);
  const gw_until_ms = Date.parse(gateway.valid_until_utc);
  if (issued_ms < gw_from_ms || issued_ms > gw_until_ms) {
    throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION: Gateway key not valid at receipt issuance time');
  }
  if (gateway.compromised) {
    throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION: Gateway key is compromised');
  }
  if (gateway.revoked) {
    if (!gateway.revoked_at_utc) throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION: Revoked gateway missing revocation timestamp');
    validateIso8601UtcTimestamp(gateway.revoked_at_utc);
    const gw_rev_ms = Date.parse(gateway.revoked_at_utc);
    if (issued_ms >= gw_rev_ms) {
      throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION: Gateway receipt issued at or after revocation timestamp');
    }
  }

  // 6. Cryptographic Ed25519 Signature Verification
  const { gateway_signature, ...unsigned_rcpt } = rcpt;
  const unsigned_bytes = Buffer.from(canonicalizeJCS(unsigned_rcpt), 'utf8');
  if (!ed25519Verify(unsigned_bytes, gateway.public_key_hex, gateway_signature)) {
    throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION: Gateway Ed25519 signature invalid');
  }
}

export function validateAttesterAuthority(
```

```
  attester_id: string,
  claimed_role: AttesterRole,
  claim_type: 'SOURCE_AUTHENTICITY' | 'PRICE_CONFIRMATION' | 'STOCK_AVAILABILITY' | 'COMMERCIAL_TERMS',
  attested_at_utc: string,
  current_time_utc: string
): RegisteredPrincipal {
  validateIso8601UtcTimestamp(attested_at_utc);
  validateIso8601UtcTimestamp(current_time_utc);

  const principal = PRINCIPAL_AUTHORITY_REGISTRY[attester_id];
  if (!principal) {
    throw new Error('E-ERR-018: INSUFFICIENT_ATTESTER_ROLE: Unregistered principal');
  }

  if (principal.principal_type === 'HUMAN_OPERATOR' && claimed_role === 'VERIFIED_EXTERNAL_ORACLE') {
    throw new Error('E-ERR-018: INSUFFICIENT_ATTESTER_ROLE: Human operator cannot assert external oracle role');
  }

  const required_roles: Record<string, AttesterRole[]> = {
    SOURCE_AUTHENTICITY: ['COMPLIANCE_OFFICER', 'PORTFOLIO_MANAGER', 'RISK_OFFICER'],
    PRICE_CONFIRMATION: ['PORTFOLIO_MANAGER', 'COMMERCIAL_SOURCING_SPECIALIST', 'VERIFIED_EXTERNAL_ORACLE'],
    STOCK_AVAILABILITY: ['PORTFOLIO_MANAGER', 'WAREHOUSE_OPERATIONS_MANAGER', 'SUPPLY_CHAIN_ANALYST', 'VERIFIED_EXTERNAL_ORACLE'],
    COMMERCIAL_TERMS: ['PORTFOLIO_MANAGER', 'COMMERCIAL_SOURCING_SPECIALIST', 'LEGAL_COUNSEL']
  };
  if (!principal.assigned_roles.includes(claimed_role) || !required_roles[claim_type].includes(claimed_role)) {
    throw new Error('E-ERR-018: INSUFFICIENT_ATTESTER_ROLE: Role not assigned or not authorized for claim type');
  }

  const attested_ms = Date.parse(attested_at_utc);
  const current_ms = Date.parse(current_time_utc);
  const from_ms = Date.parse(principal.valid_from_utc);
  const until_ms = Date.parse(principal.valid_until_utc);

  if (attested_ms > current_ms + 300000) {
    throw new Error('E-ERR-026: FUTURE_DATED_ATTESTATION: Attestation timestamp exceeds maximum allowable future clock skew (300s)');
  }
  if (attested_ms < from_ms || attested_ms > until_ms) {
    throw new Error('E-ERR-019: OPERATOR_KEY_REVOKED: Principal key validity window elapsed at attestation time');
  }

  if (principal.revoked) {
    if (principal.compromised) {
      throw new Error('E-ERR-019: OPERATOR_KEY_REVOKED: Principal key compromised (retroactive invalidation)');
    }
    const revoked_ms = Date.parse(principal.revoked_at_utc || '');
    if (!isNaN(revoked_ms) && attested_ms >= revoked_ms) {
      throw new Error('E-ERR-019: OPERATOR_KEY_REVOKED: Attestation signed at or after key revocation timestamp');
    }
  }

  return principal;
}
```

Strict Builder with Schema Validation & Exact Lifetime Check (AUDIT-13)

```
export function validateAndBuildClaimSigningPayload(
  offer_id: string,
  product_id: string,
  claim: AnyVerificationClaim
): ClaimSigningPayload {
  if (typeof offer_id !== 'string' || !offer_id.startsWith('off:') ||
      typeof product_id !== 'string' || !product_id.startsWith('prod:')) {
    throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Malformed offer_id or product_id');
  }

  validateIso8601UtcTimestamp(claim.attestation.attested_at_utc);
  if (!/^[a-f0-9]{64}$/.test(claim.attestation.evidence_sha256)) {
    throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Malformed evidence SHA-256');
  }

  // Exact 300-second lifetime enforcement on gateway endorsements (AUDIT-13)
  if (claim.attestation.auth.auth_mechanism === 'GATEWAY_CLAIM_ENDORSEMENT') {
    const rcpt = claim.attestation.auth.endorsement_receipt;
    validateIso8601UtcTimestamp(rcpt.issued_at_utc);
    validateIso8601UtcTimestamp(rcpt.expires_at_utc);
    const iss_ms = Date.parse(rcpt.issued_at_utc);
    const exp_ms = Date.parse(rcpt.expires_at_utc);
    if (exp_ms - iss_ms !== 300000) {
      throw new Error('E-ERR-027: INVALID_GATEWAY_RECEIPT_LIFETIME: Expiration interval must be exactly 300,000 ms (5 minutes)');
    }
  }

  let claim_parameters: Record<string, unknown>;

  if (claim.claim_type === 'SOURCE_AUTHENTICITY') {
    const params = claim.claim_parameters as SourceAuthenticityParameters;
    const validMechs = ['BUSINESS_REGISTRY_GSTIN_LOOKUP', 'DOMAIN_CERTIFICATE_PKI', 'PHYSICAL_AUDIT_ON_SITE'];
    if (!params.verification_mechanism || !validMechs.includes(params.verification_mechanism)) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid verification mechanism');
    }
    claim_parameters = { verification_mechanism: params.verification_mechanism };
  } else if (claim.claim_type === 'PRICE_CONFIRMATION') {
    const params = claim.claim_parameters as PriceConfirmationParameters;
    const validKinds = ['PROFORMA_INVOICE', 'SIGNED_QUOTATION', 'PUBLISHED_OFFICIAL_TARIFF'];
    if (!params.price_evidence_kind || !validKinds.includes(params.price_evidence_kind)) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid price evidence kind');
    }
    const p = params.confirmed_unit_price;
    if (!p || typeof p !== 'object') throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Missing confirmed_unit_price');
    const validatedMoney = createCanonicalMoney(p.amount_subunits, p.currency, p.scale);
    if (BigInt(validatedMoney.amount_subunits) <= 0n) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Price must be strictly positive');
    }
    claim_parameters = {
      confirmed_unit_price: validatedMoney,
      price_evidence_kind: params.price_evidence_kind
    };
  } else if (claim.claim_type === 'STOCK_AVAILABILITY') {
    const params = claim.claim_parameters as StockVerificationParameters;
    const validKinds = ['SUPPLIER_INVENTORY_REPORT', 'FACTORY_LEAD_TIME_COMMITMENT'];
    if (!params.availability_evidence_kind || !validKinds.includes(params.availability_evidence_kind)) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid stock availability evidence kind');
    }
    if (!Number.isSafeInteger(params.confirmed_quantity_available) || params.confirmed_quantity_available <= 0) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: confirmed_quantity_available must be positive safe integer');
    }
    claim_parameters = {
      availability_evidence_kind: params.availability_evidence_kind,
      confirmed_quantity_available: params.confirmed_quantity_available,
      inventory_location: typeof params.inventory_location === 'string' ? params.inventory_location : ''
    };
  } else if (claim.claim_type === 'COMMERCIAL_TERMS') {
    const params = claim.claim_parameters as CommercialTermsParameters;
    const validInco: IncotermsCode[] = ['EXW', 'FCA', 'CPT', 'CIP', 'DAP', 'DPU', 'DDP', 'FAS', 'FOB', 'CFR', 'CIF'];
    if (!validInco.includes(params.incoterms)) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid Incoterms code');
    }
    if (!Number.isSafeInteger(params.minimum_order_quantity) || params.minimum_order_quantity <= 0) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: minimum_order_quantity must be positive safe integer');
    }
    if (!Number.isSafeInteger(params.payment_terms_days) || params.payment_terms_days < 0) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: payment_terms_days must be non-negative safe integer');
    }
    const validKinds = ['SUPPLIER_CONTRACT', 'PROFORMA_INVOICE', 'FORMAL_TERMS_SCHEDULE'];
    if (!params.terms_evidence_kind || !validKinds.includes(params.terms_evidence_kind)) {
      throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Invalid terms evidence kind');
    }
    validateIso8601UtcTimestamp(params.valid_until_utc);
    claim_parameters = {
      incoterms: params.incoterms,
      minimum_order_quantity: params.minimum_order_quantity,
      payment_terms_days: params.payment_terms_days,
      terms_evidence_kind: params.terms_evidence_kind,
      valid_until_utc: params.valid_until_utc
    };
  } else {
    throw new Error('E-ERR-024: INVALID_CLAIM_PARAMETERS: Unknown claim type');
  }
```

```
  return {
    schema_version: 'POC-001/CLAIM_ATTESTATION/v1',
    domain_tag: 'CLAIM_ATTESTATION',
    offer_id,
    product_id,
    claim_type: claim.claim_type,
    claimed_role: claim.attestation.claimed_role,
    evidence_id: claim.attestation.evidence_id,
    evidence_sha256: claim.attestation.evidence_sha256,
    attester_id: claim.attestation.attester_id,
    attested_at_utc: claim.attestation.attested_at_utc,
    claim_parameters
  };
}
```

3.3 Normative Four-Claim Promotion Gate & Unified Atomic Persistence (AUDIT-12, AUDIT-14)

```
export interface CommercialReconciliationResult {
  reconciled: boolean;
  assigned_offer_status: CommercialOfferStatus;
  assigned_verification_status: VerificationStatus;
  rejection_reason?: string;
}

export function reconcileOfferClaims(
  offer: CanonicalOffer,
  assessment: VerificationClaimsAssessment,
  resolved_evidence: Record<string, ResolvedEvidenceEnvelope>,
  current_time_utc: string
): CommercialReconciliationResult {
  const current_ms = Date.parse(current_time_utc);

  // 1. Commercial Terms Reconciliation & Observation Freshness
  const terms = assessment.claim_commercial_terms.claim_parameters;
  const terms_ev = resolved_evidence[assessment.claim_commercial_terms.attestation.evidence_id];
  validateIso8601UtcTimestamp(terms.valid_until_utc);
  const valid_until_ms = Date.parse(terms.valid_until_utc);
  if (current_ms > valid_until_ms) {
    return { reconciled: false, assigned_offer_status: 'EXPIRED_OFFER', assigned_verification_status: 'STALE', rejection_reason: 'Commercial terms validity expired' };
  }
  validateIso8601UtcTimestamp(terms_ev.provenance.observed_at_utc);
  const terms_obs_ms = Date.parse(terms_ev.provenance.observed_at_utc);
  if (terms_obs_ms > current_ms + 300000) throw new Error('E-ERR-007: CLOCK_SKEW_EXCEEDED');
  if (current_ms - terms_obs_ms > 604800000) {
    return { reconciled: false, assigned_offer_status: 'INDICATIVE_QUOTE', assigned_verification_status: 'PARTIALLY_VERIFIED', rejection_reason: 'Commercial terms evidence observation exceeds 7-day f
  }
  if (terms.minimum_order_quantity !== offer.minimum_order_quantity) throw new Error('E-ERR-029: OFFER_MOQ_CLAIM_MISMATCH');
  if (terms.payment_terms_days !== offer.payment_terms_days) throw new Error('E-ERR-030: OFFER_PAYMENT_TERMS_MISMATCH');
  if (terms.incoterms !== offer.incoterms) throw new Error('E-ERR-031: OFFER_INCOTERMS_MISMATCH');

  // 2. Price Confirmation Reconciliation & Freshness
  const price = assessment.claim_price_confirmation.claim_parameters.confirmed_unit_price;
  if (price.currency !== offer.unit_price.currency ||
      price.scale !== offer.unit_price.scale ||
      BigInt(price.amount_subunits) !== BigInt(offer.unit_price.amount_subunits)) {
    throw new Error('E-ERR-028: OFFER_PRICE_CLAIM_MISMATCH');
  }
  const price_ev = resolved_evidence[assessment.claim_price_confirmation.attestation.evidence_id];
  const price_ts = (price_ev.provenance.is_authenticated_source_feed && price_ev.provenance.occurred_at_raw)
    ? price_ev.provenance.occurred_at_raw
    : price_ev.provenance.observed_at_utc;
  validateIso8601UtcTimestamp(price_ts);
  const price_ms = Date.parse(price_ts);
  if (price_ms > current_ms + 300000) throw new Error('E-ERR-007: CLOCK_SKEW_EXCEEDED');
  if (current_ms - price_ms > 604800000) {
    return { reconciled: false, assigned_offer_status: 'INDICATIVE_QUOTE', assigned_verification_status: 'PARTIALLY_VERIFIED', rejection_reason: 'Price confirmation exceeds 7-day freshness window' };
  }

  // 3. Stock Availability Reconciliation & Freshness
  const stock = assessment.claim_stock_availability.claim_parameters;
  const stock_ev = resolved_evidence[assessment.claim_stock_availability.attestation.evidence_id];
  validateIso8601UtcTimestamp(stock_ev.provenance.observed_at_utc);
  const stock_ms = Date.parse(stock_ev.provenance.observed_at_utc);
  if (stock_ms > current_ms + 300000) throw new Error('E-ERR-007: CLOCK_SKEW_EXCEEDED');
  if (current_ms - stock_ms > 172800000) {
    return { reconciled: false, assigned_offer_status: 'INDICATIVE_QUOTE', assigned_verification_status: 'PARTIALLY_VERIFIED', rejection_reason: 'Stock availability exceeds 48-hour freshness window'
  }
  if (stock.confirmed_quantity_available < offer.minimum_order_quantity) {
    return { reconciled: false, assigned_offer_status: 'INDICATIVE_QUOTE', assigned_verification_status: 'PARTIALLY_VERIFIED', rejection_reason: 'Confirmed stock quantity below offer MOQ' };
  }

  // 4. Source Authenticity Freshness
  const src_ev = resolved_evidence[assessment.claim_source_authenticity.attestation.evidence_id];
  validateIso8601UtcTimestamp(src_ev.provenance.observed_at_utc);
  const src_ms = Date.parse(src_ev.provenance.observed_at_utc);
  if (src_ms > current_ms + 300000) throw new Error('E-ERR-007: CLOCK_SKEW_EXCEEDED');
  if (current_ms - src_ms > 2592000000) {
    return { reconciled: false, assigned_offer_status: 'INDICATIVE_QUOTE', assigned_verification_status: 'PARTIALLY_VERIFIED', rejection_reason: 'Source authenticity evidence exceeds 30-day freshness
  }

  return { reconciled: true, assigned_offer_status: 'VERIFIED_EXECUTABLE_OFFER', assigned_verification_status: 'VERIFIED' };
}

export interface ClaimEvidenceReference {
  claim_type: 'SOURCE_AUTHENTICITY' | 'PRICE_CONFIRMATION' | 'STOCK_AVAILABILITY' | 'COMMERCIAL_TERMS';
  evidence_id: string;
  evidence_sha256: string;
  parameters_canonical_sha256: string;
  attester_id: string;
  claimed_role: AttesterRole;
  auth_mechanism: 'DIRECT_KEY_SIGNATURE' | 'GATEWAY_CLAIM_ENDORSEMENT';
  gateway_id?: string;
  gateway_receipt_sha256?: string;
  gateway_nonce?: string;
}

/**
 * Strict Canonical Stream Sequence Brand & Validator (AUDIT-42)
 * Prohibits leading zeros ('01' vs '1'), negative numbers, and non-numeric representations.
 */
export type CanonicalStreamSequence = string & { readonly __brand: unique symbol };
```

```
export function parseCanonicalStreamSequence(raw: string): CanonicalStreamSequence {
  if (typeof raw !== 'string' || !/^(0|[1-9]\d*)$/.test(raw)) {
    throw new Error(`E-ERR-024: INVALID_SEQUENCE_FORMAT: Sequence must be a non-negative decimal string without leading zeros, got '${raw}'`);
  }
  const val = BigInt(raw);
  const maxUint64 = 18446744073709551615n;
  if (val > maxUint64) {
    throw new Error('E-ERR-014: ARITHMETIC_OVERFLOW: Sequence exceeds uint64 maximum boundary');
  }
  return raw as CanonicalStreamSequence;
}

export interface PromotionEvaluationDraft {
  schema_version: 'POC-001/PROMOTION_EVALUATION_DRAFT/v1';
  offer_id: string;
  expected_stream_sequence: CanonicalStreamSequence;
  assessment_id: string;
  assessment_sha256: string;
  evaluator_auth_token: string; // HMAC-SHA256 proving draft was generated by trusted evaluator (AUDIT-41, AUDIT-46)
  claims_evidence_summary: Record<string, ClaimEvidenceReference>;
  policy_version: 'POC-001/PROMOTION_POLICY/v1';
  evaluator_version: 'POC-001/EVALUATOR/v1';
  evaluated_at_utc: string;
  promoted: boolean;
  assigned_offer_status: CommercialOfferStatus;
  assigned_verification_status: VerificationStatus;
  rejection_reason?: string;
  nonces_to_consume: { nonce: string; expires_at_utc: string; gateway_id: string }[];
}

/**
 * Sealed Validated Promotion Evaluation Snapshot (AUDIT-41, AUDIT-47)
 * Holds the exact canonical assessment bytes produced during single canonicalization,
 * plus retained unmanipulated claim objects for commit-time cryptographic authority revalidation.
 */
export interface ValidatedPromotionEvaluation {
  readonly draft: Readonly<PromotionEvaluationDraft>;
  readonly raw_claims: Readonly<Record<string, AnyVerificationClaim>>;
  readonly canonical_assessment_bytes: Buffer;
  readonly assessment_sha256: string;
  readonly validated_at_utc: string;
}

/**
 * System Trusted Clock Interface (AUDIT-39, AUDIT-43, AUDIT-49)
 * Injected at runtime to provide monotonic, kernel-verified UTC timestamps.
 * Throws E-ERR-007: TRUSTED_CLOCK_UNAVAILABLE fail-closed if clock source is unreachable,
 * desynchronized (> 5,000 ms skew against trusted NTP/PTP reference), or non-monotonic.
 */
export interface TrustedClock {
  nowUtc(): string;
}

/**
 * Co-located Relational Transaction Interface (AUDIT-36, AUDIT-37, AUDIT-42)
 * Guarantees that stream locking, nonce consumption with gateway ID, canonical assessment
 * storage, offer stream event append, and stream head update commit within the single transaction.
 */
export interface OfferPersistenceDatabaseTransaction {
  getOfferStreamHeadForUpdate(offer_id: string): Promise<{ sequence_number: CanonicalStreamSequence; offer_status: CommercialOfferStatus }>;
  insertNonceIfUnused(nonce: string, gateway_id: string, expires_at_utc: string, consumed_at_utc: string): Promise<boolean>;
  putAssessment(assessment_sha256: string, canonical_bytes: Buffer, created_at_utc: string): Promise<void>;
  getAssessment(assessment_sha256: string): Promise<Buffer | null>;
  appendOfferStreamEvent(event: {
    stream_id: string;
    sequence_number: CanonicalStreamSequence;
    payload_kind: string;
    payload: OfferPromotionEventPayload;
    event_sha256: string;
  }): Promise<void>;
  updateOfferStreamHead(offer_id: string, resulting_sequence_number: CanonicalStreamSequence, resulting_status: CommercialOfferStatus, updated_at_utc: string): Promise<void>;
  getOfferStreamEvent(stream_id: string, sequence_number: CanonicalStreamSequence): Promise<{
    stream_id: string;
    sequence_number: CanonicalStreamSequence;
    payload_kind: string;
    payload: OfferPromotionEventPayload;
    event_sha256: string;
    created_at_utc: string;
  } | null>;
}

export interface OfferPersistenceDatabaseSession {
  executeTransaction<T>(fn: (tx: OfferPersistenceDatabaseTransaction) => Promise<T>): Promise<T>;
  getAssessment(assessment_sha256: string): Promise<Buffer | null>;
}

export const PROMOTION_POLICY_REGISTRY: Record<string, { policy_version: string; allowlist_version: string }> = {
  'POC-001/PROMOTION_POLICY/v1': { policy_version: 'POC-001/PROMOTION_POLICY/v1', allowlist_version: '18-TUPLE-ALLOWLIST/v1' }
};

export const EVALUATOR_REGISTRY: Record<string, { evaluator_version: string; engine_release: string; service_key_id: string }> = {
  'POC-001/EVALUATOR/v1': { evaluator_version: 'POC-001/EVALUATOR/v1', engine_release: 'GATE-M1-EVAL-v1.13', service_key_id: 'op:sec:evaluator-service-01' }
};

/**
 * Deep Freeze Utility for Immutability Snapshots (AUDIT-35, AUDIT-41)
```

```
 */
function deepFreeze<T>(obj: T): Readonly<T> {
  if (obj && typeof obj === 'object' && !Object.isFrozen(obj)) {
    Object.freeze(obj);
    for (const key of Object.getOwnPropertyNames(obj)) {
      deepFreeze((obj as Record<string, unknown>)[key]);
    }
  }
  return obj;
}

/**
 * Canonical Evaluator Decision Signing Payload (AUDIT-46)
 * Covers the entire security-relevant decision state under domain separator POC-001/EVALUATOR_DECISION_AUTH/v1.
 * Serialized strictly via RFC 8785 JCS prior to HMAC-SHA256 generation.
 */
export interface EvaluatorDecisionSigningPayload {
  domain_tag: 'POC-001/EVALUATOR_DECISION_AUTH/v1';
  offer_id: string;
  expected_stream_sequence: CanonicalStreamSequence;
  assessment_id: string;
  assessment_sha256: string;
  policy_version: string;
  evaluator_version: string;
  promoted: boolean;
  assigned_offer_status: CommercialOfferStatus;
  assigned_verification_status: VerificationStatus;
  rejection_reason?: string;
  claims_evidence_summary: Record<string, ClaimEvidenceReference>;
  nonces_to_consume: { nonce: string; expires_at_utc: string; gateway_id: string }[];
  evaluated_at_utc: string;
}

/**
 * Trusted Evaluator Token Verifier Interface (AUDIT-46)
 * Encapsulates the evaluator secret within the trusted service boundary.
 * The verifier reconstructs the canonical RFC 8785 payload, recomputes HMAC-SHA256,
 * and performs constant-time byte comparison using crypto.timingSafeEqual.
 */
export interface EvaluatorTokenVerifier {
  verifyEvaluatorToken(draft: PromotionEvaluationDraft): boolean;
}

export class ServiceBoundaryEvaluatorTokenVerifier implements EvaluatorTokenVerifier {
  private readonly secret_buffer: Buffer;

  constructor(evaluator_secret_hex: string) {
    if (!evaluator_secret_hex || !/^[0-9a-fA-F]{64}$/.test(evaluator_secret_hex)) {
      throw new Error('E-ERR-017: INVALID_EVALUATOR_SECRET: Secret must be a 64-character (256-bit) hex string');
    }
    this.secret_buffer = Buffer.from(evaluator_secret_hex, 'hex');
  }

  public verifyEvaluatorToken(draft: PromotionEvaluationDraft): boolean {
    if (!draft.evaluator_auth_token || !/^[0-9a-fA-F]{64}$/.test(draft.evaluator_auth_token)) {
      return false;
    }

    const signingPayload: EvaluatorDecisionSigningPayload = {
      domain_tag: 'POC-001/EVALUATOR_DECISION_AUTH/v1',
      offer_id: draft.offer_id,
      expected_stream_sequence: draft.expected_stream_sequence,
      assessment_id: draft.assessment_id,
      assessment_sha256: draft.assessment_sha256,
      policy_version: draft.policy_version,
      evaluator_version: draft.evaluator_version,
      promoted: draft.promoted,
      assigned_offer_status: draft.assigned_offer_status,
      assigned_verification_status: draft.assigned_verification_status,
      ...(draft.rejection_reason !== undefined ? { rejection_reason: draft.rejection_reason } : {}),
      claims_evidence_summary: draft.claims_evidence_summary,
      nonces_to_consume: draft.nonces_to_consume,
      evaluated_at_utc: draft.evaluated_at_utc
    };

    const canonical_bytes = Buffer.from(canonicalizeJCS(signingPayload), 'utf8');
    const expected_token_hex = hmacSha256(this.secret_buffer, canonical_bytes);

    const expected_buf = Buffer.from(expected_token_hex, 'hex');
    const actual_buf = Buffer.from(draft.evaluator_auth_token, 'hex');
    if (expected_buf.length !== actual_buf.length) {
      return false;
    }
    return crypto.timingSafeEqual(expected_buf, actual_buf);
  }
}

/**
 * Generate HMAC-SHA256 Authenticity Token for PromotionEvaluationDraft (AUDIT-41, AUDIT-46)
 * Authenticates the complete decision payload under domain tag POC-001/EVALUATOR_DECISION_AUTH/v1 via RFC 8785 JCS.
 */
export function generateEvaluatorDecisionAuthToken(
  evaluator_secret_hex: string,
  signing_payload: EvaluatorDecisionSigningPayload
): string {
  const canonical_payload_json = canonicalizeJCS(signing_payload);
  return hmacSha256(Buffer.from(evaluator_secret_hex, 'hex'), Buffer.from(canonical_payload_json, 'utf8'));
```

```
}

/**
 * Complete Runtime Integrity Validator for PromotionEvaluationDraft (AUDIT-21, AUDIT-27, AUDIT-35, AUDIT-41, AUDIT-46)
 * 1. Validates raw_assessment complete structure before dereferencing.
 * 2. Canonicalizes assessment exactly once, computes SHA-256 digest, and retains immutable bytes.
 * 3. Cryptographically verifies evaluator_auth_token via injected EvaluatorTokenVerifier using timingSafeEqual.
 * 4. Unpacks the 4 mandatory claims, verifies parameter digests, compares all fields, and enforces 1-to-1 nonces.
 * 5. Deep-freezes assessment, claims, and draft, returning sealed ValidatedPromotionEvaluation.
 */
export function validateEvaluationDraftIntegrity(
  draft: PromotionEvaluationDraft,
  offer: CanonicalOffer,
  expected_sequence: string,
  raw_assessment: unknown,
  token_verifier: EvaluatorTokenVerifier,
  trusted_time_utc: string
): ValidatedPromotionEvaluation {
  if (!draft || typeof draft !== 'object') {
    throw new Error('E-ERR-024: INVALID_DRAFT: Draft must be a non-null object');
  }
  if (draft.schema_version !== 'POC-001/PROMOTION_EVALUATION_DRAFT/v1') {
    throw new Error('E-ERR-024: INVALID_DRAFT: Unsupported draft schema version');
  }
  if (draft.offer_id !== offer.offer_id) {
    throw new Error(`E-ERR-024: INVALID_DRAFT: Draft offer_id '${draft.offer_id}' does not match target offer '${offer.offer_id}'`);
  }

  // Canonical decimal sequence validation (AUDIT-42)
  const canonical_expected_seq = parseCanonicalStreamSequence(expected_sequence);
  const canonical_draft_seq = parseCanonicalStreamSequence(draft.expected_stream_sequence);
  if (BigInt(canonical_draft_seq) !== BigInt(canonical_expected_seq)) {
    throw new Error(`E-ERR-024: INVALID_DRAFT: Draft expected sequence '${draft.expected_stream_sequence}' does not match requested sequence '${expected_sequence}'`);
  }

  validateIso8601UtcTimestamp(draft.evaluated_at_utc);
  validateIso8601UtcTimestamp(trusted_time_utc);

  // 1. Structural Pre-Validation Before Field Dereferencing (AUDIT-41)
  validateAssessmentStructure(raw_assessment);

  // 2. Single Canonicalization & Digest Verification (AUDIT-41)
  const canonical_assessment_bytes = Buffer.from(canonicalizeJCS(raw_assessment), 'utf8');
  const computed_assessment_sha256 = sha256(canonical_assessment_bytes);
  if (draft.assessment_sha256 !== computed_assessment_sha256) {
    throw new Error('E-ERR-024: INVALID_DRAFT: Draft assessment_sha256 does not match canonical assessment digest');
  }

  // 3. Cryptographic Verification of Evaluator Authenticity Token via Injected Verifier (AUDIT-41, AUDIT-46)
  if (!token_verifier.verifyEvaluatorToken(draft)) {
    throw new Error('E-ERR-017: INVALID_EVALUATOR_AUTH_TOKEN: Draft does not bear a valid signature from trusted promotion evaluator');
  }

  // 4. Validate Registries
  if (!PROMOTION_POLICY_REGISTRY[draft.policy_version]) {
    throw new Error(`E-ERR-024: INVALID_DRAFT: Unregistered policy_version '${draft.policy_version}'`);
  }
  if (!EVALUATOR_REGISTRY[draft.evaluator_version]) {
    throw new Error(`E-ERR-024: INVALID_DRAFT: Unregistered evaluator_version '${draft.evaluator_version}'`);
  }

  // 5. Unpack and Validate the 4 Required Assessment Claims (AUDIT-35)
  const rawObj = raw_assessment as Record<string, unknown>;
  const assessmentClaims: Record<string, AnyVerificationClaim> = {
    SOURCE_AUTHENTICITY: rawObj.claim_source_authenticity as AnyVerificationClaim,
    PRICE_CONFIRMATION: rawObj.claim_price_confirmation as AnyVerificationClaim,
    STOCK_AVAILABILITY: rawObj.claim_stock_availability as AnyVerificationClaim,
    COMMERCIAL_TERMS: rawObj.claim_commercial_terms as AnyVerificationClaim
  };

  const requiredClaims = ['SOURCE_AUTHENTICITY', 'PRICE_CONFIRMATION', 'STOCK_AVAILABILITY', 'COMMERCIAL_TERMS'];
  const summaryKeys = Object.keys(draft.claims_evidence_summary);
  if (summaryKeys.length !== 4) {
    throw new Error('E-ERR-024: INVALID_DRAFT: claims_evidence_summary must contain exactly 4 claim references');
  }

  const expectedNonces: { nonce: string; gateway_id: string; expires_at_utc: string }[] = [];
  const allowedRefKeys = [
    'claim_type', 'evidence_id', 'evidence_sha256', 'parameters_canonical_sha256',
    'attester_id', 'claimed_role', 'auth_mechanism', 'gateway_id', 'gateway_receipt_sha256', 'gateway_nonce'
  ];

  for (const c of requiredClaims) {
    const claim = assessmentClaims[c];
    if (!claim || typeof claim !== 'object' || claim.claim_type !== c) {
      throw new Error(`E-ERR-024: INVALID_DRAFT: Missing or malformed claim '${c}' in raw_assessment`);
    }

    const ref = draft.claims_evidence_summary[c];
    if (!ref || typeof ref !== 'object') {
      throw new Error(`E-ERR-024: INVALID_DRAFT: Missing claim evidence reference for '${c}' in draft summary`);
    }

    // Verify no extraneous keys exist on ref (AUDIT-35)
    for (const k of Object.keys(ref)) {
      if (!allowedRefKeys.includes(k) && (ref as unknown as Record<string, unknown>)[k] !== undefined) {
        throw new Error(`E-ERR-024: INVALID_DRAFT: Extraneous key '${k}' in claims_evidence_summary['${c}']`);
```

```
      }
    }

    // Recompute canonical parameter digest directly from assessment claim parameters
    const params_canonical_bytes = Buffer.from(canonicalizeJCS(claim.claim_parameters), 'utf8');
    const expected_params_sha256 = sha256(params_canonical_bytes);

    // Strict field-by-field equality check against derived assessment claim
    if (ref.claim_type !== c) throw new Error(`E-ERR-024: INVALID_DRAFT: claim_type mismatch in summary for '${c}'`);
    if (ref.evidence_id !== claim.attestation.evidence_id) throw new Error(`E-ERR-024: INVALID_DRAFT: evidence_id mismatch in summary for '${c}'`);
    if (ref.evidence_sha256 !== claim.attestation.evidence_sha256) throw new Error(`E-ERR-024: INVALID_DRAFT: evidence_sha256 mismatch in summary for '${c}'`);
    if (ref.parameters_canonical_sha256 !== expected_params_sha256) throw new Error(`E-ERR-024: INVALID_DRAFT: parameters_canonical_sha256 mismatch in summary for '${c}'`);
    if (ref.attester_id !== claim.attestation.attester_id) throw new Error(`E-ERR-024: INVALID_DRAFT: attester_id mismatch in summary for '${c}'`);
    if (ref.claimed_role !== claim.attestation.claimed_role) throw new Error(`E-ERR-024: INVALID_DRAFT: claimed_role mismatch in summary for '${c}'`);
    if (ref.auth_mechanism !== claim.attestation.auth.auth_mechanism) throw new Error(`E-ERR-024: INVALID_DRAFT: auth_mechanism mismatch in summary for '${c}'`);

    if (!['DIRECT_KEY_SIGNATURE', 'GATEWAY_CLAIM_ENDORSEMENT'].includes(ref.auth_mechanism)) {
      throw new Error(`E-ERR-017: INVALID_AUTH_MECHANISM: Unsupported auth_mechanism '${ref.auth_mechanism}'`);
    }

    if (claim.attestation.auth.auth_mechanism === 'GATEWAY_CLAIM_ENDORSEMENT') {
      const rcpt = claim.attestation.auth.endorsement_receipt;
      if (!rcpt || typeof rcpt !== 'object') {
        throw new Error(`E-ERR-017: INVALID_RECEIPT: Missing endorsement receipt in gateway claim '${c}'`);
      }
      const rcpt_bytes = Buffer.from(canonicalizeJCS(rcpt), 'utf8');
      const expected_rcpt_sha256 = sha256(rcpt_bytes);

      if (ref.gateway_id !== rcpt.auth_gateway_id) throw new Error(`E-ERR-024: INVALID_DRAFT: gateway_id mismatch in summary for '${c}'`);
      if (ref.gateway_receipt_sha256 !== expected_rcpt_sha256) throw new Error(`E-ERR-024: INVALID_DRAFT: gateway_receipt_sha256 mismatch in summary for '${c}'`);
      if (ref.gateway_nonce !== rcpt.nonce) throw new Error(`E-ERR-024: INVALID_DRAFT: gateway_nonce mismatch in summary for '${c}'`);

      expectedNonces.push({ nonce: rcpt.nonce, gateway_id: rcpt.auth_gateway_id, expires_at_utc: rcpt.expires_at_utc });
    } else if (claim.attestation.auth.auth_mechanism === 'DIRECT_KEY_SIGNATURE') {
      if (ref.gateway_id !== undefined || ref.gateway_receipt_sha256 !== undefined || ref.gateway_nonce !== undefined) {
        throw new Error(`E-ERR-024: INVALID_DRAFT: Direct key signature claim '${c}' cannot contain gateway fields`);
      }
    }
  }

  // 6. Verify Gateway Nonce Uniqueness across Claims (AUDIT-35)
  const uniqueNonces = new Set(expectedNonces.map(n => n.nonce));
  if (uniqueNonces.size !== expectedNonces.length) {
    throw new Error('E-ERR-024: DUPLICATE_NONCE_IN_ASSESSMENT: Duplicate gateway nonce detected across claims');
  }

  // 7. Invariant Evaluation: Success vs Rejection States & Nonce Bijection
  if (draft.promoted) {
    if (draft.assigned_offer_status !== 'VERIFIED_EXECUTABLE_OFFER' || draft.assigned_verification_status !== 'VERIFIED') {
      throw new Error('E-ERR-024: INVALID_DRAFT: Contradictory status: promoted=true requires VERIFIED_EXECUTABLE_OFFER and VERIFIED');
    }
    if (draft.rejection_reason !== undefined) {
      throw new Error('E-ERR-024: INVALID_DRAFT: Contradictory draft: promoted=true cannot contain rejection_reason');
    }

    // Verify 1-to-1 bijection between derived gateway nonces and draft.nonces_to_consume
    if (draft.nonces_to_consume.length !== expectedNonces.length) {
      throw new Error(`E-ERR-024: INVALID_DRAFT: nonces_to_consume count (${draft.nonces_to_consume.length}) does not match derived gateway claims (${expectedNonces.length})`);
    }
    const derivedNonceMap = new Map(expectedNonces.map(n => [n.nonce, n]));
    for (const item of draft.nonces_to_consume) {
      const derived = derivedNonceMap.get(item.nonce);
      if (!derived || derived.gateway_id !== item.gateway_id || derived.expires_at_utc !== item.expires_at_utc) {
        throw new Error(`E-ERR-024: INVALID_DRAFT: Unbound or mismatched nonce '${item.nonce}' in nonces_to_consume`);
      }
      validateIso8601UtcTimestamp(item.expires_at_utc);
    }
  } else {
    if (draft.assigned_offer_status === 'VERIFIED_EXECUTABLE_OFFER' || draft.assigned_verification_status === 'VERIFIED') {
      throw new Error('E-ERR-024: INVALID_DRAFT: Contradictory status: promoted=false cannot assign VERIFIED statuses');
    }
    if (draft.nonces_to_consume.length > 0) {
      throw new Error('E-ERR-024: INVALID_DRAFT: Contradictory draft: promoted=false must have empty nonces_to_consume');
    }
    if (typeof draft.rejection_reason !== 'string' || draft.rejection_reason.trim().length === 0) {
      throw new Error('E-ERR-024: INVALID_DRAFT: Contradictory draft: promoted=false requires non-empty rejection_reason string');
    }
  }

  // 8. Single Canonicalization Snapshot Retention & Deep Freeze (AUDIT-35, AUDIT-41, AUDIT-47)
  return deepFreeze({
    draft,
    raw_claims: assessmentClaims,
    canonical_assessment_bytes,
    assessment_sha256: computed_assessment_sha256,
    validated_at_utc: trusted_time_utc
  });
}

function validateAssessmentStructure(assessment: unknown): void {
  if (!assessment || typeof assessment !== 'object') {
    throw new Error('E-ERR-024: INVALID_ASSESSMENT: Assessment must be a non-null object');
  }
  const obj = assessment as Record<string, unknown>;
  const allowedKeys = [
    'claim_source_authenticity',
    'claim_price_confirmation',
```

```
    'claim_stock_availability',
    'claim_commercial_terms'
  ];

  for (const k of Object.keys(obj)) {
    if (!allowedKeys.includes(k)) {
      throw new Error(`E-ERR-024: INVALID_ASSESSMENT: Unexpected property '${k}' in assessment envelope`);
    }
  }

  for (const k of allowedKeys) {
    if (!obj[k] || typeof obj[k] !== 'object') {
      throw new Error(`E-ERR-024: INVALID_ASSESSMENT: Missing mandatory claim slot '${k}' in assessment`);
    }
  }
}

/**
 * Pure Read-Only Promotion Evaluator (Phase 1)
 * Zero nonces consumed; zero stream mutations.
 * Signs output draft with RFC 8785 canonical EvaluatorDecisionSigningPayload HMAC (AUDIT-41, AUDIT-46).
 */
export function evaluateOfferPromotionReadOnly(
  offer: CanonicalOffer,
  assessment: unknown,
  expected_stream_sequence: string,
  evaluator_secret_hex: string,
  current_time_utc: string
): PromotionEvaluationDraft {
  const canonical_seq = parseCanonicalStreamSequence(expected_stream_sequence);
  validateAssessmentStructure(assessment);
  validateIso8601UtcTimestamp(current_time_utc);

  const assessment_bytes = Buffer.from(canonicalizeJCS(assessment), 'utf8');
  const assessment_sha256 = sha256(assessment_bytes);
  const assessment_id = `asmt:${offer.offer_id}:${assessment_sha256.substring(0, 16)}`;

  const claims: AnyVerificationClaim[] = [
    (assessment as Record<string, AnyVerificationClaim>).claim_source_authenticity,
    (assessment as Record<string, AnyVerificationClaim>).claim_price_confirmation,
    (assessment as Record<string, AnyVerificationClaim>).claim_stock_availability,
    (assessment as Record<string, AnyVerificationClaim>).claim_commercial_terms
  ];

  const resolved_evidence: Record<string, ResolvedEvidenceEnvelope> = {};
  const claims_evidence_summary: Record<string, ClaimEvidenceReference> = {};
  const nonces_to_consume: { nonce: string; expires_at_utc: string; gateway_id: string }[] = [];

  for (const claim of claims) {
    const payload = validateAndBuildClaimSigningPayload(offer.offer_id, offer.product_id, claim);
    const payload_bytes = Buffer.from(canonicalizeJCS(payload), 'utf8');
    const payload_sha256 = sha256(payload_bytes);
    const params_sha256 = sha256(Buffer.from(canonicalizeJCS(payload.claim_parameters), 'utf8'));

    const ev = resolveStreamEvidence(claim.attestation.evidence_id);
    if (!ev || ev.event_sha256 !== claim.attestation.evidence_sha256) {
      throw new Error('E-ERR-012: UNRESOLVABLE_EVIDENCE_REFERENCE');
    }
    resolved_evidence[claim.claim_type] = ev;

    let claim_ref: ClaimEvidenceReference;

    if (claim.attestation.auth.auth_mechanism === 'DIRECT_KEY_SIGNATURE') {
      const sig = claim.attestation.auth.direct_signature;
      if (!sig || !/^[0-9a-fA-F]{128}$/.test(sig)) {
        throw new Error('E-ERR-017: INVALID_SIGNATURE_FORMAT');
      }

      const principal = validateAttesterAuthority(
        claim.attestation.attester_id,
        claim.attestation.claimed_role,
        claim.claim_type,
        claim.attestation.attested_at_utc,
        current_time_utc
      );

      if (!ed25519Verify(payload_bytes, principal.public_key_hex, sig)) {
        throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION');
      }

      claim_ref = {
        claim_type: claim.claim_type,
        evidence_id: claim.attestation.evidence_id,
        evidence_sha256: claim.attestation.evidence_sha256,
        parameters_canonical_sha256: params_sha256,
        attester_id: claim.attestation.attester_id,
        claimed_role: claim.attestation.claimed_role,
        auth_mechanism: 'DIRECT_KEY_SIGNATURE'
      };
    } else if (claim.attestation.auth.auth_mechanism === 'GATEWAY_CLAIM_ENDORSEMENT') {
      const rcpt = claim.attestation.auth.endorsement_receipt;
      if (!rcpt) throw new Error('E-ERR-017: MISSING_GATEWAY_RECEIPT');

      if (rcpt.claim_payload_digest !== params_sha256) {
        throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION');
      }
```

```
      validateAttesterAuthority(
        claim.attestation.attester_id,
        claim.attestation.claimed_role,
        claim.claim_type,
        claim.attestation.attested_at_utc,
        current_time_utc
      );

      validateGatewayEndorsementReceipt(rcpt, current_time_utc, GATEWAY_AUTHORITY_REGISTRY);

      const rcpt_bytes = Buffer.from(canonicalizeJCS(rcpt), 'utf8');
      const rcpt_sha256 = sha256(rcpt_bytes);

      claim_ref = {
        claim_type: claim.claim_type,
        evidence_id: claim.attestation.evidence_id,
        evidence_sha256: claim.attestation.evidence_sha256,
        parameters_canonical_sha256: params_sha256,
        attester_id: claim.attestation.attester_id,
        claimed_role: claim.attestation.claimed_role,
        auth_mechanism: 'GATEWAY_CLAIM_ENDORSEMENT',
        gateway_id: rcpt.auth_gateway_id,
        gateway_receipt_sha256: rcpt_sha256,
        gateway_nonce: rcpt.nonce
      };

      nonces_to_consume.push({ nonce: rcpt.nonce, expires_at_utc: rcpt.expires_at_utc, gateway_id: rcpt.auth_gateway_id });
    } else {
      throw new Error('E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION');
    }

    claims_evidence_summary[claim.claim_type] = claim_ref;
  }

  const recon = reconcileOfferClaims(offer, assessment, resolved_evidence, current_time_utc);
  const promoted = recon.reconciled;
  const policy_version = 'POC-001/PROMOTION_POLICY/v1';

  // Construct Canonical Decision Signing Payload covering complete decision state (AUDIT-46)
  const signing_payload: EvaluatorDecisionSigningPayload = {
    domain_tag: 'POC-001/EVALUATOR_DECISION_AUTH/v1',
    offer_id: offer.offer_id,
    expected_stream_sequence: canonical_seq,
    assessment_id,
    assessment_sha256,
    policy_version,
    evaluator_version: 'POC-001/EVALUATOR/v1',
    promoted,
    assigned_offer_status: recon.assigned_offer_status,
    assigned_verification_status: recon.assigned_verification_status,
    ...(recon.rejection_reason !== undefined ? { rejection_reason: recon.rejection_reason } : {}),
    claims_evidence_summary,
    nonces_to_consume: promoted ? nonces_to_consume : [],
    evaluated_at_utc: current_time_utc
  };

  const eval_token = generateEvaluatorDecisionAuthToken(evaluator_secret_hex, signing_payload);

  return {
    schema_version: 'POC-001/PROMOTION_EVALUATION_DRAFT/v1',
    offer_id: offer.offer_id,
    expected_stream_sequence: canonical_seq,
    assessment_id,
    assessment_sha256,
    evaluator_auth_token: eval_token,
    claims_evidence_summary,
    policy_version,
    evaluator_version: 'POC-001/EVALUATOR/v1',
    evaluated_at_utc: current_time_utc,
    promoted,
    assigned_offer_status: recon.assigned_offer_status,
    assigned_verification_status: recon.assigned_verification_status,
    rejection_reason: recon.rejection_reason,
    nonces_to_consume: promoted ? nonces_to_consume : []
  };
}

/**
 * Unified Atomic Promotion Persistence Command (AUDIT-12, AUDIT-20, AUDIT-21, AUDIT-27, AUDIT-35, AUDIT-36, AUDIT-37, AUDIT-41, AUDIT-42, AUDIT-43, AUDIT-46, AUDIT-47, AUDIT-49, AUDIT-51)
 * Enforces single canonicalization, samples trusted time inside transaction at Post-Lock Authorization Instant (T_auth = clock.nowUtc()),
 * checks CAS with BigInt numeric comparison, revalidates receipts and attester authorities against T_auth,
 * consumes nonces with gateway ID, stores retained canonical assessment bytes, updates stream head sequence and status,
 * and appends promotion event atomically.
 */
export async function commitPromotionIfVersionAndConsumeNonces(
  db: OfferPersistenceDatabaseSession,
  clock: TrustedClock,
  offer: CanonicalOffer,
  expected_stream_sequence: string,
  evaluation: PromotionEvaluationDraft,
  raw_assessment: unknown,
  token_verifier: EvaluatorTokenVerifier
): Promise<OfferPromotionEventPayload> {
  // 1. Single Canonicalization, Structural Validation & Evaluator Auth Verification (AUDIT-41, AUDIT-46)
  const pre_tx_time_utc = clock.nowUtc();
  const validated = validateEvaluationDraftIntegrity(
    evaluation,
```

```
  offer,
  expected_stream_sequence,
  raw_assessment,
  token_verifier,
  pre_tx_time_utc
);

return await db.executeTransaction(async (tx) => {
  // 2. Lock Offer Stream Head (Pessimistic Row Lock: SELECT ... FOR UPDATE) (AUDIT-37)
  const streamHead = await tx.getOfferStreamHeadForUpdate(offer.offer_id);
  if (['REJECTED_OFFER', 'EXPIRED_OFFER'].includes(streamHead.offer_status)) {
    throw new Error(`E-ERR-036: OFFER_ALREADY_TERMINAL: Offer is in terminal status '${streamHead.offer_status}'`);
  }

  // 3. Sample Trusted Clock INSIDE Transaction Boundary AFTER Lock Acquisition (AUDIT-43, AUDIT-49, AUDIT-51)
  // Formally designated as the Post-Lock Authorization Instant (T_auth)
  const tx_auth_time_utc = clock.nowUtc();
  validateIso8601UtcTimestamp(tx_auth_time_utc);
  const tx_auth_ms = Date.parse(tx_auth_time_utc);

  // 4. Strict Numeric BigInt CAS Check on Canonical Decimal Sequences (AUDIT-42)
  if (BigInt(streamHead.sequence_number) !== BigInt(validated.draft.expected_stream_sequence)) {
    throw new Error('E-ERR-037: CONCURRENCY_CONFLICT: Offer stream has advanced since evaluation started');
  }

  // 5. Post-Lock Cryptographic Receipt, Authority & Digest Binding Revalidation (AUDIT-47, AUDIT-51)
  // Inside the locked transaction, revalidate all attestation authorities and gateway receipts
  // against the lock-acquired Post-Lock Authorization Instant (T_auth) and live registries.
  for (const [claimType, claim] of Object.entries(validated.raw_claims)) {
    // Revalidate attester authority, role binding, and revocation status
    validateAttesterAuthority(
      claim.attestation.attester_id,
      claim.attestation.claimed_role,
      claim.claim_type,
      claim.attestation.attested_at_utc,
      tx_auth_time_utc
    );

    // Revalidate gateway endorsement receipt if gateway auth mechanism
    if (claim.attestation.auth.auth_mechanism === 'GATEWAY_CLAIM_ENDORSEMENT') {
      const rcpt = claim.attestation.auth.endorsement_receipt;

      // Revalidate receipt lifetime, active key in registry, future clock skew, and Ed25519 signature against T_auth
      validateGatewayEndorsementReceipt(rcpt, tx_auth_time_utc, GATEWAY_AUTHORITY_REGISTRY);

      // Revalidate claim payload digest binding
      const param_canonical_bytes = Buffer.from(canonicalizeJCS(claim.claim_parameters), 'utf8');
      const expected_param_digest = sha256(param_canonical_bytes);
      if (rcpt.claim_payload_digest !== expected_param_digest) {
        throw new Error(`E-ERR-017: DIGEST_BINDING_MISMATCH: Endorsement receipt digest does not match claim parameters for '${claimType}'`);
      }
    }
  }

  // 6. Consume Nonces with Gateway ID Atomically under Database Constraint (AUDIT-42, AUDIT-43, AUDIT-47, AUDIT-51)
  const consumed_nonces: string[] = [];
  if (validated.draft.promoted && validated.draft.nonces_to_consume.length > 0) {
    const nonceSet = new Set(validated.draft.nonces_to_consume.map(n => n.nonce));
    if (nonceSet.size !== validated.draft.nonces_to_consume.length) {
      throw new Error('E-ERR-024: DUPLICATE_NONCE_IN_BATCH: Evaluation contains duplicate nonces');
    }
    for (const item of validated.draft.nonces_to_consume) {
      const exp_ms = Date.parse(item.expires_at_utc);
      // Expiration boundary inclusivity: receipt is valid iff tx_auth_ms <= exp_ms.
      // If tx_auth_ms > exp_ms, reject fail-closed with E-ERR-027.
      if (tx_auth_ms > exp_ms) {
        throw new Error(`E-ERR-027: GATEWAY_RECEIPT_EXPIRED: Gateway nonce '${item.nonce}' has expired at post-lock authorization instant (T_auth > T_expires)`);
      }
      // Bind and persist gateway_id directly into offer_nonce_store (AUDIT-42)
      const inserted = await tx.insertNonceIfUnused(item.nonce, item.gateway_id, item.expires_at_utc, tx_auth_time_utc);
      if (!inserted) {
        throw new Error('E-ERR-023: GATEWAY_ENDORSEMENT_NONCE_REPLAYED: Endorsement nonce already consumed');
      }
      consumed_nonces.push(item.nonce);
    }
  }

  // 7. Monotonic Canonical Decimal Sequence Increment (AUDIT-29, AUDIT-42)
  const priorSeqBigInt = BigInt(validated.draft.expected_stream_sequence);
  const resulting_stream_sequence = parseCanonicalStreamSequence((priorSeqBigInt + 1n).toString());

  // 8. Store Retained Canonical Assessment Bytes in Relational Table (AUDIT-36, AUDIT-41, AUDIT-47, AUDIT-51)
  // Persists the exact bytes verified during single canonicalization; zero re-canonicalization in tx.
  await tx.putAssessment(validated.draft.assessment_sha256, validated.canonical_assessment_bytes, tx_auth_time_utc);

  // 9. Append Immutable Offer Promotion Event with Complete Evidence Linkage (AUDIT-20, AUDIT-47, AUDIT-51)
  const eventPayload: OfferPromotionEventPayload = {
    offer_id: offer.offer_id,
    expected_stream_sequence: validated.draft.expected_stream_sequence,
    resulting_stream_sequence,
    assessment_id: validated.draft.assessment_id,
    assessment_sha256: validated.draft.assessment_sha256,
    claims_evidence_summary: validated.draft.claims_evidence_summary,
    policy_version: validated.draft.policy_version,
    evaluator_version: validated.draft.evaluator_version,
    target_offer_status: validated.draft.assigned_offer_status,
    target_verification_status: validated.draft.assigned_verification_status,
```

```
      rejection_reason: validated.draft.rejection_reason,
      consumed_nonces,
      decision_timestamp_utc: tx_auth_time_utc
    };

    const event_bytes = Buffer.from(canonicalizeJcs(eventPayload), 'utf8');
    const event_sha256 = crypto.createHash('sha256').update(event_bytes).digest('hex');

    await tx.appendOfferStreamEvent({
      stream_id: `stream:offer:${offer.offer_id}`,
      sequence_number: resulting_stream_sequence,
      payload_kind: 'OFFER_STATUS_TRANSITION',
      payload: eventPayload,
      event_sha256
    });

    // 10. Atomically Advance Offer Stream Head Sequence and Status (AUDIT-42, AUDIT-47, AUDIT-51)
    await tx.updateOfferStreamHead(
      offer.offer_id,
      resulting_stream_sequence,
      validated.draft.assigned_offer_status,
      tx_auth_time_utc
    );

    return eventPayload;
  });
}

/**
 * Result of Authoritative Pessimistic Reconciliation for Ambiguous Commit Outcomes (AUDIT-51)
 */
export interface AmbiguousCommitReconciliationResult {
  outcome: 'COMMITTED' | 'ROLLED_BACK' | 'STREAM_ADVANCED_UNRELATED' | 'OUTCOME_UNKNOWN';
  stream_sequence: string | null;
  promotion_event?: OfferPromotionEventPayload;
  event_sha256?: string;
  reason: string;
}

export async function reconcileAmbiguousPromotionCommit(
  db: OfferPersistenceDatabaseSession,
  offer_id: string,
  expected_prior_sequence: string,
  attempted_resulting_sequence: string,
  attempted_assessment_sha256: string,
  expected_event_sha256?: string
): Promise<AmbiguousCommitReconciliationResult> {
  try {
    return await db.executeTransaction(async (tx) => {
      // 1. Acquire pessimistic row-lock on stream head (SELECT ... FOR UPDATE) on authoritative primary
      // This query blocks on the PostgreSQL primary until any in-flight transaction on this row resolves.
      const head = await tx.getOfferStreamHeadForUpdate(offer_id);

      // 2. Inspect the event at attempted_resulting_sequence, regardless of whether head.sequence_number
      // equals attempted_resulting_sequence or has advanced further (e.g., head is sequence 13 after a sequence 12 promotion).
      const stream_id = `stream:offer:${offer_id}`;
      const attemptedSeq = parseCanonicalStreamSequence(attempted_resulting_sequence);
      const eventRecord = await tx.getOfferStreamEvent(stream_id, attemptedSeq);

      // 3 & 4. If an event exists at attempted_resulting_sequence, independently recompute and verify all cryptographic digests
      if (eventRecord && eventRecord.payload) {
        const payload = eventRecord.payload;
        const matchesOffer = payload.offer_id === offer_id;
        const matchesAssessmentId = payload.assessment_sha256 === attempted_assessment_sha256;

        // Mandatory cryptographic verification: independently recompute event hash from canonical payload bytes
        const canonicalEventJson = canonicalizeJcs(payload);
        const recomputedEventSha256 = crypto.createHash('sha256').update(Buffer.from(canonicalEventJson, 'utf8')).digest('hex');

        // Constant-time check: stored event_sha256 must match recomputed event hash
        const storedMatchesRecomputed = (eventRecord.event_sha256.length === 64) && crypto.timingSafeEqual(
          Buffer.from(eventRecord.event_sha256, 'hex'),
          Buffer.from(recomputedEventSha256, 'hex')
        );

        // If expected_event_sha256 is supplied, verify caller's expected digest matches recomputed digest
        const callerHashMatches = !expected_event_sha256 || (
          expected_event_sha256.length === 64 && crypto.timingSafeEqual(
            Buffer.from(expected_event_sha256, 'hex'),
            Buffer.from(recomputedEventSha256, 'hex')
          )
        );

        const eventHashValid = storedMatchesRecomputed && callerHashMatches;

        // Mandatory cryptographic verification: retrieve assessment bytes and recompute assessment SHA-256
        const assessmentBytes = await tx.getAssessment(attempted_assessment_sha256);
        let assessmentHashValid = false;
        if (assessmentBytes && assessmentBytes.length > 0) {
          const recomputedAssessmentSha256 = crypto.createHash('sha256').update(assessmentBytes).digest('hex');
          assessmentHashValid = crypto.timingSafeEqual(
            Buffer.from(recomputedAssessmentSha256, 'hex'),
            Buffer.from(attempted_assessment_sha256, 'hex')
          );
        }

        if (matchesOffer && matchesAssessmentId && eventHashValid && assessmentHashValid) {
```

```
          return {
            outcome: 'COMMITTED',
            stream_sequence: head.sequence_number,
            promotion_event: payload,
            event_sha256: recomputedEventSha256,
            reason: 'Original promotion transaction committed successfully; independently recomputed and verified event digest and durable assessment digest on disk (regardless of subsequent stream a
          };
        } else {
          // Event exists at sequence, but failed mandatory cryptographic or identity verification
          return {
            outcome: 'STREAM_ADVANCED_UNRELATED',
            stream_sequence: head.sequence_number,
            reason: `Event at sequence ${attempted_resulting_sequence} failed cryptographic integrity verification (offer_match=${matchesOffer}, assessment_id_match=${matchesAssessmentId}, event_hash
          };
        }
      }

      // 5. If the attempted event is absent AND the head remains at expected_prior_sequence,
      // the original transaction definitively resolved via ROLLBACK (established by holding the row lock).
      if (head.sequence_number === expected_prior_sequence) {
        return {
          outcome: 'ROLLED_BACK',
          stream_sequence: head.sequence_number,
          reason: 'Stream head unchanged after row-lock acquisition and attempted event absent; original transaction resolved via rollback. Safe to retry with unconsumed nonces.'
        };
      }

      // 6. If the head has advanced beyond expected_prior_sequence and the attempted event was absent or inconclusive:
      // An unrelated transaction advanced the stream. The draft is permanently stale. Do NOT infer rollback.
      return {
        outcome: 'STREAM_ADVANCED_UNRELATED',
        stream_sequence: head.sequence_number,
        reason: `Stream head advanced to ${head.sequence_number} without the attempted promotion event at sequence ${attempted_resulting_sequence}. Prior draft is permanently stale and must be re-eva
      };
    });
  } catch (err) {
    // 7. Lock acquisition timed out or database unreachable -> OUTCOME_UNKNOWN (NEVER infer rollback; sequence is strictly unknown)
    return {
      outcome: 'OUTCOME_UNKNOWN',
      stream_sequence: null,
      reason: `Reconciliation lock acquisition failed or timed out: ${err instanceof Error ? err.message : String(err)}. State preserved as unknown (no stream head observed); retry prohibited until a
    };
  }
}

/**
 * Nonce Lifecycle, Failure Policy & Retry Semantics (AUDIT-26)
 * 1. Single-Use upon Committed Promotion: A gateway nonce is consumed atomically if and only if the offer promotion transaction successfully commits to disk.
 * 2. Retry Without Nonce Burn on Failed Assessment: If an offer promotion assessment fails, a failure audit event is logged, but gateway nonces are NOT consumed.
 * 3. Re-Presentation Constraints: The unconsumed receipt may be presented in a retry attempt provided it remains within its 300-second window and unused.
 * 4. Bound Cryptographically: The receipt binds (offer_id, product_id, claim_type, evidence_sha256, parameters_sha256) and cannot authorize a different offer or claim.
 */
```

---

4. Promotion State Machine, CAS Concurrency & Event Updates

4.1 Transactional Consistency Boundary, Storage Guarantees & Stale-Draft Invalidation (AUDIT-21, AUDIT-29, AUDIT-30, AUDIT-36, AUDIT-37, AUDIT-42, AUDIT-43, AUDIT-47, AUDIT-49)

All persistence operations for offer promotion, canonical assessment storage, stream locking, and nonce lifecycle are bound to a single normative database transactional consistency boundary:

1. **Normative PostgreSQL Storage Contract & Row-Level Locking (AUDIT-37)**:
   - Database sessions require PostgreSQL `READ COMMITTED` isolation backed by strict pessimistic write-intent row-level locking via `SELECT ... FOR UPDATE` on `offer_stream_heads`.
   - The stream head row lock is acquired before reading current stream sequence or evaluating terminal status, guaranteeing serialized execution of concurrent promotion attempts against the same offer stream.

2. **Colocated Relational Tables & Primary Key Constraints (AUDIT-36, AUDIT-37, AUDIT-42, AUDIT-49)**:
   All four storage tables reside on the **same relational database connection and transaction boundary**:

```
-- Gateway endorsement nonce tracking (enforces single-use consumption and gateway binding)
CREATE TABLE offer_nonce_store (
  nonce VARCHAR(64) PRIMARY KEY,
  gateway_id VARCHAR(64) NOT NULL,
  expires_at_utc TIMESTAMP WITH TIME ZONE NOT NULL,
  consumed_at_utc TIMESTAMP WITH TIME ZONE NOT NULL
);

-- Offer stream head tracking (pessimistic lock, sequence CAS anchor, and canonical decimal format bounded by uint64)
CREATE TABLE offer_stream_heads (
  stream_id VARCHAR(128) PRIMARY KEY,
  sequence_number VARCHAR(32) NOT NULL CHECK (sequence_number ~ '^(0|[1-9][0-9]*)$' AND CAST(sequence_number AS NUMERIC) <= 18446744073709551615),
  offer_status VARCHAR(64) NOT NULL,
  updated_at_utc TIMESTAMP WITH TIME ZONE NOT NULL
);

-- Immutable append-only offer stream events (composite primary key enforces monotonic uniqueness bounded by uint64)
CREATE TABLE offer_stream_events (
  stream_id VARCHAR(128) NOT NULL,
  sequence_number VARCHAR(32) NOT NULL CHECK (sequence_number ~ '^(0|[1-9][0-9]*)$' AND CAST(sequence_number AS NUMERIC) <= 18446744073709551615),
  payload_kind VARCHAR(64) NOT NULL,
  payload_json TEXT NOT NULL,
  event_sha256 VARCHAR(64) NOT NULL,
  created_at_utc TIMESTAMP WITH TIME ZONE NOT NULL,
  PRIMARY KEY (stream_id, sequence_number)
);

-- Durable content-addressed canonical assessment storage (AUDIT-36, AUDIT-41)
CREATE TABLE offer_assessment_store (
  assessment_sha256 VARCHAR(64) PRIMARY KEY,
  canonical_jcs_payload TEXT NOT NULL,
  created_at_utc TIMESTAMP WITH TIME ZONE NOT NULL
);
```

3. **Atomic Co-location & Unified Transaction Mutation Lifecycle (AUDIT-36, AUDIT-42, AUDIT-47, AUDIT-51)**:
   Inside `commitPromotionIfVersionAndConsumeNonces()`, all mutations execute within a single transaction block:
   - Acquire pessimistic row lock on `offer_stream_heads` (`SELECT ... FOR UPDATE`).
   - Sample trusted clock inside the lock boundary at the **Post-Lock Authorization Instant** ($T_{\text{auth}} = \text{clock.nowUtc()}$, `tx_auth_time_utc`).
   - Validate monotonic CAS sequence equality (`BigInt(head.sequence_number) === BigInt(expected_stream_sequence)`).
   - Revalidate cryptographic authorities against T_{auth}: Ed25519 receipt signature, active gateway status in registry, attester authority/revocation, and claim payload digest binding (AUDIT-47, AUDIT-51).
   - Revalidate nonce expiry against T_{auth} ($T_{\text{auth}} \le T_{\text{expires}}$) and insert nonces with `gateway_id` into `offer_nonce_store`.
   - Store retained `canonical_assessment_bytes` directly into `offer_assessment_store`.
   - Append immutable event payload to `offer_stream_events` at `resulting_stream_sequence`.
   - Atomically advance `offer_stream_heads` to `resulting_stream_sequence` and new status via `updateOfferStreamHead()`.
     If any operation fails or conflicts, the entire transaction rolls back atomically. Nonces inserted during the transaction are aborted by the database engine, guaranteeing zero burned nonces.

4. **Strict Canonical Decimal Sequence Representation & uint64 Bound (AUDIT-42, AUDIT-49)**:
   - Stream sequence numbers are represented as canonical non-negative integer decimal strings without leading zeros bounded by uint64 maximum ($2^{64}-1 = 18446744073709551615$) (`CanonicalStreamSequence`, regex `/^(0|[1-9]\d*)$/`).
   - Representations such as `"01"`, `"+1"`, or `"-1"`, as well as numbers exceeding uint64 max, are rejected fail-closed at the type boundary (`E-ERR-024` / `E-ERR-014`) and prohibited by the SQL database `CHECK (sequence_number ~ '^(0|[1-9][0-9]*)$' AND CAST(sequence_number AS NUMERIC) <= 18446744073709551615)`.
   - All comparisons and CAS evaluations parse strings to native unsigned 64-bit integers (`BigInt`). Lexicographical sorting is strictly prohibited. Database queries order events strictly by `ORDER BY CAST(sequence_number AS NUMERIC) ASC`.

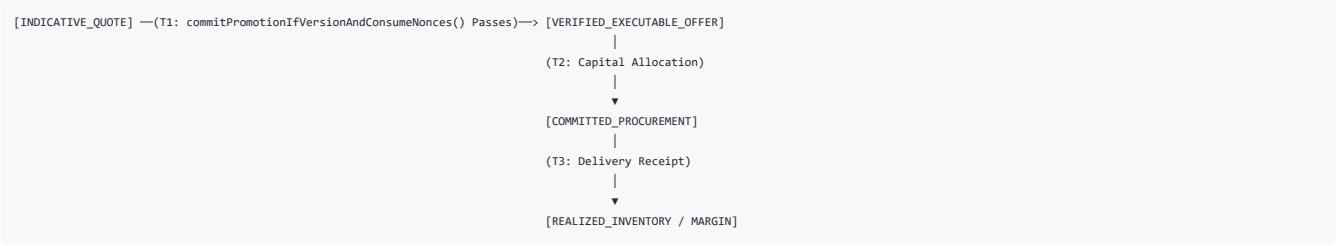
5. **Stale-Draft Invalidation on Assessment Failure (AUDIT-30)**:
   - When a promotion evaluation fails (e.g. price mismatch, stale terms), a structured failure audit event is appended to the offer stream, advancing the stream sequence from s_n to $s_n + 1$.
   - Consequently, the evaluation draft bound to s_n becomes permanently invalid for subsequent attempts. Any retry attempt MUST re-read the updated stream head (s_{n+1}), re-evaluate Phase 1 evaluation, and submit a fresh draft bound to sequence s_{n+1}. Submitting an older draft will fail CAS closed with `E-ERR-037: CONCURRENCY_CONFLICT`.

6. **Post-Lock Authorization Instant (T_{auth}), Expiration Inclusivity & Rollback Guarantees (AUDIT-51)**:
   - **Formal Designation**: The evaluation timestamp used for all transactional validity checks is formally designated as the **Post-Lock Authorization Instant** (T_{auth}), sampled immediately after the pessimistic row lock on the offer stream head is acquired (`tx_auth_time_utc = clock.nowUtc()`). Sampling after lock acquisition guarantees that any time spent waiting in the database lock queue is accounted for in T_{auth}.
   - **Expiration Inclusivity Boundary**: Gateway endorsement receipts enforce exact inclusive validity:
     $$T_{\text{auth}} \le T_{\text{expires}} \implies \text{VALID}$$
     $$T_{\text{auth}} > T_{\text{expires}} \implies \text{REJECTED fail-closed with } \texttt{E-ERR-027: GATEWAY_RECEIPT_EXPIRED}$$
     Receipts evaluated exactly at the expiration boundary millisecond ($T_{\text{auth}} = T_{\text{expires}}$) are accepted; any timestamp strictly exceeding expiration ($T_{\text{auth}} > T_{\text{expires}}$) is rejected fail-closed.
   - **Lock Queue Delay & Timeout Protection**: If concurrent promotion transactions queue for the stream head lock, the wait duration does not invisibly erode the receipt's security window. If the queue delay causes T_{auth} to exceed T_{expires}, the transaction rejects fail-closed with `E-ERR-027`. Furthermore, database sessions configure strict `lock_timeout` (e.g. 5,000 ms) and `statement_timeout` to prevent hung transactions.
   - **TrustedClock Fail-Closed Behavior**: If `clock.nowUtc()` encounters clock unreachable, desynchronization (> 5,000 ms), or non-monotonicity, it throws `E-ERR-007: TRUSTED_CLOCK_UNAVAILABLE` fail-closed.

- **Pre-Commit Failure Guarantees**: Any error raised inside `executeTransaction()` before the `COMMIT` command is issued—including `E-ERR-027`, `E-ERR-007`, `E-ERR-037` (CAS concurrency conflict), `E-ERR-017` (invalid authority/digest), `E-ERR-023` (replayed nonce), database `lock_timeout`, or `statement_timeout`—triggers an explicit PostgreSQL `ROLLBACK`. Under `ROLLBACK`, zero nonces are inserted into `offer_nonce_store`, zero assessment bytes are written to `offer_assessment_store`, zero events are appended to `offer_stream_events`, and zero status mutations occur on `offer_stream_heads`. Gateway nonces remain unburned, permitting retry within their unexpired validity window.
- **Ambiguous Commit Outcome & Pessimistic Row-Lock Reconciliation (AUDIT-51)**:
  If the database connection is severed while `COMMIT` is in flight, the caller cannot determine from the severed connection alone whether the server committed or aborted the transaction. Under PostgreSQL `READ COMMITTED` isolation, an ordinary non-locking read (`SELECT ... FROM offer_stream_heads`) is strictly invalid: MVCC snapshot isolation may observe the prior committed row version while the original transaction is still finalizing its commit in the background. Concluding that an unchanged stream head proves rollback is false and creates hazardous race conditions.
  To establish the true outcome without ambiguity, the caller MUST execute the **Pessimistic Row-Lock Authoritative Reconciliation Procedure** (`reconcileAmbiguousPromotionCommit`):
  1. **Acquire Pessimistic Stream-Head Row Lock on Primary**: Open a dedicated reconciliation transaction against the authoritative PostgreSQL primary and execute:

     ```
     SELECT sequence_number, offer_status
     FROM offer_stream_heads
     WHERE stream_id = $1
     FOR UPDATE;
     ```

     Because the stream head was locked by the original transaction, this reconciliation query blocks until the original in-flight transaction definitively resolves (either commits and releases the lock, or aborts/rolls back). Acquiring this lock proves that the original transaction is no longer in progress.
  2. **Inspect Attempted Event Independent of Stream Head (`attempted_resulting_sequence`)**: Query `offer_stream_events` for the event at `attempted_resulting_sequence`, regardless of whether `sequence_number` equals `attempted_resulting_sequence` or has advanced to a higher sequence due to subsequent legitimate events (e.g. promotion committed at sequence 12, subsequent event advanced head to 13).
  3. **Outcome A — Original Transaction Committed (`COMMITTED`)**: If an event exists at `attempted_resulting_sequence`, verify its payload identity (`offer_id`) and execute mandatory cryptographic verification:
     - Independently recompute the SHA-256 of the stored canonical event payload (`crypto.createHash('sha256').update(canonicalizeJcs(payload))`) and verify with `crypto.timingSafeEqual` that it matches both the stored `event_sha256` and any caller-specified expected event hash.
     - Retrieve the assessment bytes from `offer_assessment_store` (`tx.getAssessment`), recompute their SHA-256 digest, and verify with `crypto.timingSafeEqual` that it matches `attempted_assessment_sha256`.
       If these cryptographic integrity checks pass, the promotion committed successfully. Return `COMMITTED` with the verified `OfferPromotionEventPayload`, recomputed event digest, and current stream sequence, regardless of whether subsequent events have advanced the head. The promotion is definitively complete; do NOT retry.
  4. **Outcome B — Original Transaction Rolled Back (`ROLLED_BACK`)**: If the attempted event at `attempted_resulting_sequence` is absent AND the current stream sequence equals `expected_prior_sequence` *after* the row lock has been granted, the original transaction definitively resolved via rollback. Because the lock was acquired, no in-flight commit remains pending. The system now formally establishes that zero promotion events were appended, zero assessment records were stored, and zero nonces were consumed. Only after this lock-acquired verification is complete may the caller permit a fresh Phase 1 evaluation and retry (subject to gateway receipt freshness).
  5. **Outcome C — Advanced Stream / Inconclusive Event (`STREAM_ADVANCED_UNRELATED`)**: If the stream sequence has advanced beyond `expected_prior_sequence` and the attempted promotion event at `attempted_resulting_sequence` is absent or fails cryptographic integrity validation (e.g. event-hash or assessment-hash recomputation mismatch), an unrelated transaction modified the stream. The current promotion evaluation draft is permanently stale. The caller MUST NOT infer rollback from an absent event or higher sequence, MUST NOT attempt to commit the stale draft, and MUST re-evaluate against the current stream head.
  6. **Outcome D — Lock Timeout / Database Unavailability (`OUTCOME_UNKNOWN`)**: If the reconciliation transaction encounters a `lock_timeout` (e.g. 5,000 ms) while waiting for the row lock, or if primary database connectivity remains unavailable, the outcome is strictly preserved as `OUTCOME_UNKNOWN` with truthful `stream_sequence: null`. The caller MUST NOT infer rollback, MUST NOT report an assumed sequence, MUST NOT assume nonces are unburned, and MUST NOT initiate a retry. The system preserves the pending transaction state and schedules a reconciliation retry once authoritative primary connectivity and lock acquisition succeed.



- **Offer Repudiation / Risk Kill**: Transitions `offer_status` to `'REJECTED_OFFER'` and `verification_status` to `'REJECTED'`.
- **Offer Expiration**: Transitions `offer_status` to `'EXPIRED_OFFER'` and `verification_status` to `'STALE'`.
- **Immutable Historical Supersession**: An earlier erroneous observation is not overwritten in place. A correction event is appended with `supersedes_event_id: "<old_event_id>"`.

---

5. Event Ordering, Hash-Chaining & Independent Checkpoint Anchoring

5.1 Virtual Genesis Head Convention

- **Genesis State**: Every stream initializes at virtual sequence number `-1` with genesis digest:
  $$H_{-1} = \text{"0000000000000000000000000000000000000000000000000000000000000000"}$$
- **First Real Event**: The initial real event is strictly assigned `sequence_number: "0"`, and its `previous_event_hash` MUST equal H_{-1}.
- **Subsequent Events**: The N-th event has `sequence_number: N.toString()`, and its `previous_event_hash` MUST match `event_sha256` of event $N-1$.
- **Ten-Event Verification**: Ten real events are indexed exactly `0, 1, 2, 3, 4, 5, 6, 7, 8, 9`.

5.2 Independent Checkpoint Trust Anchoring (AUDIT-15)

```
export interface TrustAnchorDefinition {
  anchor_id: string;              // e.g. "anchor:ed25519:operator-primary-01"
  algorithm: 'ED25519';
  public_key_hex: string;         // 64-character lowercase hex
  valid_from_utc: string;         // ISO-8601 UTC
  valid_until_utc: string;       // ISO-8601 UTC
  revoked: boolean;
  revoked_at_utc?: string;
  compromised?: boolean;          // If true, invalidates all historical signatures
}

export const TEST_FIXTURE_ANCHOR_REGISTRY: Record<string, TrustAnchorDefinition> = {
  'anchor:ed25519:test-fixture-01': {
    anchor_id: 'anchor:ed25519:test-fixture-01',
    algorithm: 'ED25519',
    public_key_hex: 'a5c0b11a0d3f82054f0a28f804791e3e7f2c69b2d8e411b98c56d7732a39a67a',
    valid_from_utc: '2026-01-01T00:00:00Z',
    valid_until_utc: '2027-01-01T00:00:00Z',
    revoked: false
  }
};

export interface StreamCheckpointReceipt {
  checkpoint_id: string;
  stream_id: string;
  head_sequence_number: string;
  head_event_hash: string;
  checkpoint_timestamp_utc: string;
  prior_git_commit_sha: string;   // Audit pointer: HEAD commit BEFORE this receipt was staged
  trust_anchor_id: string;        // Must resolve in TRUSTED_ANCHOR_REGISTRY
  anchor_signature: string;       // 128-char hex Ed25519 signature over canonical unsigned receipt bytes
  checkpoint_sha256: string;      // Canonical hash of unsigned receipt bytes
}

export interface ExpectedStreamState {
  stream_id: string;
  head_sequence_number: string;
  head_event_hash: string;
}

/**
 * Normative Checkpoint Verifier with Operational vs Historical Modes (AUDIT-15)
 */
export function verifyStreamCheckpointReceipt(
  checkpoint: StreamCheckpointReceipt,
  current_time_utc: string,
  trust_anchors: Record<string, TrustAnchorDefinition>,
  expected_stream: ExpectedStreamState,
  mode: 'CURRENT_OPERATIONAL' | 'HISTORICAL_AUDIT' = 'CURRENT_OPERATIONAL'
): boolean {
  validateIso8601UtcTimestamp(checkpoint.checkpoint_timestamp_utc);
  validateIso8601UtcTimestamp(current_time_utc);
  if (!/^[a-f0-9]{64}$/.test(checkpoint.head_event_hash) ||
      !/^[a-f0-9]{64}$/.test(checkpoint.checkpoint_sha256) ||
      !/^[a-f0-9]{128}$/.test(checkpoint.anchor_signature)) {
    throw new Error('E-ERR-024: INVALID_CHECKPOINT_FORMAT: Malformed hex hash or signature');
  }
  if (!/^(0|[1-9][0-9]*)$/.test(checkpoint.head_sequence_number)) {
    throw new Error('E-ERR-024: INVALID_CHECKPOINT_FORMAT: head_sequence_number must be string-encoded unsigned integer');
  }

  const chk_ms = Date.parse(checkpoint.checkpoint_timestamp_utc);
  const cur_ms = Date.parse(current_time_utc);
  if (chk_ms > cur_ms + 300000) {
    throw new Error('E-ERR-007: CLOCK_SKEW_EXCEEDED: Checkpoint timestamp exceeds future time tolerance (300s)');
  }

  // Anchor Pre-Validation
  const anchor = trust_anchors[checkpoint.trust_anchor_id];
  if (!anchor) throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Anchor not in registry');
  if (anchor.algorithm !== 'ED25519' || !/^[a-f0-9]{64}$/.test(anchor.public_key_hex)) {
    throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Malformed anchor key');
  }
  validateIso8601UtcTimestamp(anchor.valid_from_utc);
  validateIso8601UtcTimestamp(anchor.valid_until_utc);
  const from_ms = Date.parse(anchor.valid_from_utc);
  const until_ms = Date.parse(anchor.valid_until_utc);
  if (from_ms >= until_ms) {
    throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Malformed validity window (valid_from >= valid_until)');
  }

  // Compromise Check: Invalidates fail-closed in ALL modes!
  if (anchor.compromised) {
    throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Trust anchor is compromised (all historical signatures invalidated)');
  }

  if (mode === 'CURRENT_OPERATIONAL') {
    // Current operational mode requires anchor valid AT CURRENT TRUSTED TIME and not revoked
    if (cur_ms < from_ms || cur_ms > until_ms) {
      throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Anchor is expired or not yet valid at current time');
    }
    if (anchor.revoked) {
      throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Trust anchor is currently revoked');
    }
```

```
    if (chk_ms < from_ms || chk_ms > until_ms) {
      throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Checkpoint timestamp outside anchor signing window');
    }
  } else {
    // HISTORICAL_AUDIT mode evaluates anchor validity at signing time
    if (chk_ms < from_ms || chk_ms > until_ms) {
      throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Checkpoint timestamp outside anchor validity window at signing time');
    }
    if (anchor.revoked) {
      if (!anchor.revoked_at_utc) throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Revoked anchor missing revoked_at_utc');
      validateIso8601UtcTimestamp(anchor.revoked_at_utc);
      const rev_ms = Date.parse(anchor.revoked_at_utc);
      if (chk_ms >= rev_ms) {
        throw new Error('E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED: Checkpoint signed at or after anchor revocation timestamp');
      }
    }
  }

  // Stream State Equality Checks
  if (checkpoint.stream_id !== expected_stream.stream_id) throw new Error('E-ERR-015: CHECKPOINT_STREAM_ID_MISMATCH');
  if (checkpoint.head_sequence_number !== expected_stream.head_sequence_number) throw new Error('E-ERR-015: CHECKPOINT_SEQUENCE_MISMATCH');
  if (checkpoint.head_event_hash !== expected_stream.head_event_hash) throw new Error('E-ERR-015: CHECKPOINT_HEAD_HASH_MISMATCH');

  // Reconstruct Unsigned Payload & Verify Signature
  const { anchor_signature, checkpoint_sha256, ...unsigned_receipt } = checkpoint;
  const unsigned_bytes = Buffer.from(canonicalizeJCS(unsigned_receipt), 'utf8');
  if (checkpoint.checkpoint_sha256 !== sha256(unsigned_bytes)) {
    throw new Error('E-ERR-015: CHECKPOINT_SIGNATURE_VERIFICATION_FAILED: Canonical digest mismatch');
  }
  if (!ed25519Verify(unsigned_bytes, anchor.public_key_hex, anchor_signature)) {
    throw new Error('E-ERR-015: CHECKPOINT_SIGNATURE_VERIFICATION_FAILED: Ed25519 signature invalid');
  }

  return true;
}
```

6. Deterministic Identity & Complete RFC 8785 Canonicalization

6.1 Production RFC 8785 (JCS) Conformance Contract, Package Provenance & Input Domain (AUDIT-22, AUDIT-32, AUDIT-39, AUDIT-45)

- **Disambiguated Package Provenance & Standard Reference Repository (AUDIT-45)**:
  To maintain complete audit integrity and truthful provenance records, POC-001 strictly distinguishes the runtime NPM implementation from the upstream IETF standard reference repository:
  1. **Runtime NPM Package Implementation**:
     - Package Name: `json-canonicalize`
     - Version: `3.0.1`
     - NPM Registry URL: `https://registry.npmjs.org/json-canonicalize/-/json-canonicalize-3.0.1.tgz`
     - Upstream Package Repository: `https://github.com/snowyu/json-canonicalize.ts` (Author: Riceball LEE)
     - Tarball Integrity (SHA-512): `sha512-xEzkTucfHhDooWOSMOvrlll9oTk6/mXZIerF2DFWA2chHZsnc2zrmAKjVOyDbSYHr0e3Djx9F89nZ3U2Bn+lSw==`
     - Tarball Checksum (SHA-1): `9da7f919f28e2622610b34a32e51f06a6c3c02cb`
     - Registry Verification Command:

       ```
       npm view json-canonicalize@3.0.1 dist.integrity
       curl -sL https://registry.npmjs.org/json-canonicalize/-/json-canonicalize-3.0.1.tgz | sha512sum
       ```

  2. **Upstream IETF RFC 8785 Reference Standard & Conformance Test Suite Source**:
     - Normative Specification: IETF RFC 8785 (JSON Canonicalization Scheme)
     - Standard Reference Repository: `https://github.com/cyberphone/json-canonicalization` (Author: Anders Rundgren, RFC 8785 co-author)
     - Reference Commit SHA: `19d51d7fe467d4706a3ff08adf8a748f29fc21e0` (Dec 13, 2024)
     - Standard Test Data Source: Provides the official 6-file test manifest and `numgen.js` test vector generator.
     - Provenance Separation Boundary: The upstream commit `19d51d7fe467d4706a3ff08adf8a748f29fc21e0` belongs strictly to the `cyberphone/json-canonicalization` reference repository, not the `snowyu/json-canonicalize.ts` npm package. Runtime conformance of `json-canonicalize@3.0.1` will be verified against the official test vectors extracted from `cyberphone/json-canonicalization`.
- **Audit Verification Status**:
  - The 6-file official test manifest + numgen is formally specified in the table below.
  - In accordance with Gate M0 zero-dependency discipline, full-corpus test execution against this package is designated strictly as **`Specified — Pending Verification`** until implementation authorization and package installation.
- **Pre-Canonicalization Restricted Input Domain**:
  Before passing any payload to canonicalization, the engine executes fail-closed input validation:
  1. **Non-Finite Number Prohibition**: `NaN`, `Infinity`, and `-Infinity` are strictly prohibited and throw `E-ERR-022: JCS_UNSUPPORTED_INPUT: Non-finite numbers prohibited`.
  2. **Numeric Safety**: Number values must be finite numbers within IEEE 754 binary64 range. Monetary values and large integer quantities must be represented as canonical decimal strings (`CanonicalMoney.amount_subunits`).
  3. **Unicode & Surrogate Sanitization**: Input strings must not contain unpaired lone surrogates or unassigned control codepoints (`E-ERR-022: JCS_UNSUPPORTED_INPUT: Lone surrogate or invalid UTF-16 code units prohibited`).
  4. **Strict Duplicate Key Rejection**: Parsers must strictly reject duplicate JSON object keys at the JSON parser boundary fail-closed with `E-ERR-022: CANONICAL_SERIALIZATION_DUPLICATE_KEY_ERROR`.
- **Pinned Conformance Corpus**:
  Full RFC 8785 conformance requires executing against the pinned canonical test suite:

- Verified 6-File Manifest with Exact Full 64-Character SHA-256 Digests:

Fixture Name	Input File Path & Git Blob SHA	Input Full SHA-256 (Length)	Canonical Output Path & Git Blob SHA	
arrays.json	`testdata/input/arrays.json` `blob` `20e62263c430a205944284778973c7633d5879b1`	`e503b6d71d1afa595b1c74b1016445c944cd89f90418066b23de1aeda7d17563` (62 bytes)	`testdata/output/arrays.json` `blob` `5efb93db77cb8aaeffcda0c9bf7c474d5183360b`	`0996` (32 b
french.json	`testdata/input/french.json` `blob` `4ff6d3de24ce8a488a205ac97f04c480cbc38ab8`	`03676a951cd8753ac62589f72eb2105cc782c33425418cfe1d517c111f6e5d5a` (150 bytes)	`testdata/output/french.json` `blob` `2e15cd1b22fe2723596e4ba513e0e269a56939c6`	`d99d` (130
structures.json	`testdata/input/structures.json` `blob` `eb71efb84cf0bb9a022d0c3cbe1e286f0000f824`	`d66893805be1784116af50af3110d08766c70a6b4aad93374723f72346e7aaa6` (138 bytes)	`testdata/output/structures.json` `blob` `dc21e2424e9cc4c84e4cc1114826f3d146bd6156`	`605f` (98 b
unicode.json	`testdata/input/unicode.json` `blob` `4b5bc7699670c930aab223a46114c2c5c3e612cd`	`4621864e014d4a805a563f55b9ea20aba4a2d2dc09c7394f625496998c00702c` (39 bytes)	`testdata/output/unicode.json` `blob` `ee60fd121be2f97943e77eb16294c211a915faa7`	`0d99` (30 b
values.json	`testdata/input/values.json` `blob` `f7712c2fbb73d7afd30e14a15d8c5a2791ce1cd9`	`c4a041b503d6bc236036ef44db4dac499272f60fc22c40dc3b7a54870ba6f1c3` (182 bytes)	`testdata/output/values.json` `blob` `29b720b6e754a0dd9c4dbddac69bdc5cd3ab8792`	`2d5e` (118
weird.json	`testdata/input/weird.json` `blob` `53fabe67bc60bb1897a91c9387caf4b8abb5ef29`	`a3a905266bd4a49a969274ea69baa14ee0c4af0ead926d6fa2b7612b4af75387` (283 bytes)	`testdata/output/weird.json` `blob` `62c83a333e3141f9e16767a7829debbd34bfa913`	`6af5` (214
numgen.js	`testdata/numgen.js` `blob` `6c26777538bfef527f09d30d7ecd0d3a26991e8d`	`31921639b97dfab4eeadfe6da175810eb0dd5b32baa6e2191c3191f246c3f60d` (5,872 bytes)	N/A	N/A

6.2 Known-Answer Test (KAT) Vectors

```
KAT-01 (Project-Defined RFC 8785 Serialization Test Vector):
Input Object:   {"b": 1, "a": "test", "c": [3, 2]}
Canonical JCS:  {"a":"test","b":1,"c":[3,2]}
UTF-8 Bytes:    7b2261223a2274657374222c2262223a312c2263223a5b332c325d7d
SHA-256 Digest: f656707a086145917b61a58bd0c544699796e78bd7b218eb774b239fcabb505f

KAT-02 (Canonical Money Vector):
Input Object:   {"currency": "INR", "amount_subunits": "105000", "scale": 4}
Canonical JCS:  {"amount_subunits":"105000","currency":"INR","scale":4}
SHA-256 Digest: 103316c562e9c5239fccd29452cb55f0018e4b19775d79e8f61ac86beeaf7eed
Note:           "105000" at scale 4 represents INR 10.5000 (wholesale unit price).

KAT-03 (Exact Safe-Integer Volumetric Weight Vector):
Input Dimensions: length_mm = 300, width_mm = 200, height_mm = 100
Volume Product:   6,000,000 mm^3 (Safe integer <= 10^12)
Calculation:      Math.floor(6000000 / 5000) + ((6000000 % 5000) >= 2500 ? 1 : 0)
Expected Weight:  1,200 grams (1.2 kg calculated volumetric weight)

KAT-04 (Exact Rational FX Conversion Vectors):
Part A — Halfway Banker's Rounding (HALF_EVEN at Stage 1):
  Source: USD 1.0001 (subunits: "10001", scale: 4)
  Rate:   "86.500000" (scale: 6, INR per USD)
  Target: INR (scale: 4)
  Exact Calculation: Quotient = 865086, Remainder = 500000 == Half. Quotient is even.
  Expected Target:   "865086" (rounds to even 865086)

  Source: USD 1.0003 (subunits: "10003", scale: 4)
  Rate:   "86.500000" (scale: 6, INR per USD)
  Target: INR (scale: 4)
  Exact Calculation: Quotient = 865259, Remainder = 500000 == Half. Quotient is odd.
  Expected Target:   "865260" (rounds up to even 865260)

Part B — Large Value IEEE 754 Precision Loss Exposure:
  Source: USD 900,719,925,474.0993 (subunits: "9007199254740993", scale: 4)
  Note:   Subunit value exceeds Number.MAX_SAFE_INTEGER (2^53 - 1 = 9007199254740991)
  Rate:   "86.500000" (scale: 6, INR per USD)
  Target: INR (scale: 4)
  Exact BigInt Result:              "779122735535095894"
  Float64 Representable Integer:    "779122735535095808" (via BigInt(Number(source) * 86.5))
  Exact Float64 Error:              86 subunits (= ₹0.0086 at scale 4, where 1 INR = 10,000 subunits)
  Expected Target:                  "779122735535095894" (Strict BigInt exactness enforced)

KAT-05A (Direct Operator Key Signing Vector with Bound claimed_role):
Deterministic Key Seed:  0102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f20
Public Key Hex (32B):    79b5562e8fe654f94078b112e8a98ba7901f853ae695bed7e0e3910bad049664
Canonical JCS String:
{"attested_at_utc":"2026-10-10T12:00:00Z","attester_id":"op:sec:shrom-principal-01","claim_parameters":{"confirmed_unit_price":{"amount_subunits":"105000","currency":"INR","scale":4},"price_evidence_
SHA-256 Digest:          d602df8714f7112c689b077b93e73432bc1ce81e95931fe6beb4fb7c1de5a397
Ed25519 Signature (64B): 7ff10297c25b650b464d8e8a409c8b87ce0d51381caa665c40c622037d638e85cea0a7bac8783f4c41a401adfc7bfd9d0aa10bd018266d4f12ec9e32421ac006
Verification Outcome:    VALID (Independently verified in Node.js)
Confirmed Unit Price:    INR 10.5000 (105,000 subunits at scale 4, wholesale unit price per piece)

KAT-05B (Gateway Claim Endorsement Vector with Exact 300s Expiration Window):
Deterministic Gateway Seed: 020304050607080 90a0b0c0d0e0f101112131415161718191a1b1c1d1e1f2021
Gateway Public Key Hex:     43cdc023d22d5f9e107d1a0693457d35d1d10eb7d21c721192f56f5de40665d3
Canonical Endorsement JCS String:
{"auth_gateway_id":"gw:auth:platform-core-01","claim_signing_payload_sha256":"d602df8714f7112c689b077b93e73432bc1ce81e95931fe6beb4fb7c1de5a397","domain_tag":"GATEWAY_CLAIM_ENDORSEMENT","expires_at_ut
SHA-256 Digest:             c434bdfd9172eb19c1942ac643918cb6419be7b8f3d8b390bf324d139d1695bd
Gateway Signature (64B):    ab67747ddbb5430d8318316dbe943f3dfab81d6dc4100ac4d2be4887534ef132b76cae58fb2e793eb604bd9dbdd8aa921e05cb2f0ed29f9ba15a14b02aae160e
Verification Outcome:       VALID (Independently verified in Node.js via explicit property omission)
Lifetime Window:            Exactly 300,000 ms (5 minutes between issued_at_utc and expires_at_utc)
```

7. Financial Metrics & Accounting Definitions

7.1 Two-Tier Gross-Clearing Accounting Model, Liquidity Boundary & Cumulative Operating Flow Series (AUDIT-34, AUDIT-38, AUDIT-44, AUDIT-48)

To prevent marketplace escrow-held funds from distorting deployable purchasing liquidity, and to eliminate double-counting of proceeds and deductions between marketplace platforms and bank accounts, POC-001 mandates the **Two-Tier Gross-Clearing Accounting Model**:

```
[ TIER 1: TRADE-LEVEL ECONOMIC CLEARING ASSET (B_clearing) ]
Point of Sale:
  Customer Checkout Payment (Gross) ----->  +P_gross (Operating Inflow to Clearing)
Settlement Deductions:
  Marketplace Platform Commission  ----->  -C_fee (Operating Deduction from Clearing)
  Payment Processor Interchange    ----->  -C_pg (Operating Deduction from Clearing)
  Fulfillment & Pick/Pack Charges  ----->  -C_fulfillment (Operating Deduction from Clearing)
  Output GST/VAT Withheld at Source ----->  -C_tax_withheld (Operating Deduction from Clearing)
  Customer Returns & Chargebacks   ----->  -C_refund (Operating Deduction from Clearing)
Rolling Reserve Retention:
  Escrow Delivery Hold (7-14 Days) ----->  Remains Encumbered in B_clearing
                                                  |
                                                  | Net Merchant Bank Payout:
                                                  | P_net = P_gross - Deductions - Reserves
                                                  | (Liquidity Transfer, NOT Revenue)
                                                  ▼
[ TIER 2: TREASURY OPERATING BANK CASH (B_bank / L_bank) ]
Deployable Bank Liquidity:
  Net Marketplace Bank Deposit     ----->  +P_net (Liquidity Transfer from B_clearing)
Procurement Disbursements:
  Supplier Purchase Order Outlay   ----->  -C_po (Funded strictly out of B_bank)
  Inbound Freight & Drayage        ----->  -C_freight (Funded strictly out of B_bank)
  Customs Clearance & Duties       ----->  -C_customs (Funded strictly out of B_bank)
  Direct Statutory Tax Remittance  ----->  -C_tax_remit (Funded strictly out of B_bank)
Tax Recoveries:
  Recoverable Input Tax Credit     ----->  +C_itc (Cash refund received in B_bank)
```

1. **Tier 1: Trade-Level Economic Clearing Asset (B_{clearing})**:
   - At order execution (T_{order}): Gross customer checkout proceeds are recognized as gross operating proceeds:
     $$\Delta B_{\text{clearing}} = + P_{\text{gross}}$$
     Gross proceeds represent the full contractual commercial value inclusive of merchandise price and statutory customer-borne sales taxes (output GST/VAT), prior to platform commissions, processing fees, or channel deductions.
   - At settlement deduction (T_{deduct}): Channel deductions are recognized as discrete negative cash events against the clearing balance:
     $$\Delta B_{\text{clearing}} = - (C_{\text{fee}} + C_{\text{pg}} + C_{\text{fulfillment}} + C_{\text{tax_withheld}} + C_{\text{refund}})$$
   - Escrow reserve retentions (e.g., rolling 7-day or 14-day delivery reserve holds) remain encumbered within B_{clearing} and cannot fund subsequent procurement orders until released.
2. **Tier 2: Treasury Operating Bank Cash (B_{bank}) & Available Bank Liquidity (L_{bank})**:
   - Real-world capital outlays for new inventory (Supplier PO payments $C_{\text{po}} < 0$, Inbound Freight $C_{\text{freight}} < 0$, Customs Duties $C_{\text{customs}} < 0$, Direct Statutory Tax Remittance $C_{\text{tax_remit}} < 0$) are funded strictly out of **Available Operating Bank Cash** (B_{bank}).
   - When marketplace escrow releases funds to the operating bank account ($T_{\text{disbursement}}$), the net merchant disbursement amount (P_{net}):
     $$P_{\text{net}} = P_{\text{gross}} - \sum \text{Itemized Deductions} - \sum \text{Held Reserves}$$
     is recorded as a **Liquidity Transfer** from B_{clearing} to B_{bank}:
     $$\Delta B_{\text{clearing}} = - P_{\text{net}}, \quad \Delta B_{\text{bank}} = + P_{\text{net}}$$
   - **Double-Counting Prohibition**: Net disbursements (P_{net}) are balance-sheet liquidity transfers between clearing and bank accounts. They are **NEVER secondary economic revenue inflows**. Adding both P_{gross} and P_{net} to the cash-flow series, or netting platform fees against P_{net}, is strictly prohibited fail-closed (`E-ERR-038: CASH_FLOW_DOUBLE_COUNTING_PROHIBITED`).
3. **Separation of Available Bank Liquidity from Cumulative Operating Cash Flow Series (AUDIT-48)**:
   To ensure financial calculations accurately measure the capital requirements and returns of trading operations, the model strictly isolates banking treasury liquidity from the operating position series:
   - **Available Bank Liquidity Series ($L_{\text{bank}}(t)$)**:
     $$L_{\text{bank}}(t) = B_0 + \sum_{j \le t} F_j + \sum_{j \le t} C_j^{\text{bank_ops}}$$
     where B_0 is the starting bank account balance, F_j represents owner equity injections or debt draws/repayments (financing cash flows), and $C_j^{\text{bank_ops}}$ represents operational bank disbursements and net receipts. $L_{\text{bank}}(t)$ governs purchasing power and solvency ($L_{\text{bank}}(t) \ge C_{\text{outlay}}$ to prevent insolvency), but does NOT determine operating return metrics.
   - **Cumulative Operating Cash Flow Series ($B_{\text{ops}}(k)$)**:
     Position inception is anchored at timestamp t_0, representing the **pre-event baseline state**:
     $$B_{\text{ops}}(0) \equiv 0$$
     Operating transactions are strictly indexed as positive integers $j \in {1, 2, \dots, K}$ occurring at strictly ordered timestamps $t_1 \le t_2 \le \dots \le t_K$. The cumulative operating cash balance after sequence step $k \in {0, 1, \dots, K}$ is defined as:
     $$B_{\text{ops}}(k) = B_{\text{ops}}(0) + \sum_{j=1}^k C_j^{\text{ops}} = \sum_{j=1}^k C_j^{\text{ops}} \quad (\text{with } B_{\text{ops}}(0) = 0)$$
     where C_j^{ops} contains strictly trade operating cash flows (procurement disbursements $C_{\text{po}} < 0$, freight $C_{\text{freight}} < 0$, duties $C_{\text{customs}} < 0$, tax remittances $C_{\text{tax_remit}} < 0$, realized net merchant proceeds $P_{\text{net}} > 0$, and recovered ITC $C_{\text{itc}} > 0$). Financing flows (B_0, F_j) are strictly excluded.
   - **Intraday Peak Operating Funding Requirement ($C_{\text{peak, ops}}$)**:
     $$C_{\text{peak, ops}} = \max\left(0, \max_{k \le K} \left(-B_{\text{ops}}(k) \right)\right) = \max\left(0, \max_{k \le K} \left(-\sum_{j=1}^k C_j^{\text{ops}} \right)\right)$$
     Peak operating funding evaluates the maximum operating cash deficit reached across all sequence steps $k \in {0, 1, \dots, K}$ starting from $B_{\text{ops}}(0) = 0$. It is mathematically invariant to starting equity B_0 or owner financing injections F_j. A position requiring ₹20,000 of procurement outlays has $C_{\text{peak, ops}} = ₹20,000$ whether the company bank account started with ₹0 or ₹1,000,000 in equity.
   - **Marked-to-Market Return on Peak Funding (RoPF_{mtm})**:
     $$\text{RoPF}_{\text{mtm}} = \frac{\sum_{j=1}^K C_j^{\text{ops}} + \text{Valuation}(T)}{C_{\text{peak, ops}}} \times 100$$
     Requires $C_{\text{peak, ops}} > 0$. If $C_{\text{peak, ops}} = 0$ (cumulative operating cash flows never entered deficit), the engine emits:
     `{ status: 'ZERO_OPERATING_OUTLAY', note: 'Cumulative operating cash flow series never entered deficit; RoPF undefined' }` .
     Missing or unverified ending inventory valuation emits `DATA_REQUIRED` (never coerced to zero).
4. **Statutory Tax & Chargeback Treatment**:

- **Output Tax (GST/VAT collected)**: Collected as part of P_{gross}; tracked as a statutory tax liability until remitted to tax authorities ($C_{\text{tax_remit}} < 0$).
- **Input Tax Credit (ITC)**: Taxes paid on purchase orders are part of PO outlay; recoverable ITC is recognized as a cash inflow ($C_{\text{itc}} > 0$) strictly upon actual bank refund receipt, never upon filing or accrual.
- **Refunds & Chargebacks**: Customer return disbursements ($C_{\text{refund}} < 0$) relieve revenue and reduce clearing balance B_{clearing}.

5. **Deterministic Ordering & Midnight UTC Aggregation**:
   - Transactions are ordered strictly by the composite key `(entry_timestamp_utc, sequence_number)`:
     $$T_a < T_b \iff (\text{entry_timestamp}_a < \text{entry_timestamp}_b) \lor (\text{entry_timestamp}_a = \text{entry_timestamp}_b \land \text{sequence_number}_a < \text{sequence_number}_b)$$
   - Net daily cash flows are aggregated at **midnight UTC** (`00:00:00.000Z` to `23:59:59.999Z`):
     $$C_d^{\text{ops}} = \sum_{k \in \text{operating transactions on date } d} C_k^{\text{ops}}$$

7.2 Disambiguated Peak Funding Metrics & Committed Capital Netting (AUDIT-16, AUDIT-18, AUDIT-48)

All funding and balance metrics are computed **independently per single currency**. Consolidating cross-currency metrics requires an explicit authenticated `FXConversionReceipt` validated at point-of-use (`validateFXReceiptAtPointOfUse`); unreceipted cross-currency additions are rejected at runtime (`E-ERR-008: CROSS_CURRENCY_SUMMATION_ILLEGAL`).

Committed Capital Lifecycle Formula (AUDIT-18):

Committed capital evaluates legally or operationally encumbered capital across the complete purchase order lifecycle:
$$C_{\text{committed}}(t) = \sum_{e \in \mathcal{E}{\text{PO}}(t)} \Delta C{\text{committed}}(e)$$
Where each PO maintains an unfulfilled commitment balance $U_{\text{po}} \ge 0$:

- **Order Issuance (`PO_AUTHORIZED`)**: Adds $\Delta C = + \text{PO.authorized_amount}$; initializes $U_{\text{po}} = \text{PO.authorized_amount}$.
- **Order Cancellation (`PO_CANCELLED`)**: Relieves remaining unfulfilled commitment: $\Delta C = - U_{\text{po}}$; sets $U_{\text{po}} = 0$.
- **Order Fulfillment / Delivery (`PO_FULFILLED`)**: Relieves delivered commitment: $\Delta C = - \min(\text{delivery_amount}, U_{\text{po}})$; decrements U_{po}, transferring value to deployed inventory landed cost.
- **Order Amendment (`PO_AMENDED`)**: Adjusts commitment: $\Delta C = + (\text{new_amount} - \text{prior_amount})$; updates U_{po}.
- **Order Replacement (`PO_REPLACED`)**: Formally relieves prior order's remaining commitment: $\Delta C = - U_{\text{po}}$; sets $U_{\text{po}} = 0$. The replacement PO issues under a distinct identifier as a separate `PO_AUTHORIZED` event.
- **Idempotency Protection**: Ingestion enforces monotonic PO state transitions; duplicate terminal events (`PO_CANCELLED` , `PO_REPLACED`) are rejected fail-closed (`E-ERR-035`) and cannot double-relieve commitment.

Metric Name	Mathematical Definition	Purpose & Context
Intraday Peak Operating Funding Requirement ($C_{\text{peak, ops}}$)	$\max\left(0, \max_{k \le K} \left(-\sum_{j=1}^k C_j^{\text{ops}} \right)\right) = \max\left(0, \max_{k \le K} (-B_{\text{ops}}(k))\right)$	**True peak financing required by operating cycle**. Evaluates sequenced operating transaction deficits from pre-event baseline $B_{\text{ops}}(0) = 0$ over $j \in {1, \dots, K}$, strictly invariant to initial owner capital B_0 or financing F_j.
Peak Net Daily Operating Funding Requirement ($C_{\text{peak, daily_net_ops}}$)	$\max\left(0, \max_{d \le D} \left(-\sum_{\tau=t_0}^d C_{\tau}^{\text{ops}} \right)\right)$	**Portfolio balance sheet operating metric**. Evaluates netted end-of-day operating balances, excluding intraday financing swings.
Committed Capital ($C_{\text{committed}}(t)$)	Evaluated via lifecycle formula above	Total capital legally or operationally encumbered across active purchase orders.
Net Operating Cash Funding Deficit ($C_{\text{deficit}}(t)$)	$\max\left(0, -B_{\text{ops}}(t)\right)$	Cumulative operating cash deficit at sequence t (renamed from 'Actual Capital Deployed').
Realized Operating Cash Multiple (CoC)	$\frac{\sum_{C_j^{\text{ops}} > 0} C_j^{\text{ops}}}{	\sum_{C_j^{\text{ops}} < 0} C_j^{\text{ops}}
Marked-to-Market Return on Peak Funding (%) (RoPF_{mtm})	$\frac{\sum C_j^{\text{ops}} + \text{Valuation}(T)}{C_{\text{peak, ops}}} \times 100$	Return on peak operating financing including verified ending inventory valuation. Cash outflows are negative ($C_j^{\text{ops}} < 0$), inflows positive ($C_j^{\text{ops}} > 0$). Denominator requires $C_{\text{peak, ops}} > 0$. Zero outlay emits `ZERO_OPERATING_OUTLAY` . Missing or unverified valuation emits `DATA_REQUIRED` .
Conventional Holding-Period Return (HPR)	$\frac{V(T) - V(0) + D(0, T)}{V(0)} \times 100$	Standard single-period return for an isolated investment with initial capital $V(0) > 0$, ending value $V(T)$, and interim distributions $D(0, T)$.
Money-Weighted Rate of Return (XIRR)	Exact internal rate of return solving $\sum_{j} \frac{C_j^{\text{ops}}}{(1 + r)^{(t_j - t_0)/365}} = 0$	Standard money-weighted rate of return for irregular multi-date portfolio operating cash flows. Computed independently per single currency.

7.3 Disambiguated Post-Outlay Capital Recovery Metrics (AUDIT-16, AUDIT-48, AUDIT-50)

Position inception is defined at timestamp t_0, establishing the **pre-event baseline state**:
$$B_{\text{ops}}(0) \equiv 0$$
Subsequent operating transactions are indexed strictly as positive integers $j \in {1, 2, \dots, K}$ at ordered timestamps $t_1 \le t_2 \le \dots \le t_K$.
The cumulative operating cash balance at sequence step $k \in {0, 1, \dots, K}$ is:
$$B_{\text{ops}}(k) = B_{\text{ops}}(0) + \sum_{j=1}^k C_j^{\text{ops}} = \sum_{j=1}^k C_j^{\text{ops}} \quad (\text{with } B_{\text{ops}}(0) = 0)$$

1. **Funded Outlay Precondition**:
   There must exist at least one sequence point $k_{\text{outlay}} \in {1, \dots, K}$ where cumulative operating balance entered negative territory:
   $$B_{\text{ops}}(k_{\text{outlay}}) < 0$$
   The initial capital outlay index k_{outlay} is the earliest sequence point of deficit:
   $$k_{\text{outlay}} = \min \left{ \kappa \in {1, \dots, K} \mid B_{\text{ops}}(\kappa) < 0 \right}$$

with corresponding outlay timestamp $T_{\text{outlay}} = t_{k_{\text{outlay}}}$.
If for all $k \in {0, \dots, K}$, $B_{\text{ops}}(k) \ge 0$, capital was never deployed at risk, and the engine reports:
`{ status: 'ZERO_OPERATING_OUTLAY', note: 'Cumulative operating cash balance never entered negative territory' }`.

2. **Strict Post-Outlay Zero-Crossing**:
   Recovery index k_{rec} is the earliest sequence step strictly following k_{outlay} where cumulative operating balance returns to zero or above:
   $$k_{\text{rec}} = \min \left\{ k \in {1, \dots, K} \mid k > k_{\text{outlay}} \text{ and } B_{\text{ops}}(k) \ge 0 \right\}$$
   with corresponding recovery timestamp $T_{\text{rec}} = t_{k_{\text{rec}}}$.

3. **Separately Named Recovery Metrics**:
   - `Capital Cash Recovery Duration (from Outlay)`: Exact integer days $\left\lfloor \frac{T_{\text{rec}} - T_{\text{outlay}}}{86{,}400{,}000\text{ ms}} \right\rfloor$ (days from first capital deployment to operating cash balance breakeven).
   - `Position Lifecycle Recovery Duration`: Exact integer days $\left\lfloor \frac{T_{\text{rec}} - t_0}{86{,}400{,}000\text{ ms}} \right\rfloor$ (days from position inception to operating cash breakeven).
   - `Economic Recovery Distinction`: True economic profit requires both operating cash breakeven ($B_{\text{ops}} \ge 0$) AND full liquidation of inventory liabilities. Cash recovery does not imply inventory liquidation.

4. **Concrete Numerical Demonstration: Acceptance Fixture** `REC-OUTLAY-001`:
   The disambiguated indexing and metric calculation are concretely demonstrated using acceptance fixture `REC-OUTLAY-001`:
   - **Pre-event Baseline ($k=0$ at t_0, Day 0)**:
     $$B_{\text{ops}}(0) \equiv 0$$
   - **Event $j=1$ at t_1 (Day 0, customer advance payment)**:
     $$C_1^{\text{ops}} = +500 \implies B_{\text{ops}}(1) = B_{\text{ops}}(0) + C_1^{\text{ops}} = 0 + 500 = +500$$
     (Operating balance is in surplus; no capital deployed at risk; $B_{\text{ops}}(1) \ge 0$).
   - **Event $j=2$ at t_2 (Day 10, supplier PO procurement cash outlay)**:
     $$C_2^{\text{ops}} = -1,500 \implies B_{\text{ops}}(2) = B_{\text{ops}}(1) + C_2^{\text{ops}} = 500 - 1,500 = -1,000$$
     (Operating balance enters deficit; satisfies Funded Outlay Precondition: $k_{\text{outlay}} = 2$, $T_{\text{outlay}} = \text{Day 10}$).
   - **Event $j=3$ at t_3 (Day 25, net customer recovery payment)**:
     $$C_3^{\text{ops}} = +1,200 \implies B_{\text{ops}}(3) = B_{\text{ops}}(2) + C_3^{\text{ops}} = -1,000 + 1,200 = +200$$
     (Operating balance returns to zero or above; first post-outlay zero-crossing: $k_{\text{rec}} = 3$, $T_{\text{rec}} = \text{Day 25}$).
   - **Metric Calculations**:
     - **Intraday Peak Operating Funding Requirement ($C_{\text{peak, ops}}$)**:
       $$\max\left(0, \max_{k \in {0, 1, 2, 3}} (-B_{\text{ops}}(k))\right) = \max(0, -0, -500, -(-1,000), -200) = \max(0, 0, -500, +1,000, -200) = 1,000$$
     - **Capital Cash Recovery Duration (from Outlay)**:
       $$T_{\text{rec}} - T_{\text{outlay}} = \text{Day 25} - \text{Day 10} = 15\text{ days}$$
     - **Position Lifecycle Recovery Duration**:
       $$T_{\text{rec}} - t_0 = \text{Day 25} - \text{Day 0} = 25\text{ days}$$

---

8. Executable Acceptance Test Plan

8.0 Three-State Conformance & Acceptance Convention (AUDIT-49, AUDIT-52)

POC-001 enforces an auditable three-state conformance convention across all test cases and acceptance checklist criteria:

1. `Specified — Pending Verification`: The normative contract is fully defined with complete types, preconditions, invariants, algorithms, and expected test vectors/digests, but runtime execution against the production codebase is blocked pending implementation authorization (e.g. package installation, test runner execution).
2. `Design Reviewed`: The architectural invariant, mathematical formulation, concurrency protocol, database schema, or security boundary has been audited, proven, and accepted at the specification level by governance review, demonstrating complete structural soundness without claiming runtime test execution.
3. `Verified — Test Evidence Recorded`: The designated test fixture, mathematical formula, or cryptographic vector has been independently calculated and verified bitwise with reproducible test evidence retained (specifically KAT-03 volumetric safe-integer formula proof, KAT-04 rational integer arithmetic proofs matching bitwise, and KAT-05B deterministic Ed25519 signature reproduction).

Eleven test suites are mapped with explicit test IDs, positive/negative assertions, and KAT fixtures. All tests are currently **Specified** in design; test execution is blocked pending implementation authorization.

> [!NOTE] Scoping of Automated Document Checks vs Runtime Test Suites (AUDIT-52)
> Automated specification audit scripts (such as `verify_spec_final.py`) operate strictly as document-level structural and syntax regression checks. They verify cross-section terminology, type export signatures, regex constraints, and the absence of deprecated terms across markdown clauses. They do NOT constitute runtime cryptographic proofs, runtime test suite execution, or formal mathematical verification. In accordance with the Three-State Conformance Model, all executable test suites remain classified strictly as `Specified — Pending Verification` until implementation authorization is granted.

8.1 Six Gate M1 Contract Suites

Suite ID	Test Case ID	Type	Action / Fixture	Expected Outcome	Status
SUITE-01	`JCS-KAT-001`	Positive	Feed KAT-01 into canonical serializer	Digest matches `f656707a086145917b61a58bd0c544699796e78bd7b218eb774b239fcabb505f`	Specified — Pending Verification
SUITE-01	`JCS-KAT-002`	Positive	Feed KAT-02 into canonical serializer	Digest matches `103316c562e9c5239fccd29452cb55f0018e4b19775d79e8f61ac86beeaf7eed`	Specified — Pending Verification
SUITE-01	`JCS-CONF-PINNED-001`	Positive	Execute against pinned cyberphone commit `19d51d7fe467d4706a3ff08adf8a748f29fc21e0` manifest (6 fixtures + numgen)	All 6 standard test files pass bitwise against verified SHA-256 digests	Specified — Pending Verification
SUITE-01	`JCS-EDGE-002`	Negative	Submit JSON payload containing duplicate key `{"a":1,"a":2}`	Throws `E-ERR-022: CANONICAL_SERIALIZATION_DUPLICATE_KEY_ERROR`	Specified — Pending Verification
SUITE-02	`CURR-BOUND-001`	Negative	Decimal string with leading zero `"0123"`, negative zero `"-0"`, or exceeding 128 bits	Throws `E-ERR-010: CANONICAL_DECIMAL_FORMAT_ERROR` or `E-ERR-014: ARITHMETIC_OVERFLOW`	Specified — Pending Verification
SUITE-02	`CURR-SCALE-001`	Negative	Construct INR money with `scale: 2` instead of `scale: 4`	Throws `E-ERR-010: INVALID_CURRENCY_SCALE`	Specified — Pending Verification
SUITE-02	`CURR-CROSS-002`	Negative	Attempt direct summation of ₹100 and $5 without FX receipt	Throws `E-ERR-008: CROSS_CURRENCY_SUMMATION_ILLEGAL`	Specified — Pending Verification
SUITE-02	`FX-EXACT-001`	Positive	Execute rational BigInt FX conversion against KAT-04 (Parts A & B)	Matches `865086`, `865260`, and `779122735535095894` bitwise; exposes ₹0.0086 Float64 delta	Specified — Pending Verification
SUITE-02	`RUNTIME-MONEY-INV-001`	Negative	Pass negative price subunits or unregistered currency to `createCanonicalMoney`	Throws `E-ERR-010` fail-closed	Specified — Pending Verification
SUITE-03	`ALLOW-FAIL-001`	Negative	Attempt unlisted tuple (`SYNTHETIC_TEST`, `VERIFIED`, `INDICATIVE_QUOTE`)	Throws `E-ERR-001: FAIL_CLOSED_DISALLOWED_TUPLE`	Specified — Pending Verification
SUITE-03	`PROMO-ALLOW-REJECT-011`	Negative	Submit evidence with `verification_status: 'PARTIALLY_VERIFIED'` to promotion gate	Throws `E-ERR-001: FAIL_CLOSED_DISALLOWED_TUPLE`	Specified — Pending Verification
SUITE-03	`CLAIM-EVID-MISMATCH-008`	Negative	Submit valid verified price invoice as evidence for a stock availability claim	Throws `E-ERR-033: EVIDENCE_CLAIM_TYPE_MISMATCH`	Specified — Pending Verification
SUITE-03	`ASSESSMENT-SHAPE-FAIL-016`	Negative	Submit assessment missing `claim_price_confirmation` slot or with extraneous properties	Throws `E-ERR-024: INVALID_CLAIM_PARAMETERS` before dereferencing	Specified — Pending Verification
SUITE-03	`PROMO-MISSING-CLAIM-001`	Negative	Promote offer lacking stock availability claim	Throws error for missing mandatory claim	Specified — Pending Verification
SUITE-03	`PROMO-EXPIRED-TERMS-002`	Negative	Promote offer whose commercial terms `valid_until_utc` has passed	Returns `assigned_offer_status: 'EXPIRED_OFFER'`	Specified — Pending Verification
SUITE-03	`PROMO-STALE-PRICE-003`	Negative	Promote offer with price confirmation source evidence older than 7 days	Returns `assigned_offer_status: 'INDICATIVE_QUOTE'`, `assigned_verification_status: 'PARTIALLY_VERIFIED'`	Specified — Pending Verification
SUITE-03	`PROMO-STALE-STOCK-004`	Negative	Promote offer with stock confirmation source evidence older than 48 hours	Returns `assigned_offer_status: 'INDICATIVE_QUOTE'`, `assigned_verification_status: 'PARTIALLY_VERIFIED'`	Specified — Pending Verification
SUITE-03	`PROMO-HISTORICAL-ERR-005`	Negative	Promote offer referencing backfilled evidence (`is_historical_import: true`)	Throws `E-ERR-032: HISTORICAL_EVIDENCE_NOT_EXECUTABLE`	Specified — Pending Verification
SUITE-03	`PROMO-MISMATCH-PRICE-006`	Negative	Submit claim with unit price ₹105 when offer unit price is ₹110	Throws `E-ERR-028: OFFER_PRICE_CLAIM_MISMATCH`	Specified — Pending Verification
SUITE-03	`PROMO-MISMATCH-MOQ-007`	Negative	Submit claim with MOQ 100 when offer MOQ is 10	Throws `E-ERR-029: OFFER_MOQ_CLAIM_MISMATCH`	Specified — Pending Verification

Suite ID	Test Case ID	Type	Action / Fixture	Expected Outcome	Status
SUITE-03	`PROMO-MISMATCH-PAY-008`	Negative	Submit claim with payment terms 30d when offer states 60d	Throws `E-ERR-030: OFFER_PAYMENT_TERMS_MISMATCH`	Specified — Pending Verification
SUITE-03	`PROMO-MISMATCH-INCO-009`	Negative	Submit claim with Incoterms EXW when offer states FOB	Throws `E-ERR-031: OFFER_INCOTERMS_MISMATCH`	Specified — Pending Verification
SUITE-03	`PROMO-GATE-PASS-010`	Positive	Submit offer with all 4 fresh, authenticated, exactly matching claims	Passes gate, promoted to `VERIFIED_EXECUTABLE_OFFER`	Specified — Pending Verification
SUITE-03	`PROMO-FUTURE-TS-012`	Negative	Submit attestation timestamped $> 300\text{s}$ into future	Throws `E-ERR-026: FUTURE_DATED_ATTESTATION`	Specified — Pending Verification
SUITE-03	`PROMO-EXP-KEY-013`	Negative	Submit attestation outside principal validity window	Throws `E-ERR-019: OPERATOR_KEY_REVOKED`	Specified — Pending Verification
SUITE-03	`PROMO-CAS-ATOMIC-015`	Negative	Concurrent promotions on same stream sequence; second caller fails CAS	Second transaction aborts with `E-ERR-037`; zero nonces burned	Specified — Pending Verification
SUITE-03	`DRAFT-ASSESSMENT-BIND-019`	Negative	Submit evaluation draft with assessment_sha256 that does not match canonical JCS hash of assessment	Throws `E-ERR-024` fail-closed before persistence	Specified — Pending Verification
SUITE-03	`ASSESSMENT-STORE-RETRIEVE-020`	Positive	Commit promotion; retrieve full canonical assessment from offer_assessment_store by digest	Stored bytes match canonical JCS bytes bitwise	Specified — Pending Verification
SUITE-03	`STORAGE-ACID-ROWLOCK-021`	Positive	Execute concurrent promotion under PostgreSQL READ COMMITTED with SELECT ... FOR UPDATE row locking; verify unique constraint enforcement	Primary key constraints enforce single winner and zero leaked nonces	Specified — Pending Verification
SUITE-03	`STALE-DRAFT-RETRY-INVALID-022`	Negative	Failed assessment appends failure event advancing sequence; retry submitting previous draft	Second attempt fails CAS with `E-ERR-037`; must re-evaluate fresh sequence	Specified — Pending Verification
SUITE-03	`GATEWAY-CLOCK-SKEW-5S-023`	Negative	Submit gateway receipt with issuance timestamp +5,001 ms into future	Throws `E-ERR-007: CLOCK_SKEW_EXCEEDED` (capped at 5s policy tolerance)	Specified — Pending Verification
SUITE-01	`JCS-PACKAGE-INTEGRITY-024`	Positive	Verify pinned package integrity metadata for @cyberphone/json-canonicalization v0.8.2	Package version and commit SHA verified	Specified — Pending Verification
SUITE-03	`PROD-NORM-NFC-HSN-025`	Positive/Negative	Feed Unicode denormalized string (NFD); verify NFC normalization; reject 3-digit HSN code	Produces canonical NFC output; 3-digit HSN throws `E-ERR-024`	Specified — Pending Verification
SUITE-02	`FIN-ROPF-CASHFLOW-INCLUSION-026`	Positive	Calculate RoPF with complete schedule of operating outflows, inflows, and verified valuation	Matches exact percentage; excluded financing flows omitted	Specified — Pending Verification
SUITE-03	`ASSESSMENT-SUMMARY-DERIVED-027`	Negative	Submit evaluation draft with matching digest but mismatched claim references or extraneous keys	`validateEvaluationDraftIntegrity` throws `E-ERR-024` fail-closed	Specified — Pending Verification
SUITE-03	`ASSESSMENT-STORE-TX-ATOMIC-028`	Positive	Commit promotion; verify canonical assessment payload is stored in `offer_assessment_store` within same transaction	Stored payload in relational table matches canonical JCS bytes bitwise	Specified — Pending Verification
SUITE-03	`STORAGE-PG-ROWLOCK-CONSTRAINTS-029`	Positive	Concurrent promotions with row locking; verify primary key uniqueness and BigInt sequence increment	Strict serialized execution; single winner; BigInt order preserved	Specified — Pending Verification
SUITE-02	`FIN-CASHFLOW-GROSS-DOUBLECOUNT-030`	Negative	Submit cash-flow series containing both gross sales and net merchant disbursements	Throws `E-ERR-038: CASH_FLOW_DOUBLE_COUNTING_PROHIBITED`	Specified — Pending Verification
SUITE-03	`TRUSTED-CLOCK-BOUNDARY-031`	Negative	Attempt promotion commit where nonce expires against internal `TrustedClock.nowUtc()`	Throws `E-ERR-027: GATEWAY_RECEIPT_EXPIRED` at commit boundary	Specified — Pending Verification

Suite ID	Test Case ID	Type	Action / Fixture	Expected Outcome	Status
SUITE-03	`HSN-LEGAL-ASSIGNMENT-SYNTAX-032`	Positive/Negative	Feed 8-digit HSN code ('73269099'); verify syntax acceptance without legal tariff classification	Accepts format; rejects non-numeric/invalid lengths	Specified — Pending Verification
SUITE-03	`ASSESSMENT-SINGLE-CANON-033`	Positive/Negative	Canonicalize raw_assessment once; verify evaluator_auth_token; persist exact bytes in offer_assessment_store	Zero TOCTOU re-canonicalization in tx; forged evaluator draft throws `E-ERR-017`	Specified — Pending Verification
SUITE-03	`NONCE-GATEWAY-STREAMHEAD-034`	Positive	Commit promotion; verify gateway_id is persisted in offer_nonce_store and updateOfferStreamHead advances sequence	Relational schema integrity verified; stream head and nonces updated atomically	Specified — Pending Verification
SUITE-03	`SEQUENCE-CANON-DECIMAL-035`	Negative	Submit sequence number with leading zeros ('01') or non-numeric characters	parseCanonicalStreamSequence throws `E-ERR-024: INVALID_SEQUENCE_FORMAT`	Specified — Pending Verification
SUITE-03	`CLOCK-TX-LOCK-BOUNDARY-036`	Negative	Inject 10s artificial delay before stream row lock; receipt expires during lock wait queue	Receipt revalidated against tx_auth_time_utc throws `E-ERR-027` fail-closed	Specified — Pending Verification
SUITE-02	`FIN-GROSS-CLEARING-TIER-037`	Positive	Evaluate two-tier gross-clearing model: gross sales in clearing, net payout as clearing-to-bank transfer	Peak funding evaluated strictly against bank cash; double-counting rejected	Specified — Pending Verification
SUITE-01	`JCS-PROVENANCE-SEPARATION-038`	Positive	Verify registry metadata for snowyu/json-canonicalize.ts and test suite link to cyberphone/json-canonicalization	Provenance records match npm registry metadata and IETF reference suite	Specified — Pending Verification
SUITE-03	`EVAL-DECISION-HMAC-COVERAGE-039`	Positive/Negative	Submit evaluation draft with modified statuses, rejection reason, evaluator version, or claims summary; verify HMAC	RFC 8785 canonical EvaluatorDecisionSigningPayload HMAC fails; rejected via `crypto.timingSafeEqual` fail-closed	Specified — Pending Verification
SUITE-03	`COMMIT-TX-RECEIPT-REVALIDATION-040`	Negative	Post-Lock Authorization Instant T_{auth} sampled post-row-lock exceeds receipt expiry ($T_{\text{auth}} > T_{\text{expires}}$) or attester revoked	Revalidation inside row-locked transaction detects revocation/expiration and aborts tx with explicit pre-commit ROLLBACK	Specified — Pending Verification
SUITE-03	`RECONCILE-AMBIGUOUS-COMMIT-045`	Positive/Negative	Simulate connection severed during COMMIT; execute `reconcileAmbiguousPromotionCommit()` with `SELECT ... FOR UPDATE` row lock	Waits for in-flight resolution; independently recomputes event SHA-256 and assessment SHA-256 on disk	Specified — Pending Verification
SUITE-03	`RECONCILE-SUBSEQUENT-EVENT-HEAD-046`	Positive	Promotion commits at sequence S_n; subsequent event advances stream to S_{n+1}; reconciliation inspects event at S_n	Returns `COMMITTED` with verified event payload and current head S_{n+1} (subsequent event does not undo committed promotion)	Specified — Pending Verification
SUITE-03	`RECONCILE-EVENT-HASH-MISMATCH-047`	Negative	Event exists at attempted sequence but stored `event_sha256` does not match independently recomputed digest of canonical payload	Rejects with `STREAM_ADVANCED_UNRELATED`; invalidates draft and prohibits retry	Specified — Pending Verification
SUITE-03	`RECONCILE-ASSESSMENT-HASH-MISMATCH-048`	Negative	Recomputed SHA-256 of stored assessment bytes fails to match `attempted_assessment_sha256` or assessment record missing	Rejects with `STREAM_ADVANCED_UNRELATED`; invalidates draft	Specified — Pending Verification
SUITE-03	`RECONCILE-LOCK-TIMEOUT-UNKNOWN-049`	Negative	Primary database unreachable or row lock wait exceeds `lock_timeout` (5,000 ms)	Preserves `OUTCOME_UNKNOWN` fail-closed with truthful `stream_sequence: null`; zero nonces burned, retry strictly blocked	Specified — Pending Verification
SUITE-02	`FIN-PEAK-INDEPENDENT-EQUITY-041`	Positive	Compute peak operating funding requirement $C_{\text{peak, ops}}$ on identical operating flows with ₹0 vs ₹100,000 initial bank equity B_0	$C_{\text{peak, ops}}$ is identical bitwise in both cases; RoPF denominator is strictly invariant to initial owner capital	Specified — Pending Verification
SUITE-03	`CLOCK-FAIL-CLOSED-UNAVAILABLE-042`	Negative	Simulate clock source unreachable, desynchronized (>5s), or non-monotonic	`TrustedClock.nowUtc()` throws `E-ERR-007: TRUSTED_CLOCK_UNAVAILABLE` fail-closed	Specified — Pending Verification
SUITE-03	`STREAM-SEQ-UINT64-BOUND-043`	Negative	Insert sequence number exceeding uint64 max (`18446744073709551616`) or malformed string	SQL CHECK constraint rejects insert fail-closed	Specified — Pending Verification
SUITE-03	`PROMO-EVENT-EVIDENCE-LINK-017`	Positive	Promotion committed; verify event payload binds assessment_id, assessment_sha256, 4 claims, policy and evaluator versions	Complete audit evidence linkage immutably appended to stream	Specified — Pending Verification
SUITE-03	`DRAFT-INTEGRITY-REJECT-018`	Negative	Submit evaluation draft with mismatched offer_id, wrong sequence, or contradictory statuses	`validateEvaluationDraftIntegrity` throws `E-ERR-024` before persistence	Specified — Pending Verification

Suite ID	Test Case ID	Type	Action / Fixture	Expected Outcome	Status
SUITE-03	`NONCE-RETRY-POLICY-010`	Positive	Assessment fails due to price mismatch; retry with corrected price within 300s window	Nonce not burned on failure; retry succeeds and burns nonce atomically on commit	Specified — Pending Verification
SUITE-01	`JCS-STRICT-DOMAIN-003`	Negative	Submit payload containing `NaN`, `Infinity`, lone surrogate, or duplicate JSON key	Throws `E-ERR-022` fail-closed	Specified — Pending Verification
SUITE-02	`FIN-ROPF-PEAK-007`	Positive	Compute Marked-to-Market Return on Peak Funding with verified inventory valuation	Matches exact percentage; unverified valuation emits `DATA_REQUIRED`	Specified — Pending Verification
SUITE-02	`FIN-CONV-HPR-008`	Positive	Compute conventional HPR on isolated single position with starting value and dividend distributions	Returns exact holding-period return	Specified — Pending Verification
SUITE-03	`GATEWAY-SHARED-VALID-009`	Negative	Feed receipt with duration 300,001 ms, future clock skew > 300s, or expired timestamp to shared validator	Throws `E-ERR-027` or `E-ERR-007` identically in builder and gate	Specified — Pending Verification
SUITE-03	`PROD-NORM-DETERMINISTIC-004`	Positive	Feed 4, 6, and 8-digit HSN codes; check MPN casing preservation; reject duplicate dimension keys	Deterministic normalization produces exact expected core	Specified — Pending Verification
SUITE-03	`PROD-COLL-001`	Negative	Submit MPN without `manufacturer_id` or generic item without dimension key	Throws `E-ERR-013` or `E-ERR-009`	Specified — Pending Verification
SUITE-03	`PROD-NORM-SAFE-003`	Positive	Feed MPNs with distinct casing in case-sensitive namespace (`"BP-01"` vs `"bp-01"`)	Generated distinct `product_id` hashes without unsafe merge	Specified — Pending Verification
SUITE-04	`CLAIM-SIG-VALID-001`	Positive	Verify valid direct Ed25519 claim signature against KAT-05A	Verification returns true bitwise	Specified — Pending Verification
SUITE-04	`GATEWAY-VERIFY-KAT-002`	Positive	Feed KAT-05B through production gateway verification routine	Passes verification via explicit property omission	Specified — Pending Verification
SUITE-04	`GATEWAY-LIFETIME-EXACT-008`	Negative	Submit endorsement with `expires_at_utc - issued_at_utc = 300,001 ms` or `299,999 ms`	Throws `E-ERR-027: INVALID_GATEWAY_RECEIPT_LIFETIME`	Specified — Pending Verification
SUITE-04	`CLAIM-NONCE-REPLAY-003`	Negative	Submit gateway endorsement with previously consumed nonce	Throws `E-ERR-023: GATEWAY_ENDORSEMENT_NONCE_REPLAYED`	Specified — Pending Verification
SUITE-04	`CLAIM-BATCH-NONCE-FAIL-009`	Negative	Submit 4 claims where claims 1-2 pass but claim 3 fails	Assessment fails; asserts claims 1-2 nonces remain 100% UNCONSUMED	Specified — Pending Verification
SUITE-04	`CLAIM-SIG-TAMPER-004`	Negative	Tamper unit price parameter in signed claim	Throws `E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION`	Specified — Pending Verification
SUITE-04	`CLAIM-ROLE-TAMPER-007`	Negative	Tamper `claimed_role` in signed claim payload	Throws `E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION`	Specified — Pending Verification
SUITE-04	`CLAIM-REUSE-OFFER-005`	Negative	Replay valid claim signature on differing `offer_id`	Throws `E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION`	Specified — Pending Verification
SUITE-04	`CLAIM-REUSE-TYPE-006`	Negative	Replay valid price claim signature on stock availability claim	Throws `E-ERR-017: INVALID_OR_MISSING_ATTESTATION_AUTHENTICATION`	Specified — Pending Verification
SUITE-04	`CLAIM-SCHEMA-STRICT-007`	Negative	Submit claim containing unknown nested JSON fields	Throws `E-ERR-024: INVALID_CLAIM_PARAMETERS`	Specified — Pending Verification
SUITE-04	`AUTH-ROLE-002`	Negative	Submit attestation signed by principal lacking authorized role	Throws `E-ERR-018: INSUFFICIENT_ATTESTER_ROLE`	Specified — Pending Verification
SUITE-04	`AUTH-REVOKE-003`	Negative	Submit attestation timestamped after principal key revocation	Throws `E-ERR-019: OPERATOR_KEY_REVOKED`	Specified — Pending Verification
SUITE-04	`AUTH-CLAIM-ROLE-004`	Negative	Human operator principal attempts to assert `VERIFIED_EXTERNAL_ORACLE`	Throws `E-ERR-018: INSUFFICIENT_ATTESTER_ROLE`	Specified — Pending Verification

Suite ID	Test Case ID	Type	Action / Fixture	Expected Outcome	Status
SUITE-05	`STREAM-GEN-001`	Positive	Initialize stream; assert genesis head seq `-1` and all-zeros	Genesis head matches virtual convention	Specified — Pending Verification
SUITE-05	`STREAM-ERR-002`	Negative	Submit out-of-order sequence vs stale previous hash	Emits `CONCURRENCY_CONFLICT` before `HASH_CHAIN_INTEGRITY_FAILURE`	Specified — Pending Verification
SUITE-05	`CHECK-SIG-FAIL-001`	Negative	Tamper checkpoint payload or submit invalid Ed25519 signature	Throws `E-ERR-015: CHECKPOINT_SIGNATURE_VERIFICATION_FAILED`	Specified — Pending Verification
SUITE-05	`CHECK-REVOKE-002`	Negative	Submit checkpoint signed with revoked trust anchor key	Throws `E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED`	Specified — Pending Verification
SUITE-05	`CHECK-COMPROMISE-FAIL-004`	Negative	Submit historical checkpoint signed by compromised trust anchor	Throws `E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED` fail-closed	Specified — Pending Verification
SUITE-05	`CHECK-OPERATIONAL-EXPIRED-007`	Negative	Submit checkpoint signed by anchor whose validity window expired prior to `current_time_utc` in `CURRENT_OPERATIONAL` mode	Throws `E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED`	Specified — Pending Verification
SUITE-05	`CHECK-HISTORICAL-VALID-008`	Positive	Submit historical checkpoint signed by anchor valid at signing time, in `HISTORICAL_AUDIT` mode	Passes verification bitwise	Specified — Pending Verification
SUITE-05	`CHECK-DATE-PARSE-006`	Negative	Submit trust anchor with malformed date `"2026-02-30"` or `valid_from > valid_until`	Throws `E-ERR-011: TRUST_ANCHOR_INVALID_OR_REVOKED` fail-closed	Specified — Pending Verification
SUITE-05	`CHECK-FUTURE-FAIL-005`	Negative	Submit checkpoint timestamped $> 300\text{s}$ into future	Throws `E-ERR-007: CLOCK_SKEW_EXCEEDED`	Specified — Pending Verification
SUITE-05	`CHECK-VERIFY-EXEC-003`	Positive	Verify valid checkpoint against `verifyStreamCheckpointReceipt()`	Returns true bitwise	Specified — Pending Verification
SUITE-06	`ACTUALS-GUARD-001`	Negative	Invoke actuals ledger mutation with simulation context	Throws `E-ERR-020: ACTUALS_LEDGER_WRITE_VIOLATION`	Specified — Pending Verification
SUITE-06	`SIM-GUARD-002`	Negative	Invoke simulation ledger mutation with actuals context	Throws `E-ERR-025: SIMULATION_LEDGER_WRITE_VIOLATION`	Specified — Pending Verification
SUITE-06	`ACTUALS-AUTH-003`	Negative	Invoke actuals ledger mutation with expired or forged auth ticket	Throws `E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT`	Specified — Pending Verification
SUITE-06	`ACTUALS-AUTH-EXACT-004`	Negative	Tamper mutation amount or debit account relative to ticket `tx_mutation_sha256`	Throws `E-ERR-026: ACTUALS_LEDGER_UNAUTHORIZED_CONTEXT`	Specified — Pending Verification
SUITE-06	`ACTUALS-TX-ATOMIC-005`	Negative	Inject database failure during actuals mutation	Transaction rolls back completely; nonce remains unconsumed	Specified — Pending Verification
SUITE-06	`TEST-CONCURRENCY-NONCE-01`	Negative	Submit two concurrent requests with identical ticket nonce	Exactly one succeeds; second throws `E-ERR-023: GATEWAY_ENDORSEMENT_NONCE_REPLAYED`	Specified — Pending Verification

8.2 Five Pre-Migration Regression Suites (Pinned Baseline)

Suite ID	Test Case ID	Type	Action / Fixture	Expected Outcome	Status
SUITE-07	`REF-MATH-001`	Positive	Run independent SS304 clamp landed cost fixture	Matches independent spreadsheet to 4 decimal places	Specified — Pending Verification
SUITE-07	`VOL-SAFEINT-001`	Positive	Feed $300 \times 200 \times 100\text{ mm}$ box into safe-integer calculator	Exactly 1,200 grams (KAT-03 match)	Specified — Pending Verification
SUITE-07	`VOL-BOUND-002`	Negative	Feed box with length $15,000\text{ mm}$ (> 10m bound)	Throws `E-ERR-021: DIMENSION_BOUNDS_EXCEEDED`	Specified — Pending Verification
SUITE-07	`VOL-ROUND-HALFUP-003`	Positive	Feed dimensions yielding remainder exactly 2500	Rounds up to quotient + 1 (configured round-half-up)	Specified — Pending Verification
SUITE-07	`FUND-INTRADAY-001`	Positive	Inject morning outflow -₹10,000, evening inflow +₹10,000 on Day 1	Reports $C_{\text{peak, ops}} = 10,000$, $C_{\text{peak, daily_net_ops}} = 0$	Specified — Pending Verification
SUITE-07	`REC-OUTLAY-001`	Positive	Pre-event baseline $B_{\text{ops}}(0) \equiv 0$ at t_0 (Day 0); $j=1$ advance $+500 \implies B(1)=+500$; $j=2$ outlay $-1,500 \implies B(2)=-1,000$ ($k_{\text{outlay}}=2$, Day 10); $j=3$ recovery $+1,200 \implies B(3)=+200$ ($k_{\text{rec}}=3$, Day 25)	Reports Outlay Recovery = 15 days, Lifecycle Recovery = 25 days, $C_{\text{peak, ops}} = 1,000$	Specified — Pending Verification
SUITE-07	`FIN-CROSS-UNRECEIPTED-002`	Negative	Attempt multi-currency balance consolidation without `FXConversionReceipt`	Throws `E-ERR-008: CROSS_CURRENCY_SUMMATION_ILLEGAL`	Specified — Pending Verification
SUITE-07	`FIN-PO-NETTING-003`	Positive	Lifecycle: PO $100k issued, $20k cancelled, $50k fulfilled	Committed capital nets strictly to $30k without double-counting	Specified — Pending Verification
SUITE-07	`FIN-PO-REPLACED-006`	Positive	Lifecycle: PO $100k replaced by new PO $80k	Prior commitment relieved ($U_{\text{po}} \to 0$); new commitment $80k recorded	Specified — Pending Verification
SUITE-07	`FIN-REALIZED-RETURN-004`	Positive	Calculate realized return for open position with unsold inventory	Reports cash-based CoC; unsold inventory valuation excluded from realized cash metric	Specified — Pending Verification
SUITE-07	`FIN-HPR-VALUATION-005`	Negative	Attempt Marked-to-Market Return on Peak Funding calculation when inventory valuation is unverified	Emits `status: 'DATA_REQUIRED'` (valuation not coerced to zero)	Specified — Pending Verification
SUITE-08	`PROP-INV-001`	Positive	Fuzz 1,000 inventory state transitions across 7 buckets	$\sum \text{Opening} + \text{In} - \text{Out} \equiv \sum \text{Closing}$ holds bitwise	Specified — Pending Verification
SUITE-08	`INV-QTY-BOUND-002`	Negative	Submit inventory bucket with negative quantity or $> 10^9$	Throws `E-ERR-034: INVALID_INVENTORY_QUANTITY`	Specified — Pending Verification
SUITE-09	`META-FEE-001`	Positive	Monotonically increase Amazon closing fee across ₹5 steps	Net contribution strictly non-increasing	Specified — Pending Verification
SUITE-10	`HIST-REPLAY-001`	Positive	Replay `AIStudio` commit `0c67e26` March 2026 fixture (canonical SS304 pack-of-10 clamp, landed cost ₹38.21)	Identical decision recommendation generated	Specified — Pending Verification
SUITE-11	`FUZZ-CLOCK-001`	Negative	Inject real-time capture with $+600\text{s}$ clock skew	Throws `E-ERR-007: CLOCK_SKEW_EXCEEDED`	Specified — Pending Verification

9. Gate M1 Auditable Acceptance Checklist

Using the auditable three-state convention:

- **`Specified — Pending Verification`** : Normative contract exists, but the required implementation-level test has not run.
- **`Design Reviewed`** : Logic, formulas, schemas, and error boundaries have been audited and verified without claiming runtime test suite execution.
- **`Verified — Test Evidence Recorded`** : Designated test fixture or cryptographic vector was independently verified with reproducible results retained.

Acceptance Criterion	Status	Verification Evidence / Design Closure
Evidence cannot be promoted illegally	**Design Reviewed**	18-tuple Allowlist + `validateEvidenceAllowlistEligibility()` + Two-Phase `evaluateOfferPromotionReadOnly()` + atomic `commitPromotionIfVersionAndConsumeNonces()` (4 claims, source freshness, field reconciliation, atomic batch nonces, exact 300s receipt expiry)
Financial calculations unambiguous	**Design Reviewed**	Two-Tier Gross-Clearing Accounting Model (§7.1, AUDIT-44), separate $C_{\text{peak, ops}}$ vs $C_{\text{peak, daily_net_ops}}$ strictly invariant to initial owner capital B_0 (AUDIT-48), deterministic composite ordering key `(entry_timestamp_utc, sequence_number)`, dual recovery metrics, distinct Realized Return vs Marked-to-Market Return on Peak Funding, PO replacement netting formula
Serialization is reproducible	**Design Reviewed**	Pinned `json-canonicalize@3.0.1` registry metadata and SHA-512 integrity (§6.1, AUDIT-45); reference IETF test corpus commit `19d51d7fe467d4706a3ff08adf8a748f29fc21e0`; duplicate key rejection; KAT-01, KAT-02, KAT-05A/B (with bound role and exact 300s expiration); RFC 8785 full-corpus status maintained strictly as `Specified — Pending Verification`
FX arithmetic preserves exactness	**Verified — Test Evidence Recorded**	Independently verified BigInt rational integer arithmetic against KAT-04 (Parts A & B): produces `865086`, `865260`, and `779122735535095894` bitwise; exposes ₹0.0086 Float64 drift; point-of-use FX receipt validation
Volumetric weight mathematically correct	**Verified — Test Evidence Recorded**	Independently verified safe-integer formula proof against KAT-03 ($300\times200\times100\text{ mm} = 1{,}200\text{ g}$ against configured divisor $5{,}000\text{ cm}^3/\text{kg}$); bounds $[1, 10000]\text{ mm}$
Event history is auditable	**Design Reviewed**	Virtual genesis head (seq `-1`, H_{-1}), prior-commit pointer, pinned Ed25519 signature verification routine `verifyStreamCheckpointReceipt()` with operational vs historical modes and compromise check
Money remains exact	**Design Reviewed**	Decoupled 3-tier scales, fixed scales, signed 128-bit decimal string validation, direct FX quote conventions, runtime constructor `createCanonicalMoney()`
Deterministic product identity normalization	**Design Reviewed**	Safe identity normalization `normalizeProductIdentityCore()`, namespace casing preservation, non-duplicate dimension keys, safe inventory quantities. HSN syntax check explicitly scoped as format validation, not legal assignment (§2.2, AUDIT-40)
Incomplete information explicit	**Design Reviewed**	First-class `DATA_REQUIRED`, `UNKNOWN`, and `NOT_YET_RECOVERED` states; zero coercion prohibited
Real & paper ledgers isolated	**Design Reviewed**	Bidirectional guard (`E-ERR-020` / `E-ERR-025`) + cryptographic gateway ticket verification binding exact mutation digest (`E-ERR-026`) + single ACID database transaction boundary
Migration remains controlled	**Design Reviewed**	Explicit artifact-by-artifact disposition matrix; reference-only boundaries preserved
Acceptance test suite execution	**Specified — Pending Verification**	11 suites defined with explicit test IDs, pending Gate M1 authorization for test execution
Legacy core migration	**Specified — Pending Verification**	Blocked until Gate M1 passes and Gate M2 is separately authorized